PYTHON · PYTEST · GEMINI CLI · VS CODE | 2일 과정 강의 교안

Coding Agent 실전:

AI기반 TDD 마스터 과정

AI에게 코드를 맡길수록, 테스트라는 안전망이 더 필요하다

L1 → L2 | 요구사항 → 테스트 → RED → AI 구현 → GREEN | 회귀 안전망

2일 · 하루 08:30–17:30 (점심 12:00–13:00) · 총 16시간 · 대상: TDD를 AI로 해 보고 싶은 개발자

2026 · 김대경 Ph.D.

설명 · 듣고 이해 COURSE GOAL

이 과정의 목표 — L1에서 L2로

L1 · 교육 전

TDD를 어렴풋이 안다

- "테스트를 먼저 쓴다더라"
- AI에게 코드를 바로 시켜 본 적은 있다
- 요구사항이 바뀌면 눈으로 다시 확인한다

→

L2 · 교육 후

AI와 함께 TDD 사이클을 직접 돈다

- 요구사항 한 줄 → 실패하는 테스트
- AI에게 '이 테스트만 통과'로 범위를 정해 맡긴다
- 요구사항이 바뀌면 기존 테스트로 회귀를 확인한다

이틀 동안 몸에 익힐 한 바퀴

요구사항 | 테스트 작성 | RED | AI 구현 | GREEN | 리팩터링 | 회귀 확인

원칙 "AI가 코드를 빨리 만들기 때문에 오히려 테스트라는 안전망이 필요하다"

설명 · 듣고 이해 TIMETABLE · Day 1

Day 1 시간표 — 왜 AI에 TDD가 필요한가

08:30–17:30 · 점심 12:00–13:00 · 50분 수업 · 10분 휴식

시간	내용	방식
08:30–09:00	오리엔테이션 · 환경 확인	체크리스트
09:00–09:50	실험 A — 테스트 없이 AI에게 장바구니 시키기 (A-1 ~ A-4)	개인 실습
10:00–10:50	기록표 공유 · 실험 A 확장 (A-5 ~ A-9) · Python 테스트 기초	짝 검토 + 실습
11:00–11:50	1장 테스트 개념과 원리 — 바이브 코딩을 넘어서	강의
13:00–13:50	2장 SW 테스팅과 단위 테스트 설계 (AAA · Given-When-Then) · pytest 기초 · 강사 시연	강의 + 시연
14:00–14:50	실습 1 — 문자열 덧셈 계산기 (RED → GREEN 4회)	개인 실습
15:00–15:50	실습 2 — 요구사항이 바뀔 때 (회귀 체험)	개인 실습
16:00–16:50	좋은 테스트의 모양 (AAA · 리뷰) · 점검 실습 · 골든 마스터	강의 + 실습
17:00–17:30	Day 1 회고 (17:00–17:15) · 복습 퀴즈 (17:15–17:30)	회고 + 퀴즈

방식 색: ■ 실습 · 짝 검토 ■ 시연 ■ 퀴즈 · 회고 ■ 강의 점심 12:00–13:00

설명 · 듣고 이해 TIMETABLE · Day 2

Day 2 시간표 — TDD 사용 실습

08:30–17:30 · 점심 12:00–13:00 · 50분 수업 · 10분 휴식

시간	내용	방식
08:30–09:20	애매한 요구사항에서 사양 찾기 — 필요성 · 질문 · 예시 표	짝 검토
09:30–10:20	장바구니 TDD 1 — 합계 · 문턱 할인 · 경계값	개인 실습
10:30–11:00	AI가 테스트를 고치지 못하게 — GEMINI.md	시연 + 따라하기
11:10–12:00	장바구니 TDD 2 — VIP 할인 · 불변식 · 음수 오류 · 골든 마스터	개인 실습
13:00–13:50	3장 클린 코드와 리팩토링 (디자인 패턴 포함)	강의
14:00–14:50	GREEN 상태에서 리팩터링 · 실험 B	짝 검토 + 비교
15:00–16:20	확인 기준 실습 — 배송비 계산기 (평가: 계약표 · 플랜 · TDD · 골든 마스터 · 전략 패턴)	개인 실습
16:20–16:50	확인 실습 짝 검토 · 체크리스트 점검	짝 검토
17:00–17:30	다음 단계 맛보기 · 결과 공유 · KPT 회고	강의 + 회고

방식 색: ■ 실습 · 짝 검토 ■ 시연 ■ 퀴즈 · 회고 ■ 강의 점심 12:00–13:00

설명 · 듣고 이해 CERTIFICATION

확인 사항— 이 다섯 가지를 할 수 있으면 L2

Day 2 15:00 확인 실습(배송비 계산기)에서 확인. 5개 중 4개 이상, ①과 ④는 필수. (스스로 점검: 체크리스트 슬라이드 217)

①	**요구사항 한 줄을 실패하는 테스트로 바꾼다**	RED 캡처 2장 이상	필수
②	**AI에게 범위를 정해 요청한다**	"이 테스트만 통과" · AI가 테스트를 고치지 않음	
③	**경계값 양쪽을 테스트한다**	29,999 / 30,000 처럼 짝으로	
④	**요구사항 변경 후 회귀를 확인한다**	기존 테스트가 남아 있고 모두 통과	필수
⑤	**GREEN 상태에서만 리팩터링한다**	리팩터링 전후 전체 테스트 통과 기록	

실습 · 직접 하기 SETUP

제공 폴더 C:\DEV\tdd-course
강사 배포 starter 를 C:\DEV\tdd-course 로 복사해 둔다 (사전 과제)

실습 환경과 폴더 구조 — 사전 과제로 끝내 둔다

설치 확인 (Windows는 Git Bash 또는 PowerShell)

터미널

```
> python --version
Python 3.11.9
> python -m venv .venv
> .venv\Scripts\activate
> python -m pip install pytest
> pytest --version
pytest 8.3.3
> npm install -g @google/gemini-cli
> gemini --version
0.x.x            # 버전 숫자는 설치 시점에 따라 다름
```

모든 실습 폴더에 같은 pytest.ini

```
[pytest]
pythonpath = src      # src/ 안의 모듈을 바로 import
testpaths = tests
```

과정 폴더 (VS Code로 starter 열기)

```
starter /
├─ cart-a/         # Day1 실험 A (테스트 없음)
│  └─ cart.py
├─ calc/           # Day1 실습 1·2
│  ├─ pytest.ini
│  ├─ src/calculator.py
│  └─ tests/test_calculator.py
├─ cart/           # Day2 장바구니 + 실험 B
│  ├─ GEMINI.md    # Day2 에 만듦
│  ├─ pytest.ini
│  ├─ src/cart.py
│  └─ tests/test_cart.py
└─ ship/           # Day2 확인 실습
```

참고 Gemini CLI는 Google 계정으로 로그인(첫 실행 시 브라우저). 실습 폴더마다 그 폴더에서 gemini를 실행

지금 할 일 사전 과제 · 강사 배포 starter 를 C:\DEV\tdd-course 로 복사 → VS Code 로 열기 ▸ 아래 설치 확인 명령 실행 (python · pytest · gemini)

이 과정에서 쓰는 Gemini CLI 기능 여섯 가지

입력	하는 일	예
`gemini`	대화형 세션 — 자연어로 지시 (모든 ARRR 작업)	`gemini`
`gemini -p "…"`	단발 실행 후 종료 — 스크립트·파이프라인용	`gemini -p "pytest 실패 요약"`
`@경로`	파일/폴더를 컨텍스트에 정확히 지정	`@src/cart.py 설명해줘`
`!명령`	세션 안에서 셸 명령 직접 실행	`!pytest -q`
`/명령`	내장·커스텀 슬래시 명령 (/model · /mcp · /memory)	`/tdd-red`

승인 모드 (--approval-mode)	plan	default	auto_edit	yolo
동작	읽기 전용 — 분석·계획만	매번 승인 (권장 시작점)	파일 편집만 자동 승인	전부 자동 (격리 저장소에서만)

VS Code 화면 배치

- 왼쪽: 테스트 파일 · 오른쪽: 구현 파일
- 아래 터미널 2개: Gemini 세션 / pytest 실행

승인 모드

- 기본(default): 파일 수정 전마다 승인을 묻는다 — 이 과정은 항상 기본 모드
- 변경 내용(diff)을 읽고 승인하는 것도 실습

설명 · 듣고 이해 PROMPT FORM

이 과정의 프롬프트 기본형

1 대상
@파일로 무엇을 보고 할지

2 할 일 하나
"이 테스트만 통과"처럼 한 가지

3 금지
"테스트 파일은 수정하지 마"

4 확인
"끝나면 !pytest 결과를 보여줘"

✗ 나쁜 예 — 범위가 없다

▶ 프롬프트

덧셈 계산기 만들어줘. 테스트도 같이 짜주고 잘 돌아가게 해줘.

✓ 좋은 예 — 대상·할 일·금지·확인

▶ 프롬프트

@tests/test_calculator.py @src/calculator.py
test_empty_string_returns_zero 만 통과하는 최소 구현을 해줘. 테스트 파일은 수정하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

- 나쁜 예는 AI가 테스트와 구현을 함께 만들어, 테스트가 코드를 '확인'하지 못하고 코드를 따라가게 됨.
- 좋은 예는 사람이 쓴 테스트가 기준이고, AI는 그 기준을 통과하는 일만 함.

참고 이 교안의 모든 프롬프트는 이 네 줄 형식을 따른다. 결과 예시는 실제로 실행한 코드와 pytest 출력, Gemini의 결과는 실행마다 다를 수 있다.

설명 · 듣고 이해 HOW TO READ

이 교안 보는 법 — 설명 · 시연 · 실습을 색으로 구분

슬라이드 왼쪽 띠의 색과 제목 위 배지로 '지금 듣는 시간인지, 직접 하는 시간인지'를 바로 알 수 있어야 한다.

설명

듣고 이해한다

개념 · 원리 · 결과 해설을 듣고 이해

강사 시연

강사 화면을 본다

강사 화면을 참고해, 따라 하지 않고, 같은 내용을 실습

실습

직접 한다

하단 초록 띠 '지금 할 일'을 순서대로 직접 진행

토론

모둠 · 전체 활동

기록표 확인 등 동료 검토

실습 슬라이드의 하단 띠 — 이 줄만 따라하면 됨

▶ **프롬프트** 파란 테두리 상자 = Gemini 에 그대로 붙여 넣을 프롬프트

지금 할 일 VS Code: 테스트 작성 ▸ 터미널: pytest → RED 확인 ▸ Gemini: 프롬프트 ▸ 터미널: GREEN 확인

실습 블록 앞에는 '실습 안내' 슬라이드

- 폴더 · 시간 · 방식 · 결과물
- 단계별 '어디서 · 무엇을 · 완료 기준 · 슬라이드 번호' 표
- 프롬프트는 각 폴더의 prompts.md 에서 복사

화면 배치 (권장)

- VS Code 편집기: 테스트 · 코드 · prompts.md
- 터미널 ① gemini · 터미널 ② pytest
- 슬라이드는 다른 화면을 통해 참고

D1

DAY 1

왜 AI에 TDD가 필요한가

- ✓ 먼저 테스트 없이 AI에게 기능 구현을 요청해 보고, 그 과정에서 어떤 문제와 확인이 필요 요소가 발생하는지 확인.
- ✓ 이후 간단한 계산기 예제를 통해 실패하는 테스트 작성 → 기능 구현 → 테스트 통과 → 코드 개선의 과정을 직접 실습
- ✓ 오후에는 새로운 요구사항을 추가하고 기존 요구사항을 변경해 보면서, 기존 테스트가 기능 변경 과정에서 발생할 수 있는 오류를 어떻게 발견하고 기존 기능을 보호하는지 직접 경험.

제공 폴더 C:\DEV\tdd-course\calc
starter\calc 를 복사한 폴더 · 프롬프트 calc\prompts.md

08:30–09:00

환경 확인

폴더

calc

터미널

```
> cd C:\DEV\tdd-course\calc
> python --version
Python 3.11.9
> pytest -q
no tests ran in 0.01s        # 테스트가 아직 없으니 정상
> gemini
(로그인 후 대화창이 열리면 성공)
```

- [ ] python --version이 3.11 이상
- [ ] pytest -q가 오류 없이 끝난다 ('no tests ran'은 정상)
- [ ] gemini가 열리고 Google 계정으로 로그인됐다
- [ ] VS Code에서 C:\DEV\tdd-course\calc 폴더가 열려 있다
- [ ] 터미널을 2개 띄웠다 (Gemini / pytest)

첫 대화 — 세션이 폴더를 보고 있는지 확인

▶ 프롬프트

이 폴더에 어떤 파일과 폴더가 있는지 목록만 알려줘. 아무것도 수정하지 마.

결과 예시

✦ 현재 폴더(calc) 구성이다.
pytest.ini · src/ (비어 있음) · tests/ (비어 있음)

TIP 이 단계에서 막힌 사람은 짝에게 화면을 보여 주고 함께 해결. 09:00까지 모두 '첫 대화'를 끝내야 실험 A를 같이 시작 가능.

제공 폴더 C:\DEV\tdd-course\cart-a
starter\cart-a 복사 (prompts.md 만 있다 · AI 가 코드를 만든다)

09:00–09:50

실험 A —진행 방식(테스트 없이 AI에게 맡기기)

폴더
cart-a/

방식
개인

결과물
실험 A 기록표 4줄
(결과 · 확인 방법 · 확인이 필요했던 점)

프롬프트
cart-a/prompts.md

#	어디서	할 일	완료 기준	슬라이드
1	터미널 ①	cd cart-a → gemini 실행	대화창이 열림	13
2	Gemini	A-1 프롬프트 붙여넣기 → 변경 승인	cart.py 생성	14
3	터미널 ②	python -c 로 결과를 눈으로 확인	60000 출력	14
4	기록표	결과 · 확인 방법 · 확인이 필요했던 점	한 줄 작성	14
5	반복	A-2 → A-3 → A-4 를 같은 순서로	기록표 4줄	15–17
6	터미널 ②	정확히 50,000원 장바구니를 넣어 보기	결과를 기록표에	18

TIP 규칙은 '테스트 파일 금지'. AI 결과가 슬라이드와 달라도 정상 — AI가 만든 결과가 맞는지 어떻게 확인했는지 기록할 것.

실습 · 직접 하기 Day 1 · 1교시 | 실험 A

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

09:00–09:50

실험 A — 테스트 없이 AI에게 작업 지시

규칙: 테스트 파일 금지 · 결과는 눈으로만 확인 · 단계마다 기록표 작성 · 폴더는 cart-a/

A-1 합계 | **A-2 5만 원 이상 10% 할인** | **A-3 VIP 5% 추가 할인** | **A-4 코드 정리**

실험 A 기록표 (각자 종이 또는 메모장)

단계	요청 내용	어떻게 확인했나	다시 요청	확인이 필요했던 점
A-1	합계			
A-2	5만 원 이상 10% 할인			
A-3	VIP 5% 추가 할인			
A-4	코드 정리			

참고 장바구니 상품은 {"price": 가격, "qty": 수량} 딕셔너리의 리스트. 예) 12,000원 × 3개 + 24,000원 × 1개 = 60,000원

원칙 이 실습은 평소대로 AI를 쓰면서, 매 단계 '정말 맞나?'를 어떻게 확인했는지 점검하는 것을 목적으로 함.

지금 할 일 cart-a/ 에서 gemini 실행 → A-1 ~ A-4 를 순서대로 · 단계마다 기록표 한 줄 · 테스트 파일은 만들지 않는다

실습 · 직접 하기 Day 1 · 1교시 | 실험 A · A-1

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-1 · 합계 — 결과는 그럴듯하다

▶ 프롬프트 — Gemini CLI

cart.py 에 장바구니 합계를 계산하는 total(items) 함수를 만들어줘. items 는 {"price": 가격, "qty": 수량} 딕셔너리의 리스트야.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI — cart-a/cart.py

```
✦ cart.py에 total 함수를 만들었다.
+ def total(items):
+     return sum(item["price"] * item["qty"] for item in items)
```

눈으로 확인 (python 한 줄 실행)

터미널

```
> python -c "from cart import total;
print(total([{'price': 12000, 'qty': 3}, {'price':
24000, 'qty': 1}]))“


60000
```

60,000원이 나왔다 → 맞는 것 같다. 기록표: '예제 하나를 눈으로 확인'

실습 · 직접 하기 Day 1 · 1교시 | 실험 A · A-2

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-2 · 5만 원 이상 10% 할인 — 숫자 모양이 바뀌었다

▶ 프롬프트 — Gemini CLI

합계가 50,000원 이상이면 10% 할인해줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI — cart-a/cart.py

```
✦ total 함수에 할인 조건을 추가했다.
+ def total(items):
+     amount = sum(item["price"] * item["qty"] for item in items)
+     if amount >= 50000:
+         amount = amount * 0.9
+     return amount
```

눈으로 확인 (python 한 줄 실행)

터미널

```
> python -c "from cart import total;
print(total([{'price': 12000, 'qty': 3}, {'price':
24000, 'qty': 1}]))“


54000.0
```

54,000이 맞긴 한데 '54000.0'으로 소수점이 붙었다.
괜찮은 걸까? → 기록표 '확인이 필요했던 점'에 적는다

주의 실수(float) 곱셈 때문에 금액이 소수가 됐다. 아무도 요구하지 않았지만 AI가 정한 결정 — 이런 결정을 누가 확인하나요?



지금 할 일 Gemini: A-2 프롬프트 ▸ 터미널: 같은 장바구니로 다시 확인 ▸ 기록표: 결과 · 확인 방법 · 확인이 필요했던 점

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-3 · VIP 5% 추가 할인 — 이번에도 그럴듯하다

▶ 프롬프트 — Gemini CLI

VIP 고객은 할인된 금액에서 5%를 추가로 할인해줘. total 에 is_vip 인자를 추가해.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI — cart-a/cart.py

```
✦ is_vip 매개변수를 추가했다.
+ def total(items, is_vip=False):
+     amount = sum(item["price"] * item["qty"] for item in items)
+     if amount >= 50000:
+         amount = amount * 0.9
+     if is_vip:
+         amount = amount * 0.95
+     return amount
```

눈으로 확인 (python 한 줄 실행)

터미널

```
> python -c "from cart import total;
print(total([{'price': 12000, 'qty': 3}, {'price':
24000, 'qty': 1}], is_vip=True))"


51300.0
```

60,000 → 54,000 → 51,300. 계산기로 확인하니 맞다. 이제 VIP가 아닌 일반 고객도 이전처럼 제대로 계산되는지 확인해 보세요. VIP 기능 추가 후에도 일반 고객 할인 금액 54,000원이 정상적으로 계산됨.

실습 · 직접 하기 Day 1 · 1교시 | 실험 A · A-4

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-4 · 코드 정리 — 겉보기엔 더 깔끔해졌다

▶ 프롬프트 — Gemini CLI

코드가 좀 지저분하네. 할인율을 상수로 빼고 50000원 이하는 할인 안 되게 해. 깔끔하게 정리해줘. 금액은 정수로 나오게 해줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI — cart-a/cart.py

```
✦ 상수를 분리하고 정리했다. 결과는 반올림한 정수이다.
+ THRESHOLD = 50000
+ THRESHOLD_RATE = 0.9
+ VIP_RATE = 0.95
+
+
+ def total(items, is_vip=False):
+     amount = sum(i["price"] * i["qty"] for i in items)
+     if amount > THRESHOLD:          # '50000원 이하는 할인 안 됨' → >
+         amount *= THRESHOLD_RATE
+     if is_vip:
+         amount *= VIP_RATE
+     return round(amount)
```

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import total;
print(total([{'price': 12000, 'qty': 3}, {'price':
24000, 'qty': 1}], is_vip=True))"

51300
```

51,300 — 소수점도 사라지고 깔끔하다. 대부분 여기서 '완성!'이라고 생각한다. (프롬프트의 '50000원 이하'는 정확히 50,000원을 포함한다 — 다음 장에서 확인)

지금 할 일 Gemini: A-4 프롬프트 ▸ 터미널: 확인 ▸ 기록표 작성 — 여기까지 끝나야 함.

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

정말 괜찮을까? — 정확히 50,000원을 넣어 보면

A-2 때의 코드 예시

```
    if amount >= 50000:          # 이상
```

A-4 '정리' 후의 코드 예시

```
    if amount > THRESHOLD:       # 초과
```

터미널

```
> python -c "from cart import total; print(total([{'price':25000,'qty':2}]))"
50000              # 기대값 45000 — 5만 원 손님이 할인을 못 받는 경우가 생길 수 있다.
```

위 예시를 통해, 무슨 일이 있었나 확인해야 함.

- "50000원 이하는 할인 안 되게"라는 한 문장이 >= 를 > 로 바꿨다 (A-2 는 '이상' — 서로 충돌)
- 60,000원 예제만 확인했으니 아무도 몰랐다
- A-1~A-3 결과를 다시 확인한 사람은 거의 없다

질문

- AI가 이렇게 빨리 고치는데, 기존 기능이 안 깨졌다는 것을 어떻게 확인할까요?
- 확인을 매번 손으로 할 수 있을까요?

TIP 경계값 50,000원을 직접 넣어서 기존 기능이 유지되는지 확인.

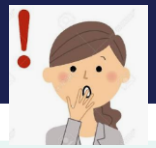
지금 할 일 터미널(cart-a/): 아래 명령으로 정확히 50,000원을 넣어 보기 → 결과를 기록표에 적기

토론 · 짝 활동 Day 1 · 2교시 | 토론 10:00–10:50

기록표 공유 — 우리는 무엇으로 '맞다'고 판단했나

단계	자주 나오는 확인 방법	놓친 것	필요한 것
A-1	예제 하나를 python으로 실행	빈 장바구니처럼 다른 경우는 확인하지 못함.	여러 경우를 한 번에
A-2	같은 예제를 다시 실행	정확히 5만 원은? 소수점은?	경계값 확인
A-3	계산기로 검산	VIP 기능을 추가한 뒤 일반 고객 할인 A-2 경우는 여전히 맞나?	이전 확인을 자동 반복
A-4	"결과가 그대로네"	프롬프트의 '50000원 이하'가 경계를 >= 에서 > 로 바꿈. 정확히 50,000원을 넣어 보기 전에는 발견하지 못할 수 있음	바뀌면 바로 알려 주는 장치

지금까지 눈으로 확인한 여러 계산 사례를 코드로 저장해 두고, 코드를 수정할 때마다 한 번에 다시 실행. 이것이 테스트!!!.



Q: A에서 가장 확인이 필요했던 순간은 언제였나요? 그때 무엇이 있었다면 안심했을까요?

지금 할 일 짝검토 → 가장 확인이 필요했던 순간 한 가지씩 공유

실습 · 직접 하기 Day 1 · 2교시 | 실험 A · 복습

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-5. 빈 장바구니 처리

▶ 프롬프트 — Gemini CLI

빈 장바구니의 합계는 `0`이 되도록 하세요.

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import total; print(total([]))"  # 예상: 0
```

A-6. 할인 경계값 확인

▶ 프롬프트 — Gemini CLI

49,999원에는 할인이 적용되지 않고, 50,000원부터 적용되는지 확인하세요.

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import total; print(total([{'price':49999,'qty':1}])); print(total([{'price':50000,'qty':1}]))"  # 예상: 49999, 45000
```

지금 할 일 현재 코드를 기준으로 독립적으로 연습할 수 있는 과제 5개. 각 과제를 구현한 뒤 프로젝트 폴더에서 한 줄 명령으로 결과를 확인한다.

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-7. 정액 쿠폰 추가

▶ 프롬프트 — Gemini CLI

`coupon` 인자를 추가하고, 모든 할인 적용 후 쿠폰 금액을 차감하세요. 최종 금액은 0원 미만이 되면 안 된다.

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import total; print(total([{'price':50000,'qty':1}], coupon=5000)); print(total([{'price':1000,'qty':1}],
coupon=5000))"  # 예상: 40000, 0
```

A-8. 잘못된 상품 검증

▶ 프롬프트 — Gemini CLI

가격이 음수이거나 수량이 1보다 작으면 `ValueError`가 발생하도록 하세요.

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import total; total([{'price':-1000,'qty':1}])"  # 예상: ValueError
```

지금 할 일 현재 코드를 기준으로 독립적으로 연습할 수 있는 과제 5개. 각 과제를 구현한 뒤 프로젝트 폴더에서 한 줄 명령으로 결과를 확인한다.

이어서 사용 C:\DEV\tdd-course\cart-a
위에서 연 폴더 그대로 · 프롬프트 cart-a\prompts.md

A-9. 할인 내역 제공

▶ 프롬프트 — Gemini CLI

최종 금액뿐 아니라 원래 합계와 할인 금액도 확인할 수 있는 `summary(items, is_vip=False)` 함수를 만들어 보세요. 반환 형식은 딕셔너리로 한다.

눈으로 확인 (python 한 줄 실행)

터미널

```
>
python -c "from cart import summary; print(summary([{'price':50000,'qty':1}], True))"  # 예상: {'subtotal': 50000, 'discount': 7250, 'total': 42750}
```

Python 테스트 기초 — 눈으로 하던 확인을 파일로 남긴다

방금 python -c 로 한 확인을 test_ 함수로 옮기면, 코드를 바꿀 때마다 pytest 한 줄로 전부 다시 확인할 수 있다.

지금까지 — 터미널에서 눈으로

```
> python -c "from cart import total;
  print(total([{'price':10000,'qty':2}]))"
20000
```

실행 — 저장해 둔 확인을 한 번에

```
> pytest -q
..  [100%]
2 passed in 0.00s
```

pytest.ini

```
[pytest]
pythonpath = src
testpaths = tests
```

앞으로 — cart-a/tests/test_cart.py 에 저장

```
from cart import total


def test_empty_cart_is_zero():
    assert total([]) == 0


def test_total_below_threshold():
    items = [{"price": 10000, "qty": 2}]
    assert total(items) == 20000
```

규칙	의미
파일 이름	test_*.py — pytest 가 자동으로 찾는다
함수 이름	test_ 로 시작 — 하나가 확인 하나
assert	조건이 거짓이면 그 테스트만 실패(F)
pytest -q	점(.) = 통과 · F = 실패 · 마지막 줄 = 요약

핵심 테스트 = '확인 방법'을 코드로 저장한 것. 실험 A 의 기록표 '어떻게 확인했나' 열이 그대로 테스트가 된다. 자세한 규칙은 4교시 'pytest 기초'에서.

01

CHAPTER 1 · 이론

테스트 개념과 원리

왜 테스트를 먼저 두는가? Gemini CLI · Python 3.11+ · pytest · Git

⏱ 11:00–11:50 · 1장 이론

문제의식 — 바이브 코딩과 C2C 단절

바이브 코딩 (Vibe Coding)

AI에게 즉흥적 요청을 반복해 기능을 얻어내는 방식. 동작은 하지만 "이 코드가 어느 요구·개념에서 나왔는가"를 역추적할 수 없다 = C2C 단절.

C2C(Concept to Code)
아이디어와 요구사항을 검증 가능한 조건으로 구체화하고, 테스트와 구현 코드까지 일관되게 연결하는 과정

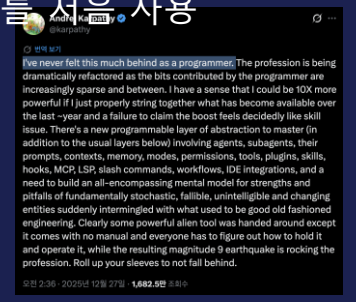
그래서 생기는 일

- 요구 만족의 근거가 없다
- 무엇을 왜 고쳤는지 나중에 추적 불가
- 에이전트가 오래 일할수록 "통과한 듯 보이는" 코드가 는다

탄생과 확산 (2025년 초)

- 2025년 2월, OpenAI 공동 창립자 안드레이 카파시(Andrej Karpathy)가 '바이브 코딩'이라는 용어를 처음 사용





- 개발자가 직접 코드를 짜는 '구현(How)' 중심에서 AI에게 방향을 지시하는 '의도(What)' 중심으로 패러다임 전환

발전과 한계 (2025년 말 ~ 2026년)

- 생성된 코드를 사람이 온전히 이해하지 못해 운영 환경에서 보안 결함·유지보수 문제가 드러남
- AI 시대에 인간 중심 언어가 필요한가, 기계 친화적 언어를 둘 것인가에 대한 새 논쟁

체험 미니 체험 — Gemini에게 "계산기 만들어줘" 한 줄만 던져 보고, 결과 코드의 각 줄이 어떤 요구에서 나왔는지 설명할 수 있는지 확인해 본다.

개발의 다섯 단계 — 리서치에서 디버깅까지 추적 가능하게 하는 것이 관건

1

리서치

요구·기존 코드·제약을 파악한다

2

계획

접근 전략과 작업 순서를 설계한다

3

코드

계획을 실제 구현으로 옮긴다

4

테스트

의도대로 동작하는지 검증한다

5

디버깅

실패 원인을 추적하고 고친다

핵심

에이전트와 함께하는 개발도 이 다섯 단계를 지난다. 관건은 각 단계를 계약과 커밋으로 추적 가능하게 남기는 것(C2C)이다. 특히 테스트는 구현이 의도대로 동작한다는 증거이자, 이후 변경으로 기존 기능이 깨지는 것을 막는 안전망이다. 바이브 코딩은 이 과정을 한 줄 요청으로 뭉갠다.

왜 구현보다 테스트를 먼저 해야 할까?

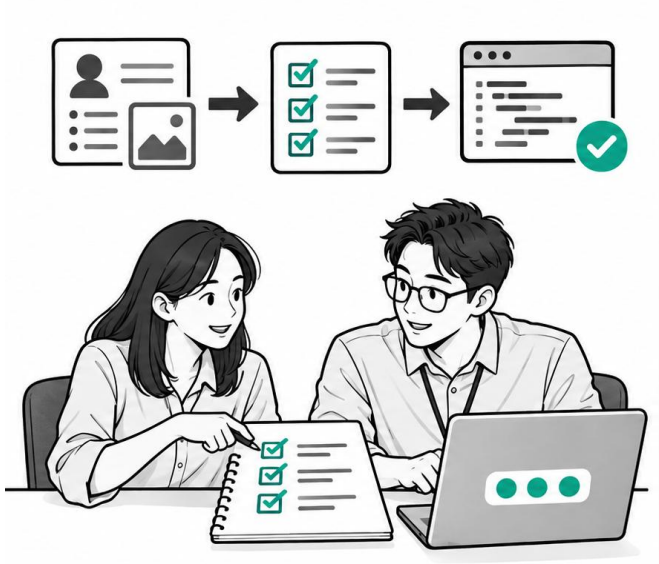
판단의 조급성 · Discovery

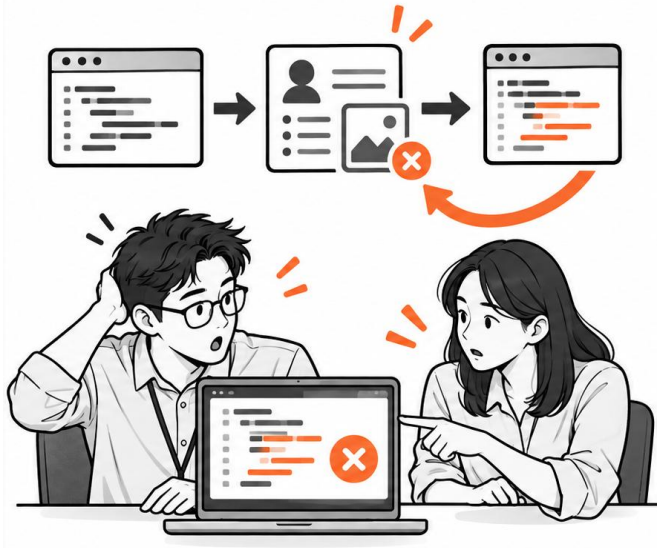
요구사항만으로는 해석이 엇갈릴 수 있다. 구현 전에 구체적인 예시와 테스트로 "어떻게 동작해야 하는가"를 합의한다.

행동의 조급성 · Delivery

먼저 합의한 테스트를 통과하도록 구현한다. 테스트가 없다면 코드가 완성돼도 요구사항을 만족했는지 판단하기 어렵다.

요구 예시 → 테스트 합의 → 구현 성공





구현 선행 → 누락 조건 발견 → 코드 재작업

문제 테스트 우선은 검증 순서만 바꾸는 기법이 아니다. 구현 전에 요구사항을 실행 가능한 예시로 구체화하여, "무엇을 만들어야 하는가"에 대한 오해와 재작업을 줄이는 방법이다.

판단 · 행동 · 검증 — 무엇을 만들고, 어떻게 동작하며, 어떤 결과를 보장해야 하는가

TDD 의 테스트는 세 가지 질문에서 나온다. 질문마다 만들어지는 테스트의 종류가 다르다.

① 요구사항 분석 Requirement Analysis

무엇이 필요한가?

관점
사용자 · 비즈니스

대상
Need

만들어지는 테스트
Acceptance Test

→

② 기능 분석 Feature Analysis

무엇을 해야 하는가?

관점
시스템 행동

대상
Capability

만들어지는 테스트
Example Test

→

③ 불변식 분석 Invariant Analysis

무엇이 깨지면 안 되는가?

관점
시스템 안전성

대상
Property

만들어지는 테스트
Property / Contract Test

핵심 질문이 바뀌면 테스트도 바뀐다 — Need는 인수 테스트로, Capability는 예제 테스트로, Property는 불변식 테스트로 구체화한다.

① 요구사항 분석 — 무엇을 만들어야 하는가?

요구사항 분석(Requirement Analysis)의 출발점은 사용자 또는 비즈니스가 무엇을 필요로 하는지 찾아내는 것이다.

장바구니 예시

REQ-01 고객은 장바구니에 담긴 상품의 최종 결제금액을 확인할 수 있어야 한다.

이것만으로는 코드를 쓸 수 없다

- 요구사항 한 줄에는 계산 규칙이 없다
- 규칙을 모르면 테스트의 기대값도 쓸 수 없다
- 그래서 질문이 이어져야 한다

이어지는 질문

1. 상품 가격과 수량은 어떻게 계산하는가?
2. 할인은 있는가?
3. VIP 는 다른가?
4. 할인의 적용 순서는?
5. 경계 금액(정확히 50,000원)은 포함인가?

Need | **Requirement** | **질문** | **구체화 (예시 표)**

핵심 Need → Requirement → 질문과 구체화 — Day 2 의 '사양 찾기'와 예시 표(C1~C7)가 바로 이 과정이다.

② 기능 분석 — 그래서 시스템은 무엇을 해야 하는가?

요구사항이 파악되면 시스템이 제공해야 하는 기능(Feature / Function)으로 분해한다. 기능마다 구체적인 규칙(Behavior / Business Rule)이 붙는다.

REQ-01 을 기능으로 분해

```
Final Total 계산
├── Subtotal 계산
├── Threshold Discount 적용
├── VIP Discount 적용
└── Final Total 반환
```

기능 하나 = 규칙 하나

Feature Threshold Discount

Rule `subtotal >= 50,000 → 10% 할인`

규칙이 정해지면 기대값을 쓸 수 있다 — 테스트가 자연스럽게 만들어진다.

tests/test_cart.py — 규칙이 그대로 테스트가 된다

```
def test_threshold_discount_at_50000():
    assert final_total([{"price": 50000, "qty": 1}], is_vip=False) == 45000
```

핵심 Requirement 는 '무엇이 필요한가', Feature 는 그것을 실현하려고 '시스템이 무엇을 해야 하는가' 이다.

③ 불변식 분석 — 어떤 결과가 어떤 상황에서도 보장되어야 하는가?

기능을 분석했다고 끝나지 않는다. 기능이 실행되고 요구사항이 바뀌더라도 반드시 유지되어야 하는 성질을 찾는다.

INV-01

`final_total >= 0`

INV-02

`final_total <= subtotal`

둘을 합치면 — 결제 금액의 불변식

$$0 \le \text{final_total} \le \text{subtotal}$$

Example — 특정 사례 하나

```
# 5만 원이면 45,000원
assert final_total(items_50000, False) == 45000
```

Property — 허용된 모든 입력

```
# 어떤 장바구니든
total = final_total(items, is_vip)
assert 0 <= total <= subtotal(items)
```

- 불변식은 Example 보다 추상화 수준이 한 단계 높다 — 하나의 정답이 아니라 '항상 지킬 약속'이다
- Day 2 · 4교시 '불변식' 실습에서 여러 장바구니 × VIP 여부 조합으로 이 성질을 테스트한다

원칙 불변식은 새 기능(쿠폰 · 포인트 …)이 추가될 때 가장 먼저 깨진다 — 그래서 가장 먼저 테스트로 구체화한다.

Example 과 Property — 예제 하나 vs 항상 성립하는 성질

Property 란 특정 예제 하나의 결과가 아니라, 일정한 입력이나 상태의 범위에서 시스템이 만족해야 하는 일반적인 성질 또는 관계이다.

Example

`add(2, 3) == 5`

특정 입력에 대한 정답

Property

`add(a, b) == add(b, a)`

허용되는 모든 a, b 에서 성립

실행 결과

터미널

```
> pytest -q
.............................                                    [100%]
31 passed in 0.01s
```

tests/test_add.py

```python
def test_example():                          # Example — 특정 입력 하나
    assert add(2, 3) == 5


VALUES = [-7, 0, 1, 2.5, 10**6]


@pytest.mark.parametrize("a, b", list(itertools.product(VALUES, VALUES)))
def test_commutative(a, b):                  # Property — 허용된 모든 a, b
    assert add(a, b) == add(b, a)


@pytest.mark.parametrize("x", VALUES)
def test_zero_is_identity(x):                # Property — 항등원
    assert add(x, 0) == x
```

Property 를 찾는 질문

- 순서를 바꿔도 같은가? (교환법칙)
- 아무것도 안 하는 값이 있는가? (항등원)
- 되돌리면 원래대로인가? (역연산)
- 결과가 항상 범위 안인가? (불변식)

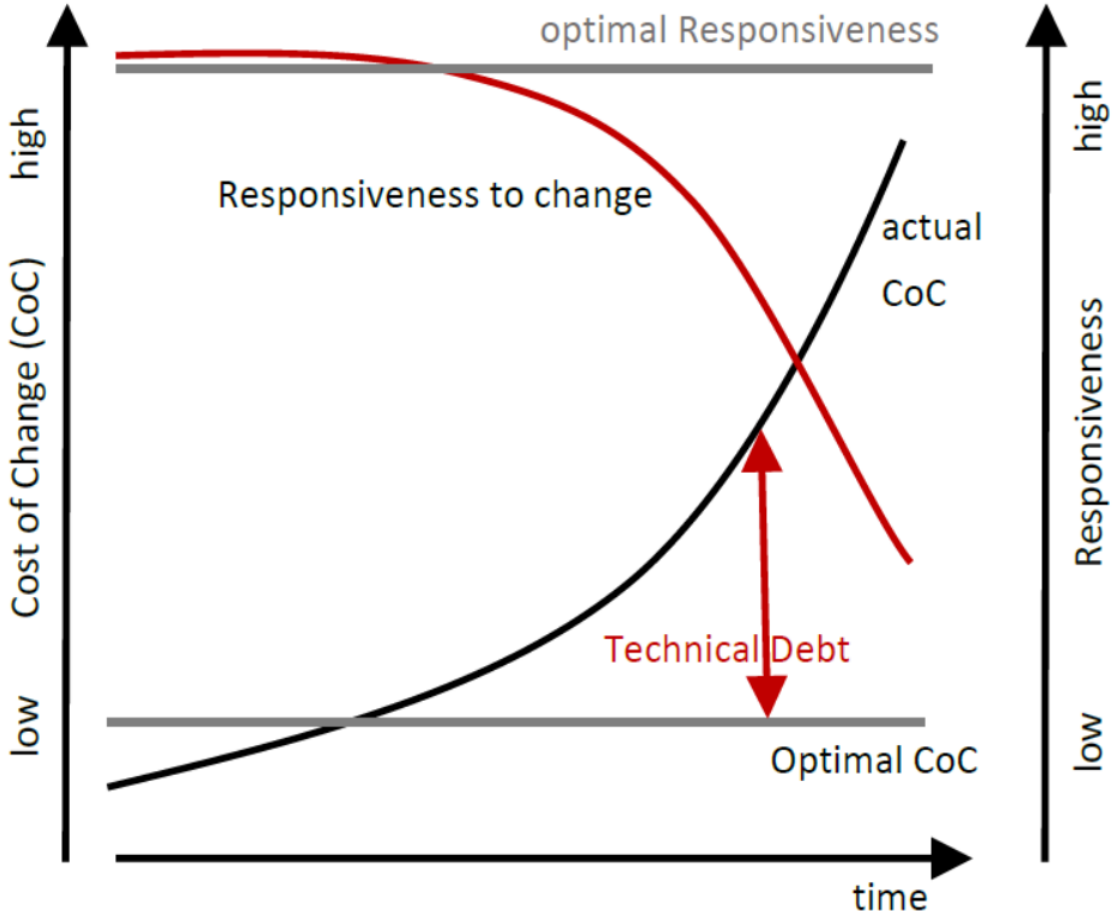
TIP Example 5개를 쓰는 대신 Property 1개를 여러 입력으로 돌리면 31개의 확인이 된다 — 실무 도구로는 Hypothesis(속성 기반 테스트)가 있다.

사칙연산 계산기에 적용 — 요구사항 · 기능 · 불변식

구분	Requirement (요구사항)	Feature (기능)	Invariant (INV, 불변식)
핵심 질문	사용자가 원하는 것은?	시스템이 무엇을 할 수 있어야 하나?	어떤 경우에도 무엇이 보장되어야 하나?
덧셈	두 수를 더할 수 있어야 한다	`add(a, b)`	`add(x, 0) == x` `add(a, b) == add(b, a)`
뺄셈	두 수를 뺄 수 있어야 한다	`subtract(a, b)`	`subtract(x, 0) == x` `subtract(x, x) == 0`
곱셈	두 수를 곱할 수 있어야 한다	`multiply(a, b)`	`multiply(x, 1) == x · multiply(x, 0) == 0` `multiply(a, b) == multiply(b, a)`
나눗셈	두 수를 나눌 수 있어야 한다	`divide(a, b)`	`divide(x, x) == 1  (x ≠ 0)` `divide(x, 0) → ZeroDivisionError`
Example Test	요구사항을 구체적인 사례로 확인	add(2, 3) == 5 divide(6, 3) == 2	—
TDD 관점	무엇을 만들어야 하는가	어떤 동작을 테스트해야 하는가	변경해도 반드시 보장되어야 할 것

핵심 같은 계산기도 세 질문으로 보면 테스트가 세 층으로 나뉜다 — 예제 테스트는 기능을, 불변식 테스트는 수학적 성질을 지킨다.

클린 코드와 변경 비용 — 시간이 지날수록



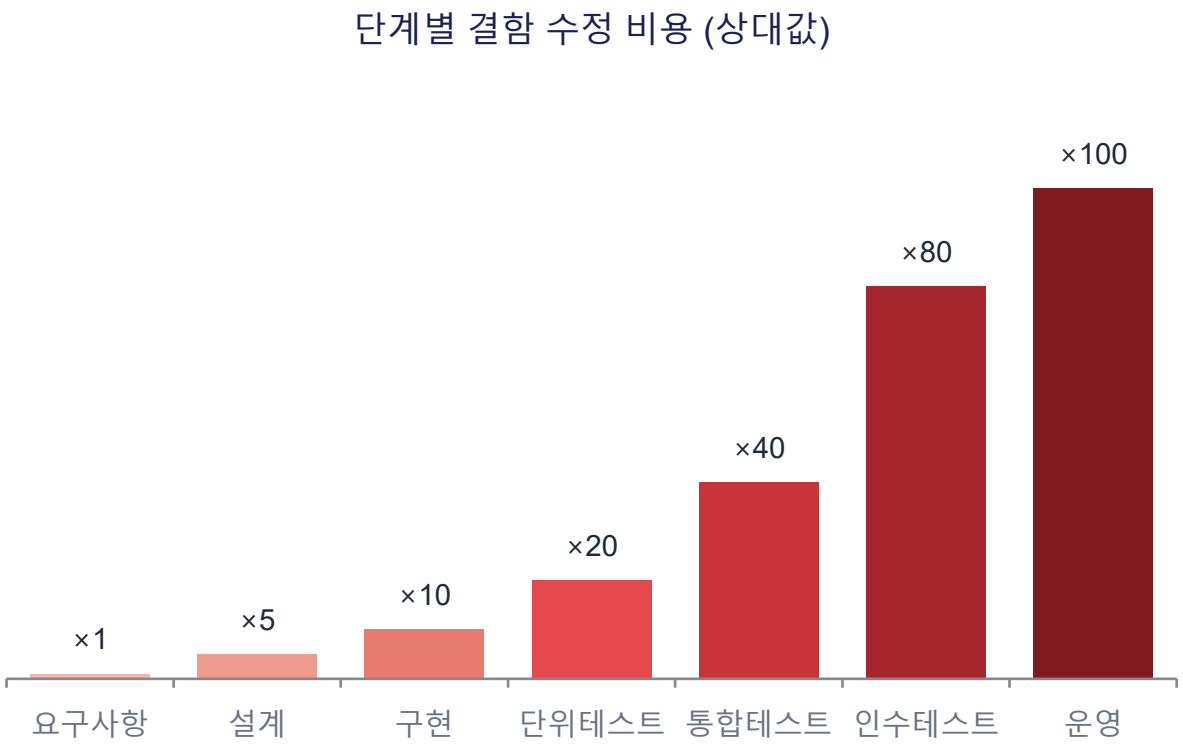
시간이 지날수록	클린 코드 ✗	클린 코드 ✓
변경 비용	기하급수적 증가	완만하게 유지
응답 속도	점점 느려짐	빠른 응대 유지
버그 발생	수정 시 부작용	테스트로 즉시 감지

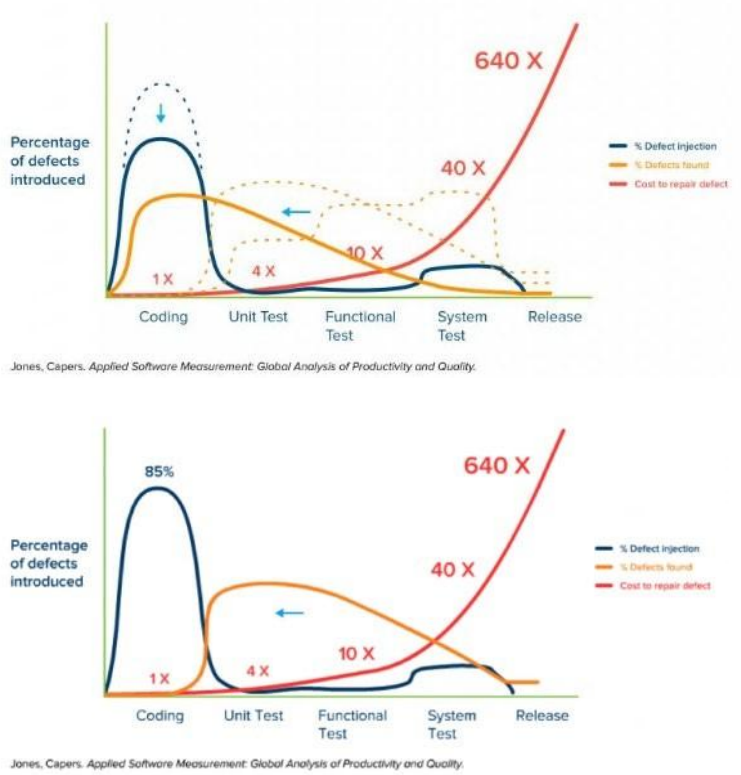
그래프 읽는 법

- 빨간 곡선: 변화에 대한 대응력(Responsiveness)이 시간이 갈수록 떨어진다
- 검은 곡선: 실제 변경 비용(CoC)이 기하급수적으로 오른다
- 두 곡선 사이의 간격 = 기술 부채(Technical Debt)

테스팅의 비용 — 결함은 늦게 발견할수록 비싸다

결함은 늦게 발견할수록 수정 비용이 기하급수적으로 증가한다 (Cost of Defect).





근거 TDD: 개발 시간 15~35% 증가 → 결함률 40~90% 감소 (Microsoft·IBM 연구)

TDD 한 사이클 — RED · GREEN · REFACTOR (Python)

RED

실패하는 테스트를 먼저

만지는 것

tests/ 만 — 구현(src/)은 건드리지 않는다

끝나는 신호

테스트가 "의도대로" 실패

GREEN

최소 구현으로 통과

만지는 것

src/ 만 — 요구사항에 없는 기능은 금지

끝나는 신호

테스트가 전부 통과

REFACTOR

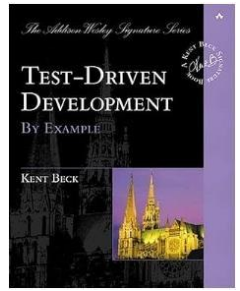
동작 그대로 구조 정리

만지는 것

중복 제거·이름 정리 — 계약 불변

끝나는 신호

여전히 GREEN 유지



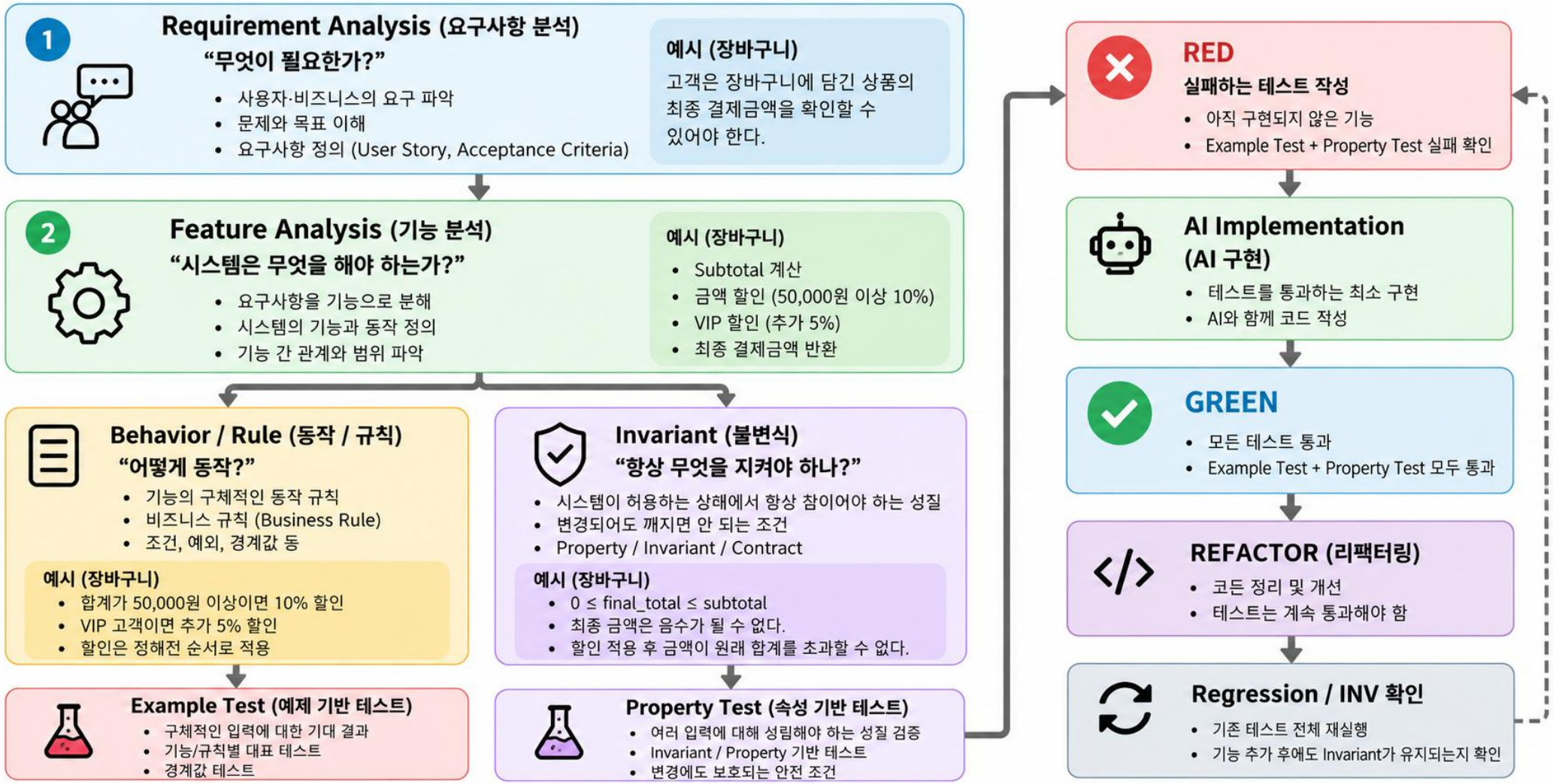
Kent Beck, Test-Driven Development



(출처: kentbeck.com)

```
# RED — tests/test_calc.py
def test_add(): assert add(2, 3) == 5
# GREEN — src/calc.py
def add(a, b): return a + b
```

TDD 한 사이클에서의 판단 · 행동 · 질문 적용



TDD 실행 프로세스 — Ask · Respond · Refine · Repeat

TDD 및 리팩터링으로 클린 코드에 이른다. 테스트는 시스템과 나누는 대화다.

1 Ask · 질문

테스트 작성으로 시스템에 질문한다

테스트 수행 결과는 실패 (RED)

2 Respond · 응답

테스트를 통과하는 코드를 작성해 질문에 대답한다

테스트 성공 (GREEN)

3 Refine · 정제

아이디어를 통합하고, 불필요한 것은 제거하고, 모호한 것은 명확히 한다

리팩터링 (REFACTOR)

4 Repeat · 반복

다음 질문으로 대화를 이어간다

다음 테스트 (다음 RED)

핵심 RED 는 '질문', GREEN 은 '대답', REFACTOR 는 '대답을 다듬기' — 질문 없이 대답(코드)부터 쓰면 무엇에 대한 대답인지 알 수 없다.

참고 출처: Kent Beck, 『Test-Driven Development: By Example』(2002) · Ask–Respond–Refine–Repeat는 이 과정을 이루는 네 단계다.

02

CHAPTER 2 · 이론

SW 테스팅과 단위 테스트 설계

SW 테스팅 · F.I.R.S.T · Right-BICEP · CORRECT · AAA · 테스트 케이스 · 자동화 — 모든 예제는 Python(pytest)으로 실제 실행한 결과다.

⏱ 13:00–13:50 · 2장 이론 → pytest 기초 · 강사 시연

SW 테스팅이란 무엇인가?

SW 테스팅은 SW 의 품질을 조사하는 과정 — 결함을 찾기 위해 SW 를 실행하는 과정이다 (Myers).

테스트 레벨과 Python 도구

테스트 레벨	목적	수행 주체	Python 도구
Unit Test	개별 함수 · 클래스 검증	개발자	pytest · unittest
Integration Test	모듈 간 연동 검증	개발자 · QA	pytest + fixture · 진짜 DB(SQLite) · testcontainers
System Test	전체 시스템 기능 검증	QA	Playwright for Python · Selenium · Robot Framework
Acceptance Test	사용자 요구 충족 확인	고객 · QA	pytest-bdd · behave (Gherkin)

구분	정의
다익스트라	테스트로 버그의 존재를 증명할 수는 있지만, 버그가 없음을 증명하는 데는 쓸 수 없다
Myers	테스팅은 결함을 찾으려는 의도로 SW 를 실행하는 과정이다

Testing 의 4 영역 — Agile Testing Quadrants



다음 중 개발팀을 지원하는 테스트는?

1. Q1, Q2
2. Q3, Q4
3. Q1, Q4
4. Q2, Q3

출처: https://blog.qatestlab.com/2020/06/04/agile-testing-practicecso/mpatibility

단위 테스트 원칙 — F.I.R.S.T (Python)

테스트가 없는 코드를 수정하면?

변경이 어디까지 영향을 미치는지 알기 어려워 수정이 조심스러워진다. 검증되지 않은 변경이 쌓이면 코드의 품질과 유지보수성이 점차 낮아진다.

테스트가 잘 갖춰진 코드를 수정하면?

잘못 고쳐도 테스트가 잡아 주니 편하게 수정한다. 리팩토링 자유도가 높아진다.

원칙	설명	Python (pytest)
Fast	테스트는 빨리 돌아야 한다	함수 단위 — ms 단위 실행
Independent	테스트는 서로 의존하면 안 된다	fixture로 초기화 (@pytest.fixture)
Repeatable	어떤 환경에서도 같은 결과	@pytest.mark.parametrize · 고정 시드
Self-Validating	결과는 bool로 자동 판단	assert expected == actual
Timely	테스트는 실 코드 직전에 작성	TDD — 테스트 함수 먼저 작성

F · Fast — 테스트는 빠르게 실행되어야 한다

❌ 느린 테스트 — 외부 연결이 매번 1초

```python
import time


class UserDB:
    """진짜 DB 를 흉내 낸다 – 연결에 1초가 걸린다"""
    def connect(self):
        time.sleep(1)

    def get_name(self, user_id):
        return {1: "Hello"}[user_id]

# tests/test_fast.py
def test_add_is_fast():                # ✅ 순수 함수 – ms 단위
    assert add(5, 5) == 10


def test_name_with_real_db():          # ❌ 외부 연결 – 매번 1초
    db = UserDB()
    db.connect()
    assert db.get_name(1) == "Hello"
```

실행 결과

```
> pytest -q --durations=2
..                                                                  [100%]
======================== slowest 2 durations ========================
1.00s call     tests/test_fast.py::test_name_with_real_db

(1 durations < 0.005s hidden.  Use -vv to show these durations.)
2 passed in 1.01s
```

주의 테스트가 느리면 덜 돌리게 되고, 덜 돌리면 회귀를 늦게 발견한다. --durations 로 느린 테스트를 찾는다.

F · 개선 — 느린 의존성은 대역으로

개선한 코드

```python
def greeting(db, user_id):
    return f"{db.get_name(user_id)}!"

# tests/test_fast.py
class FakeDB:                          # 메모리 대역 – 연결이 없다
    def get_name(self, user_id):
        return {1: "Hello"}[user_id]


def test_greeting_with_fake_db():
    assert greeting(FakeDB(), 1) == "Hello!"
```

실행 결과

```
> pytest -q --durations=1
.                                                                    [100%]
======================== slowest 1 durations ========================

(1 durations < 0.005s hidden.  Use -vv to show these durations.)
1 passed in 0.00s
```

핵심 DB · 네트워크 · sleep 이 없는 순수 로직은 ms 단위 — 의존성을 인자로 받으면(주입) 테스트에서 테스트 대역으로 바꿀 수 있다.

I · Independent — 테스트는 서로 독립적이어야 한다

❌ 모듈 전역 변수를 공유하는 테스트

```python
counter = 0                              # ❌ 모듈 전역 – 테스트끼리 공유


def test_increment():
    global counter
    counter += 1
    assert counter == 1


def test_decrement():
    global counter
    counter -= 1
    assert counter == -1                 # 혼자 돌리면 통과, 같이 돌리면 실패
```

실행 결과

```
> pytest -q --tb=line -k decrement
.                                                                    [100%]
1 passed, 1 deselected in 0.01s

> pytest -q --tb=line
.F  [100%]
=== FAILURES ===
E   assert 0 == -1
~/ex/tests/test_counter.py:13: assert 0 == -1
=== short test summary info ===
FAILED tests/test_counter.py::test_decrement - assert 0 == -1
1 failed, 1 passed in 0.01s
```

주의 혼자 실행하면 통과하지만 함께 실행하면 실패한다면, 테스트 사이에 상태가 공유되고 있을 가능성이 크다. 실행 순서나 환경에 따라 결과가 달라지는 불안정한 테스트(Flaky Test)는 신뢰하기 어렵다.

I · 개선 — 테스트마다 새 객체 (fixture)로 격리

개선한 코드

```python
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

    def decrement(self):
        self.value -= 1

# tests/test_counter.py
@pytest.fixture
def counter():                          # ✅ 테스트마다 새 객체
    return Counter()


def test_increment(counter):
    counter.increment()
    assert counter.value == 1


def test_decrement(counter):
    counter.decrement()
    assert counter.value == -1
```

실행 결과

```
> pytest -q
..                                                    [100%]
2 passed in 0.01s
```

- 전역 변수 · 싱글턴 · 파일 · DB 데이터 = 새는 상태
- pytest -p no:randomly 대신 순서를 섞어도 통과해야 한다

핵심

Fixture(픽스처)란 테스트를 실행하는 데 필요한 사전 준비 상태나 데이터

@pytest.fixture 는 테스트마다 새로 만들어진다 — 공유 상태가 없으니 어떤 순서 · 어떤 조합으로 실행해도 같은 결과.

R · Repeatable — 언제 어디서 돌려도 같은 결과

❌ 현재 시각에 의존하는 테스트

```python
import time


def now_ms():
    return int(time.time() * 1000)

# tests/test_stamp.py
def test_now_ms():                          # ❌ 실행할 때마다 값이 다르다
    assert now_ms() == 1_700_000_000_000
```

실행 결과

```
> pytest -q --tb=line
F  [100%]
=== FAILURES ===
E   assert 1791077991468 == 1700000000000
     +  where 1791077991468 = now_ms()
~/ex/tests/test_stamp.py:5: assert 1791077991468 == 1700000000000
=== short test summary info ===
FAILED tests/test_stamp.py::test_now_ms - assert 1791077991468 == 1...
1 failed in 0.01s
```

주의 현재 시각 · 난수 · 네트워크 · 환경 변수에 기대는 테스트는 실행할 때마다 결과가 달라진다.

R · 개선 — 시간에 영향을 받지 않는 테스트 만들기

개선한 코드

```python
import time


def now_ms(clock=time.time):            # ✅ 시계를 주입할 수 있게
    return int(clock() * 1000)

# tests/test_stamp.py
def fixed_clock():
    return 1_700_000_000.0


def test_now_ms_with_fixed_clock():
    assert now_ms(fixed_clock) == 1_700_000_000_000
```

실행 결과

```
> pytest -q
.                                                        [100%]
1 passed in 0.01s
```

- 시간 · 난수 → 주입하거나 고정값 (random.seed, 고정 clock)
- 기본값을 두면 호출하는 쪽 코드는 바뀌지 않는다

핵심 현재 시각을 외부에서 전달받도록 설계하면, 운영 환경에서는 실제 시각을 사용하고 테스트에서는 특정 시각을 지정할 수 있다. 따라서 테스트할 때마다 같은 결과를 확인할 수 있다.

S · Self-validating — 통과 / 실패를 테스트가 스스로 판정

❌ 출력만 하는 테스트 — 오타가 있어도 통과

```
def print_greeting(name):
    print(f"Helo, {name}!")            # 오타가 있다

# tests/test_greet.py
def test_print_only():                 # ❌ 출력만 — 사람이 봐야 안다
    print_greeting("World")
```

실행 결과

```
> pytest -q
.                                                        [100%]
1 passed in 0.01s
```

주의 assert 가 없는 테스트는 항상 통과한다 — 'Helo' 오타를 사람이 화면에서 찾아야 한다면 자동 테스트가 아니다.

S · 개선 — 출력도 붙잡아서 assert

개선한 코드

```python
from greet import print_greeting


def test_greeting_is_checked(capsys):  # ✅ 출력을 붙잡아 assert
    print_greeting("World")
    assert capsys.readouterr().out == "Hello, World!\n"
```

실행 결과

```
> pytest -q --tb=line
F  [100%]
=== FAILURES ===
E   AssertionError: assert 'Helo, World!\n' == 'Hello, World!\n'

      - Hello, World!
      ?    -
      + Helo, World!
~/ex/tests/test_greet.py:6: AssertionError: assert 'Helo, World!\n' == 'Hello, World!\n'
=== short test summary info ===
FAILED tests/test_greet.py::test_greeting_is_checked - AssertionErr...
1 failed in 0.01s
```

핵심 capsys는 프로그램이 화면에 출력한 내용을 수집하는 pytest 픽스처다. readouterr().out으로 출력값을 가져와 예상 결과와 비교할 수 있다. 실패를 확인했으면 greet.py 의 'Helo' 를 고치고 다시 GREEN.

T · Timely — 테스트는 적절한 시점(구현 전)에 쓴다

✅ 구현보다 테스트가 먼저 (RED)

```python
def square(x):
    raise NotImplementedError

# tests/test_square.py
@pytest.mark.parametrize("x, expected", [(5, 25), (0, 0), (-3, 9)])
def test_square(x, expected):          # 구현보다 먼저 쓴다
    assert square(x) == expected
```

실행 결과

```
> pytest -q --tb=line
FFF  [100%]
=== FAILURES ===
E   NotImplementedError
~/ex/src/mathx.py:2: NotImplementedError
E   NotImplementedError
~/ex/src/mathx.py:2: NotImplementedError
E   NotImplementedError
~/ex/src/mathx.py:2: NotImplementedError
=== short test summary info ===
FAILED tests/test_square.py::test_square[5-25] - NotImplementedError
FAILED tests/test_square.py::test_square[0-0] - NotImplementedError
FAILED tests/test_square.py::test_square[-3-9] - NotImplementedError
3 failed in 0.01s
```

주의 구현 후 테스트를 작성하면 현재 코드의 동작을 정답으로 받아들이기 쉽다. 요구사항을 검증하기보다 구현에 맞춘 테스트가 되어, 잘못된 동작이나 설계 문제를 초기에 발견하기 어렵다.

T · 개선 — 최소 구현으로 GREEN

개선한 코드

```
def square(x):
    return x * x

# 같은 테스트 3개를 그대로 다시 실행
```

실행 결과

```
> pytest -q
...                                                    [100%]
3 passed in 0.01s
```

- ❌ 구현 후 테스트: 현재 코드의 실행 결과를 그대로 정답으로 삼기 쉽다.
- ✅ 테스트 우선: 먼저 실패를 확인하여 테스트가 오류를 제대로 찾아내는지 검증한다.

핵심 RED → GREEN → REFACTOR — 테스트가 먼저 있으면 '무엇을 만들어야 하는지'가 코드보다 먼저 정해진다.

Right-BICEP & CORRECT — 무엇을 테스트할 것인가 (Python)

Right-BICEP — 결과 검증 6관점

R	Right	결과가 옳은가? assert add(5,5) == 10
B	Boundary	경계조건? sys.maxsize, 0, None
I	Inverse	역관계? sorted → 역순 후 원본
C	Cross-check	다른 수단으로 교차 검증?
E	Error	에러 조건? pytest.raises
P	Performance	성능 조건? 타임아웃 측정

CORRECT — 경계 조건 7관점

C	Conformance	형식 준수: 이메일·날짜·JSON
O	Ordering	정렬·순서: 오름차순 보장
R	Range	범위: 나이 0~150, 인덱스
R	Reference	참조 유효성: 없는 ID·삭제된 객체
E	Existence	존재: None·빈 문자열·0
C	Cardinality	개수: 최소 1, 최대 100
T	Time	시간: 만료·타임아웃·동기화

Right · Boundary — 결과가 맞는가, 경계에서도 맞는가

tests/test_right_boundary.py

```python
import sys

from calculator import add


def test_right_result():                        # R: 결과가 맞는가
    assert add(5, 5) == 10


def test_boundary_large_int():                  # B: 경계 – 아주 큰 수
    big = sys.maxsize
    assert add(big, 1) == big + 1               # Python int 는 넘치지 않는다


def test_boundary_float():                      # B: 경계 – 실수 오차
    assert add(0.1, 0.2) != 0.3
    assert abs(add(0.1, 0.2) - 0.3) < 1e-9
```

src/calculator.py (일부)

```python
def add(a, b):
    return a + b
```

실행 결과

```
> pytest -v
test_right_boundary.py::test_right_result PASSED
test_right_boundary.py::test_boundary_large_int PASSED
test_right_boundary.py::test_boundary_float PASSED
```

핵심

Python의 정수는 값이 커져도 일반적인 오버플로가 발생하지 않는다. 그러므로 숫자가 표현할 수 있는 범위보다, 0이나 허용 가능한 최댓값처럼 처리 기준이 달라지는 지점을 테스트해야 한다.
실수 계산은 표현 오차가 있으므로 값이 정확히 같은지보다 허용 오차 범위 안에 있는지 확인한다.

Inverse · Cross-check — 거꾸로, 다른 방법으로 확인

tests/test_inverse_cross.py

```python
import pytest

from calculator import add, subtract


def test_inverse():                              # I: 역관계로 확인
    result = add(5, 5)
    assert subtract(result, 5) == 5


@pytest.mark.parametrize("a, b", [(2, 3), (-1, 7), (0, 9)])
def test_cross_check(a, b):                      # C: 다른 방법과 비교
    assert add(a, b) == add(b, a) == sum([a, b])
```

src/calculator.py (일부)

```python
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b
```

실행 결과

```
> pytest -v
test_inverse_cross.py::test_inverse PASSED
test_inverse_cross.py::test_cross_check[2-3] PASSED
test_inverse_cross.py::test_cross_check[-1-7] PASSED
test_inverse_cross.py::test_cross_check[0-9] PASSED
```

핵심 역연산(add → subtract)으로 되돌려 보고, 다른 계산 방법(sum)과 비교한다 — 기대값을 손으로 계산하지 않아도 된다.

Error · Performance — 오류 조건과 성능 한계

tests/test_error_perf.py

```python
import time

import pytest

from calculator import divide, factorial


def test_error_divide_by_zero():                # E: 오류 조건
    with pytest.raises(ZeroDivisionError, match="division by zero"):
        divide(10, 0)


def test_performance_factorial():               # P: 성능 한계
    start = time.perf_counter()
    factorial(10_000)
    elapsed = time.perf_counter() - start
    assert elapsed < 0.5                        # 0.5초 안에 끝나야 한다
```

src/calculator.py (일부)

```python
def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return a / b


def factorial(n):
    return math.factorial(n)
```

실행 결과

```
> pytest -v
test_error_perf.py::test_error_divide_by_zero PASSED
test_error_perf.py::test_performance_factorial PASSED
```

핵심 pytest.raises와 match를 사용해 예상한 오류 유형과 메시지가 발생하는지 확인한다. 성능 측정 결과는 실행 환경에 따라 달라질 수 있으므로 기준 시간을 여유 있게 설정하고, 야간 정기 테스트에서 확인한다.

C · Conformance · 형식 준수 — 데이터가 정해진 형식(이메일 · 날짜 · JSON 스키마)을 따르는가

src/ — 검증 대상

```
import re

EMAIL = re.compile(r"^[\w.+-]+@[\w-]+(\.[\w-]+)+$")


def is_valid_email(email: str) -> bool:
    return bool(EMAIL.match(email))
```

tests/test_conformance.py

```
@pytest.mark.parametrize("email, ok", [
    ("test@example.com", True),
    ("a.b+tag@mail.co.kr", True),
    ("invalid-email", False),
    ("no-domain@", False),
])
def test_email_format(email, ok):
    assert is_valid_email(email) is ok
```

실행 결과

```
> pytest -v test_conformance.py
test_conformance.py::test_email_format[test@example.com-True] PASSED
test_conformance.py::test_email_format[a.b+tag@mail.co.kr-True] PASSED
test_conformance.py::test_email_format[invalid-email-False] PASSED
test_conformance.py::test_email_format[no-domain@-False] PASSED
=== 4 passed in 0.01s ===
```

핵심 정상 형식 2개 · 잘못된 형식 2개를 짝으로 — 형식 검증은 '통과해야 하는 것'과 '막아야 하는 것'을 함께 테스트한다.

O · Ordering · 순서 — 정렬 · 시간 순서가 지켜지는가

src/ — 검증 대상

```python
from datetime import date


def is_in_order(dates: list[date]) -> bool:
    return all(a <= b for a, b in zip(dates, dates[1:]))
```

tests/test_ordering.py

```python
def test_dates_in_order():
    assert is_in_order([date(2024, 1, 1), date(2024, 12, 31)])


def test_dates_out_of_order():
    assert not is_in_order([date(2024, 12, 31), date(2024, 1, 1)])


def test_sorted_keeps_order():
    dates = [date(2024, 5, 1), date(2024, 1, 1), date(2024, 3, 1)]
    assert is_in_order(sorted(dates))
```

실행 결과

```
> pytest -v test_ordering.py
test_ordering.py::test_dates_in_order PASSED
test_ordering.py::test_dates_out_of_order PASSED
test_ordering.py::test_sorted_keeps_order PASSED
=== 3 passed in 0.01s ===
```

핵심 순서가 맞는 경우 · 뒤집힌 경우 · 정렬 후 다시 확인 — 순서 규칙은 반대 방향 예제가 있어야 의미가 있다.

R · Range · 범위 — 값이 허용된 최소 · 최대 범위 안에 있는가

src/ — 검증 대상

```python
def is_valid_age(age: int) -> bool:
    return 0 <= age <= 120
```

tests/test_range.py

```python
@pytest.mark.parametrize("age, ok", [
    (-1, False), (0, True),        # 하한 경계 양쪽
    (25, True),                    # 정상값
    (120, True), (121, False),     # 상한 경계 양쪽
])
def test_age_range(age, ok):
    assert is_valid_age(age) is ok
```

실행 결과

```
> pytest -v test_range.py
test_range.py::test_age_range[-1-False] PASSED
test_range.py::test_age_range[0-True] PASSED
test_range.py::test_age_range[25-True] PASSED
test_range.py::test_age_range[120-True] PASSED
test_range.py::test_age_range[121-False] PASSED
=== 5 passed in 0.01s ===
```

핵심 경곗값 테스트는 허용 범위의 경계 안팎을 함께 확인한다. 정상 범위가 0~120이라면 하한에서는 -1과 0, 상한에서는 120과 121을 검사해 경계값의 포함 여부가 올바르게 구현됐는지 확인한다.

R · Reference · 참조 — 참조하는 대상이 없을 때(삭제 · 잘못된 ID) 어떻게 되는가

src/ — 검증 대상

```
class UserRepository:
    def __init__(self, users):
        self._users = dict(users)

    def find_by_id(self, user_id):
        return self._users.get(user_id)          # 없으면 None
```

tests/test_reference.py

```
def test_existing_user_is_found():
    repo = UserRepository({1: "Alice"})
    assert repo.find_by_id(1) == "Alice"


def test_missing_user_is_none():
    repo = UserRepository({1: "Alice"})
    assert repo.find_by_id(9999) is None
```

실행 결과

```
> pytest -v test_reference.py
test_reference.py::test_existing_user_is_found PASSED
test_reference.py::test_missing_user_is_none PASSED
=== 2 passed in 0.01s ===
```

핵심 Python에서는 값이 없음을 None으로 나타낸다. 존재하지 않는 ID를 조회하면 None을 반환하는지 테스트로 확인한다. 예외를 발생시키도록 할지는 요구사항에 따라 결정한다.

E · Existence · 존재 — 필수 값이 비어 있거나(None · '') 아예 없을 때

src/ — 검증 대상

```python
def is_valid_user(user: dict) -> bool:
    return bool(user.get("name")) and bool(user.get("password"))
```

tests/test_existence.py

```python
@pytest.mark.parametrize("user, ok", [
    ({"name": "Alice", "password": "pw123"}, True),
    ({"name": None, "password": "pw123"}, False),   # 값이 None
    ({"name": "", "password": "pw123"}, False),     # 빈 문자열
    ({"password": "pw123"}, False),                 # 키가 없음
])
def test_required_fields(user, ok):
    assert is_valid_user(user) is ok
```

실행 결과

```
> pytest -v test_existence.py
test_existence.py::test_required_fields[user0-True] PASSED
test_existence.py::test_required_fields[user1-False] PASSED
test_existence.py::test_required_fields[user2-False] PASSED
test_existence.py::test_required_fields[user3-False] PASSED
=== 4 passed in 0.01s ===
```

핵심 None, 빈 문자열, 키가 없는 경우는 서로 구분해야 한다. 각각을 테스트해 값 조회와 참·거짓 판단이 의도대로 동작하는지 확인한다.

C · Cardinality · 개수 — 0개 · 1개 · 여러 개 · 최대 개수에서 동작하는가

src/ — 검증 대상

```python
def is_valid_name_list(names: list[str]) -> bool:
    return 1 <= len(names) <= 100
```

tests/test_cardinality.py

```python
@pytest.mark.parametrize("count, ok", [
    (0, False), (1, True),          # 0 · 1 · 다수
    (2, True),
    (100, True), (101, False),      # 최대 경계 양쪽
])
def test_name_list_size(count, ok):
    assert is_valid_name_list(["x"] * count) is ok
```

실행 결과

```
> pytest -v test_cardinality.py
test_cardinality.py::test_name_list_size[0-False] PASSED
test_cardinality.py::test_name_list_size[1-True] PASSED
test_cardinality.py::test_name_list_size[2-True] PASSED
test_cardinality.py::test_name_list_size[100-True] PASSED
test_cardinality.py::test_name_list_size[101-False] PASSED
=== 5 passed in 0.01s ===
```

핵심 항목 수는 없음(0개), 하나(1개), 여러 개, 허용 한도의 안팎(100개와 101개)으로 나누어 테스트한다. 이것을 '0·1·다수 원칙(Zero–One–Many)'이라고 한다.

T · Time · 시간 — 응답 시간 · 만료 · 순서 같은 시간 조건

src/ — 검증 대상

```
import time


def timed_call(endpoint, clock=time.perf_counter):
    start = clock()
    endpoint()
    return (clock() - start) * 1000            # ms
```

tests/test_time.py

```
def test_response_time_with_fake_clock():
    ticks = iter([10.000, 10.250])     # 가짜 시계: 250ms 경과
    ms = timed_call(lambda: None, clock=lambda: next(ticks))
    assert ms == 250


def test_response_under_one_second():
    ms = timed_call(lambda: None)      # 진짜 시계 — 상한만 느슨하게
    assert ms < 1000
```

실행 결과

```
> pytest -v test_time.py
test_time.py::test_response_time_with_fake_clock PASSED
test_time.py::test_response_under_one_second PASSED
=== 2 passed in 0.01s ===
```

핵심 경과 시간을 정확히 검사할 때는 테스트용 시각으로 250ms가 지난 상황을 재현한다. 실제 시계를 사용하는 테스트에서는 실행 환경을 고려해 여유 있는 시간 범위만 확인한다.

AAA 패턴 — 좋은 테스트의 구조 (Python)

모든 테스트는 Arrange(준비) → Act(실행) → Assert(검증) 3단계로 명확히 구분한다.

Python — pytest

```python
def test_mean_score_is_calculated():
    # Arrange — 준비
    collection = ScoreCollection()
    collection.add(5)
    collection.add(7)

    # Act — 실행
    result = collection.arithmetic_mean()

    # Assert — 검증
    assert result == 6
```

AAA 단계	pytest	핵심 원칙
Arrange	fixture / 객체 생성	테스트 픽스처 분리
Act	메서드 호출 1줄	단 하나의 동작
Assert	assert 문	명확한 성공/실패

3분할이 무너질 때

Act가 두 줄 이상이면 한 테스트가 두 동작을 검증하고 있다는 신호 — 테스트를 쪼갠다(→ 2.4 Eager Test).

Given-When-Then — 시나리오로 쓰는 테스트

AAA 와 같은 3단계를 기획자·고객도 읽을 수 있는 자연어로 쓴다 (Gherkin 스타일 · BDD).

Given — 주어진 상황

- 합계 60,000원 장바구니가 있고
- 고객은 VIP이다

When — 행동

- 결제 금액을 계산하면

Then — 기대 결과

- 결제 금액은 51,300원이다
- (60,000 → 54,000 → 51,300)

단계	말투	pytest 에서는	장바구니 C4
Given	주어진 상황	입력 · 상태를 준비 (변수 · fixture)	60,000원 · VIP
When	일어난 일	검사할 함수를 한 번 호출	final_total(items, True)
Then	기대 결과	assert 로 규칙 하나를 확인	== 51300

이렇게 쓰면 좋다

- 요구사항을 기획자 · 고객과 같이 읽을 때
- 예시 표 한 줄 = 시나리오 하나
- AI 에게 시나리오 문장으로 요청하면 테스트 이름까지 일관된다

핵심 GWT 는 새 문법이 아니라 AAA 를 자연어로 풀어 쓴 것 — 한 시나리오에 When 은 하나, Then 은 규칙 하나.

Gherkin 시나리오 → pytest — 같은 이야기를 두 가지로

cart_discount.feature — 사람이 읽는 시나리오

```gherkin
Feature: 장바구니 할인
  문턱 할인(50,000원 이상 10%)과 VIP 추가 할인(5%)을 계산한다.

  Scenario: VIP 고객이 문턱을 넘기면 두 번 할인받는다
    Given 합계가 60,000원인 장바구니가 있고
    And 고객은 VIP이다
    When 결제 금액을 계산하면
    Then 결제 금액은 51,300원이다

  Scenario Outline: 문턱 할인의 경계
    Given 합계가 <합계>원인 장바구니가 있고
    And 고객은 일반 고객이다
    When 결제 금액을 계산하면
    Then 결제 금액은 <결제 금액>원이다

    Examples:
      | 합계   | 결제 금액 |
      | 49,999 | 49,999   |
      | 50,000 | 45,000   |
      | 50,001 | 45,000   |
```

tests/test_cart_gwt.py — 같은 시나리오를 pytest 로

```python
def test_given_vip_over_threshold_when_checkout_then_both_discounts():
    # Given — 합계 60,000원 장바구니 + VIP 고객
    items = [{"price": 12000, "qty": 3}, {"price": 24000, "qty": 1}]
    is_vip = True
    # When — 결제 금액을 계산하면
    total = final_total(items, is_vip)
    # Then — 문턱 10% 뒤 VIP 5% 가 차례로 적용된다 (60,000 → 54,000 → 51,300)
    assert total == 51300


@pytest.mark.parametrize("subtotal, expected", [      # Examples
    (49999, 49999),
    (50000, 45000),
    (50001, 45000),
])
def test_given_regular_when_checkout_then_boundary_discount(subtotal, expected):
    items = [{"price": subtotal, "qty": 1}]           # Given
    total = final_total(items)                        # When
    assert total == expected                          # Then
```

참고 .feature 를 그대로 실행하는 도구(pytest-bdd)도 있다. 이 과정에서는 도구 없이 '함수 이름 + # Given / # When / # Then 주석'만 쓴다.

TIP AI 에게: "아래 시나리오를 pytest 로 바꿔줘. 이름은 test_given_..._when_..._then_... 형식, 단계마다 # Given/# When/# Then 주석."

Write Better Tests — 5 Steps

1 Production 코드처럼

테스트 코드도 실제 코드처럼 다룬다 — 읽기 쉽고 유지보수 가능하게.

2 패턴으로 가독성

AAA 등 테스트 패턴으로 의도를 명확히 드러낸다.

3 불안정 테스트 피하기

시간·랜덤·외부 의존을 제거해 Flaky를 없앤다.

4 적절한 수준 선택

단위·통합·E2E 중 검증 목적에 맞는 수준을 고른다.

5 Test Double 활용

핵심 외부 의존만 Fake/Mock으로 격리, 나머지는 실제로.

+ Test Smell 경계

다섯 원칙이 무너지는 지점이 곧 다음 슬라이드의 6가지 대표적 냄새.

Test Smell — 6가지 대표 유형

Mystery Guest
✗ 외부 파일·DB 직접 의존 → 실패 원인 불명확
✓ 데이터 인라인 / Fake 객체로 격리

Flaky Test
✗ 시간·랜덤·전역 상태 의존 → 실행마다 결과 다름
✓ monkeypatch·freezegun으로 고정, fixture 독립

Assertion Roulette
✗ assert 여러 개 → 실패 위치 불명확
✓ assert 메시지 추가 · 검증 대상별로 분리

Eager Test
✗ 한 테스트에 기능 여럿 → SRP 위반
✓ 한 테스트 = 한 관심사로 분리

Setup Bloat
✗ fixture가 비대 → 의도 파악 어려움
✓ 각 테스트에 필요한 준비만

Over-mocking
✗ 모든 것을 Mock → 실제 동작 미검증
✓ 핵심 외부 의존만 Mock, 나머지는 실제

원칙 테스트 코드도 Production 코드처럼: 읽기 쉽고, 빠르고, 독립적이어야 한다. AAA 3분할이 무너지는 곳에서 이 냄새들이 자란다.

테스트 케이스란 무엇인가

테스트 케이스(Test Case)란 요구사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행 결과 등을 명세한 것이다.

테스트 케이스 = 특정 조건 + 입력값 + 기대 결과

1 실행 사전 조건

테스트를 수행하기 위한 전제 상태

2 입력값의 묶음

특정 프로그램 경로를 실행하도록 설계된 입력 집합

3 기대 결과·사후 조건

요구사항과 일치하는지 판정할 예상 출력

테스트 케이스 작성 절차 — 사칙연산 예 (Python)

테스트 단계	입력값	예상값	중요도	결과
1 + 10	1, 10	11	상	성공
0 + 1	0, 1	1	상	성공
0 ÷ 0	0, 0	ZeroDivisionError 발생	상	성공
-1 + (-10)	-1, -10	-11	중	성공

테스트 환경

- Python 3.11+
- pytest 8.x
- VS Code + 통합 터미널
- Windows / macOS / Linux

```
import pytest
from math_utils import add, divide   # pythonpath = src

def test_add_positive():
    assert add(1, 10) == 11

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(0, 0)
```

테스트 케이스 작성 절차 — 목록 양식

ID	구분	수행 절차	기대 결과	추적성	중요도
1.1	우편번호 조회	5자 이상 입력 후 조회 클릭 → 조회 아이콘 클릭	팝업 표시 후 데이터 출력 / 없으면 안내 메시지	요구사항 ID-100	중
…	…	…	…	…	…

좋은 테스트 케이스의 요건

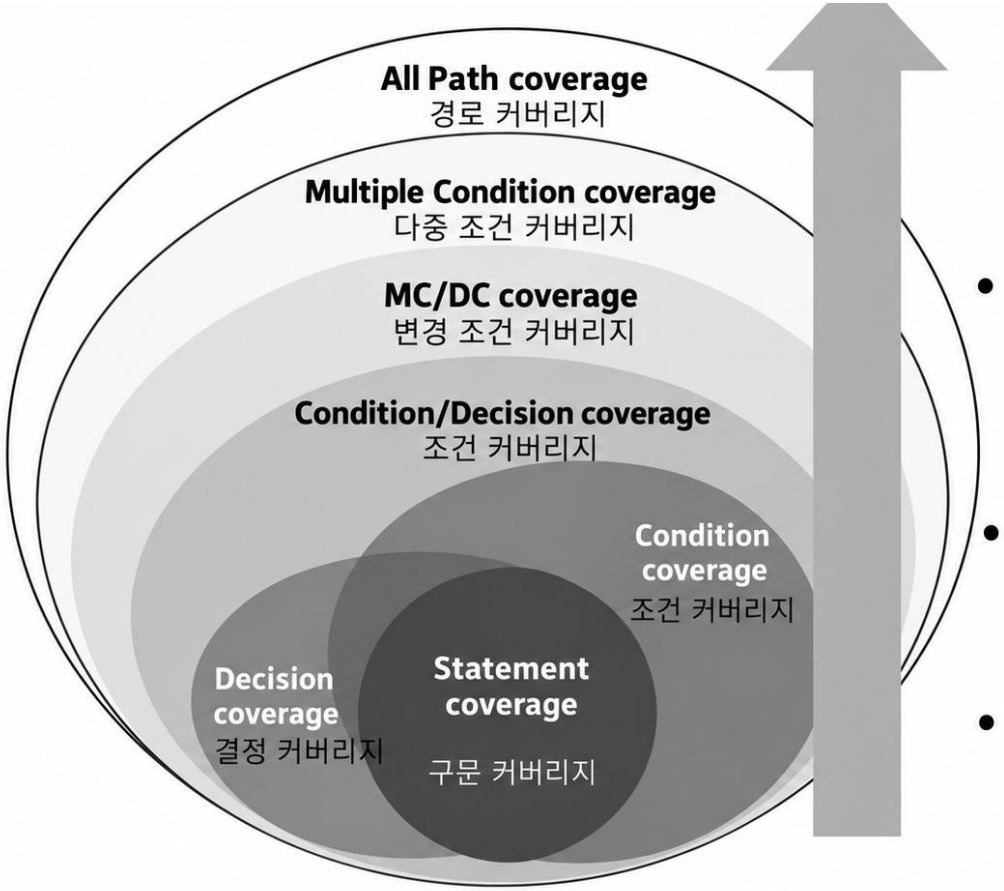
사전조건 · 수행절차 · 기대결과 · 결과 · 추적성 · 중요도 · 비고를 갖춘 검사 가능한 산출물이어야 한다 — 특히 "추적성" 칸이 요구사항 ID와 케이스를 이어 준다.

핵심 이 "추적성" 열이 1장의 C2C · 추적성 개념과 같은 역할을 한다 — 어떤 요구사항이 어떤 케이스로 검증되는지 한눈에 드러나야 한다.

테스트 방법론 — 테스트 목적에 따른 분류

방법론	설명	사례
동적 테스트	프로그램 실행을 요구하는 테스트	화이트박스, 블랙박스
정적 테스트	프로그램 실행 없이 구조를 분석하여 논리성 검증	코드 검사, 워크스루
화이트박스 테스트	프로그램 내부 로직을 보면서 테스트 (구조 테스트)	구조 테스트, 루프 테스트
블랙박스 테스트	프로그램 외부 명세를 보면서 테스트 (기능 테스트)	동등분할, 경계값 분석, 원인-결과 그래프, 오류 예측
하드웨어 테스트	하드웨어의 성능과 신뢰성 테스트	부하 테스트, 스트레스 테스트
소프트웨어 테스트	이해관계자에게 시험 대상 제품·서비스의 품질 정보를 제공하는 조사 과정	사람은 코드의 모든 결함을 찾을 수 없다 — 전략이 필요

Test Coverage — 커버리지 수준과 비용



- 커버리지 100% 달성에 필요한 테스트 케이스 수 증가
- 코드 기준 테스트 범위가 증가 → 식별 결함 수 증가
- 테스트 실행 비용 증가

읽는 법

- 안쪽(구문)에서 바깥(전체 경로)으로 갈수록 기준이 엄격해진다
- 커버리지 100%에 필요한 테스트 케이스 수가 급증한다
- 코드 기준 테스트 범위가 넓어질수록 발견하는 결함은 늘지만, 실행 비용도 함께 증가한다
- pytest --cov=src로 측정 — 목표 수치는 팀이 합의

테스팅 방법 — Edit & Pray vs Cover & Modify

출처: Michael Feathers,『Working Effectively with Legacy Code』(2004)

변경 후 기도하기 (Edit and Pray)

1. 신중한 검토
2. 변경 대상 이해
3. 변경
4. 정상 동작 확인
5. Side Effect 확인

전문적으로 보이지만 신중 ≠ 안정. 체계적인 보호 없이 변경하므로 위험성이 높다.

보호 후 수정하기 (Cover and Modify)

1. 테스트로 코드 보호
2. 변경
3. 테스트 실행
4. 문제 즉시 발견
5. 안전하게 유지보수

테스트 루틴이 피드백이다. 변경 후 문제를 사전에 감지해 Side Effect를 막는다.

핵심 TDD가 가능하게 하는 것이 바로 'Cover and Modify'다. 테스트가 기존 기능의 이상을 즉시 발견해 주기 때문에 안심하고 코드를 개선할 수 있다.

수동 테스트 vs 테스트 자동화

수동 테스트 (Manual Testing)

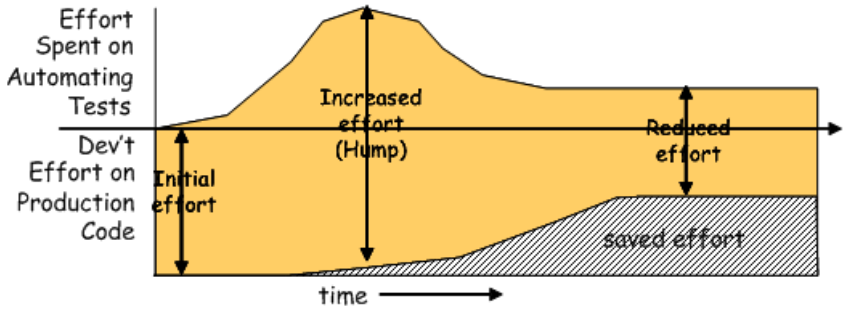
테스터가 자동화 도구 없이 수동으로 테스트 케이스를 실행하는 방식. 탐색적 검증에는 강하지만 반복·회귀에는 느리고 사람 손이 계속 든다.

테스트 자동화 — 3원칙

① 테스트는 자동화되어야 한다

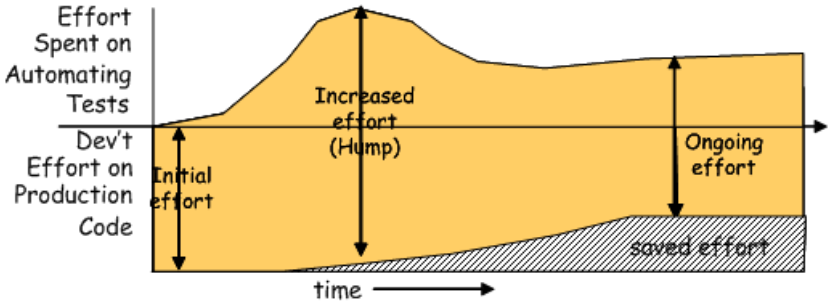
② 테스트는 빨라야 한다

③ 테스트는 어떤 경우에도 수행되어야 한다



Sketch Economics-Good embedded from Economics-Good.gif

Fig. X: An automated unit test project with a good ROI.
The cost-benefit trade off where the total cost is reduced by good test practices.



Sketch Economics-Bad embedded from Economics-Bad.gif

Fig. X: An automated unit test project with a poor ROI.
The cost-benefit trade off where the total cost is increased by poor test practices.

출처: http://xunitpatterns.com/Goals%20of%20Test%20Automation.html

단순히 자동화를 도입하는 것이 아니라 "올바르게 자동화하는 것"이 ROI를 높이는 핵심 — 수동 테스트 시간 vs 자동 테스트 작성·실행 시간을 저울질한다.

Test Automation Manifesto — 자동 테스트가 갖춰야 할 것

출처: Gerard Meszaros, 『xUnit Test Patterns』(2007) · xunitpatterns.com

항목	의미
Concise	가능한 한 단순하게, 그러나 그 이하로는 아니게
Self-Checking	스스로 결과를 보고한다 — 사람의 해석이 필요 없다
Repeatable	사람 개입 없이 여러 번 반복 실행할 수 있다
Robust	지금도 앞으로도 같은 결과 — 외부 환경 변화에 영향받지 않는다
Sufficient	검증 대상 SW 의 모든 요구사항을 검증한다
Specific · Independent	실패가 특정 기능을 가리키고, 어떤 순서로도 독립 실행된다
Maintainable · Traceable	이해 · 수정 · 확장이 쉽고, 코드와 요구사항으로 양방향 추적된다

핵심 특히 추적 가능성(Traceable)은 테스트에서 코드와 요구사항을 확인할 수 있고, 반대로 요구사항에서도 관련 테스트와 코드를 찾아갈 수 있어야 한다는 원칙이다. 이는 1장에서 설명한 C2C와 계약 ID의 취지와 같다.

테스트 목표 설정 — OKR · SMART · 측정 지표

SMART	설명
Specific	구체적
Measurable	측정 가능
Attainable	달성 가능
Realistic	현실적
Time-based	시간 제한

OKR — 목표는 영감, 핵심 결과는 실행

넓은 비전(Objective)을 세우되, 이를 달성할 구체적·측정 가능한 지표(Key Results)를 붙인다.
예: "Gmail 성공적 런칭" → 베타 100만 명 · 전송 0.1초 이하 · 만족도 90점.

측정 지표	수식
테스트 진척율	통과된 케이스 ÷ 생성된 케이스
결함 조치율	수정 완료 결함 ÷ 발견된 결함
요구사항 커버리지	케이스로 확인된 요구사항 ÷ 명세된 요구사항

핵심 품질 목표는 SMART로 세우고 측정 지표로 검증한다 — "요구사항 커버리지"가 100%에 닿는지가 곧 추적성 매트릭스에 빈칸이 없는지와 같은 질문이다.

pytest 기초 — 파일 이름, assert, 실행

tests/ test_calculator.py

```
import pytest
from calculator import add          # src/ 의 모듈


def test_single_number_returns_itself(): # test_ 로 시작
    assert add("7") == 7                 # 참이면 통과
```

규칙	내용
파일 이름	test_*.py
함수 이름	test_로 시작, 무엇을 검사하는지 문장처럼
검사	assert 기대 조건 — 거짓이면 실패
예외 검사	with pytest.raises(ValueError):

실행 명령 — 이 과정에서는 거의 첫 줄만 쓴다

터미널

```
> pytest -q --tb=short          # 짧게: 점(.) = 통과, F = 실패
> pytest -v --tb=short          # 테스트 이름까지 하나씩
> pytest -q -k empty            # 이름에 'empty'가 들어간 테스트만
```

참고 --tb=short는 실패 위치를 짧게 보여 준다. 이 교안의 모든 결과 화면은 이 옵션으로 실제 실행한 출력.

실패 메시지에서 확인해야 할 부분

터미널 – 실제 출력

```
> pytest -q --tb=short
.F  [100%]
=== FAILURES ===
___ test_single_number_returns_itself ___
tests/test_calculator.py:9: in test_single_number_returns_itself
    assert add("7") == 7
E   AssertionError: assert 0 == 7
E    +  where 0 = add('7')
=== short test summary info ===
FAILED tests/test_calculator.py::test_single_number_returns_itself
1 failed, 1 passed in 0.02s
```

① 어느 테스트?

____ 사이의 이름 →
test_single_number_returns_itself

② 기대 vs 실제

E assert 0 == 7 → 실제 0, 기대 7

③ 요약

1 failed, 1 passed → 기존 것은 통과, 새 것만 실패

핵심 '기존 것은 통과, 새 것만 실패'가 정상적인 RED. 기존 테스트까지 실패하면 그건 RED가 아니라 무언가 깨진 것.

TDD 전 과정

RED
실패하는 테스트를 먼저 쓴다

누가: 사람

→

GREEN
테스트를 통과하는 최소 코드

누가: AI

→

REFACTOR
동작은 그대로, 구조만 정리

누가: AI + 사람 검토

왜 테스트를 '먼저' 쓰나

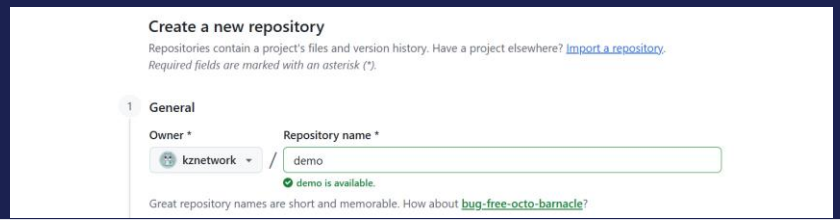
- 실패를 먼저 봐야 테스트가 진짜로 검사한다는 걸 안다
- AI에게 줄 '완료 기준'이 생긴다
- 쌓인 테스트가 다음 변경의 안전망이 된다

AI와 할 때의 역할 분담

- 사람: 요구사항을 테스트로 바꾸고, 결과를 읽고, 승인한다
- AI: 그 테스트만 통과하는 코드를 쓴다
- AI가 테스트를 바꾸려 하면 멈춘다

원칙 정답 기준을 테스트 코드로 적어 두면 확인은 테스트가 한다 — print(total(items)) 를 눈으로 보고 "맞다"고 판단하던 일을 assert total(items) == 45000 이 대신한다. 45000 이면 통과, 아니면 실패. 정답은 사람이 정하고, 테스트는 그 정답과 같은지 매번 확인한다 (실험 A).

Groundwork





보기만 C:\DEV\tdd-course\demo
강사 PC 화면을 본다 · 따라 하지 않는다

TDD 전 과정

1 테스트 작성
사람이 tests/에 한 개

2 실행 → RED
실패 이유가 '아직 없음'인지 확인

3 프롬프트
"이 테스트만 통과, 테스트 수정 금지"

4 diff 승인
Gemini가 보여 주는 변경을 읽고 승인

5 실행 → GREEN
새 테스트 + 기존 테스트 모두

6 정리 · 커밋
필요하면 리팩터링, git commit

참고 시연은 다음 두 장 — ① add 로 RED → GREEN → REFACTOR, ② divide(1, 0) 경계값. 교육생은 보기만 하고, 14:00 실습 1 에서 같은 과정을 직접 한다.

원칙 diff 승인 단계를 건너뛰지 마세요 — AI가 테스트 파일을 건드리려 하면 여기서 '아니요'를 누르는 것이 사람의 역할이다.

보기만 C:\DEV\tdd-course\demo
강사 PC 화면을 본다 · 따라 하지 않는다

강사 시연 ① 미니 체험 — add 로 RED → GREEN → REFACTOR

교육생은 보기만 한다. 같은 과정을 14:00 실습 1 에서 직접 한다. (폴더: demo/ — 강사용)

▶ ① RED — 테스트만 먼저

tests/test_calc.py 에 test_add 테스트만 작성해줘.
src/calc.py 는 NotImplementedError 스텁으로 둬.

▶ ② GREEN — 최소 구현

방금 테스트를 통과시키도록 add(a, b) 를 가장 단순하게 구현해줘. 테스트는 수정하지 마.

▶ ③ REFACTOR — 동작 불변

동작은 바꾸지 말고 타입 힌트와 docstring 만 정리해줘. 끝나면 **!pytest -q 결과를 보여줘.**

tests/test_calc.py

```
from calc import add


def test_add():
    assert add(2, 3) == 5
```

src/calc.py — GREEN

```
def add(a, b):
    return a + b
```

src/calc.py — REFACTOR

```
def add(a: float, b: float) -> float:
    """두 수의 합을 돌려준다."""
    return a + b
```

RED

```
> pytest -q --tb=short
F  [100%]
E   NotImplementedError
1 failed in 0.01s

# RED 직후
git add tests/test_calc.py
```

GREEN

```
> pytest -q
.  [100%]
1 passed in 0.01s

# GREEN 구현 후
git diff -- tests/   # 출력 없음
git diff -- src/     # 구현 변경만 표시
```

여전히 GREEN

```
> pytest -q
.  [100%]
1 passed in 0.01s
```

원칙 시연 포인트: ① RED 의 실패 이유가 NotImplementedError(아직 없음)인지 읽어 준다 ② diff 를 화면에 띄우고 tests/ 가 바뀌지 않았는지 확인한다 ③ 리팩터링 후에도 같은 1 passed.

보기만 C:\DEV\tdd-course\demo
강사 PC 화면을 본다 · 따라 하지 않는다

강사 시연 ② 경계값 — divide(1, 0) 을 테스트로 고정한다

2장의 Right-BICEP 'E(Error)' 와 CORRECT 'R(Range)' 를 한 번에 보여 준다.

▶ ④ 경계값 RED

tests/test_calc.py 에 divide(1, 0) 이 ZeroDivisionError 를 던지는지 검증하는 테스트만 추가해줘. src/ 는 수정하지 마.

▶ ⑤ GREEN

!pytest -q 실패 로그를 보고, 방금 테스트만 통과시키는 최소 구현을 해줘. 다른 기능은 추가하지 마.

tests/test_calc.py

```python
import pytest

from calc import add, divide


def test_add():
    assert add(2, 3) == 5


def test_divide_by_zero_raises():
    with pytest.raises(ZeroDivisionError):
        divide(1, 0)
```

RED — divide 는 아직 스텁

```
> pytest -q --tb=short
.F  [100%]
E   NotImplementedError
1 failed, 1 passed in 0.01s
```

```python
def divide(a: float, b: float) -> float:
    return a / b
```

```
2 passed in 0.01s
```

핵심 a / b 는 b 가 0 이면 이미 ZeroDivisionError 를 던진다 — 테스트는 그 '약속'을 고정한다. 누군가 0 을 0 으로 처리하도록 바꾸면 이 테스트가 잡는다.

이틀 동안 반복해 쓸 프롬프트 세 가지

▶ GREEN — 테스트 통과

@tests/test_<모듈>.py **@src/**<모듈>.py
<테스트 이름> 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마.
끝나면 !pytest -q --tb=short 결과를 보여줘.

▶ REFACTOR — 동작 불변 정리

@src/<모듈>.py 모든 테스트가 통과하는 상태야.
동작은 바꾸지 말고 <정리할 것 하나> 만 해줘.
테스트 파일은 수정하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

▶ REVIEW — 수정 없이 검토 (plan 모드)

@tests/test_<모듈>.py 테스트를 검토만 해줘. 파일은 수정하지 마.
관점: 테스트 이름, AAA 구조, 한 테스트에 한 규칙, 빠진 경계값.
표로: 테스트 | 문제 | 제안

TIP 꺾쇠 < > 부분만 바꿔서 쓴다. 템플릿을 메모장에 붙여 두면 실습 속도가 두 배가 된다.

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

14:00–14:50

실습 1 — 이렇게 진행한다 (R1 ~ R4, 한 사이클 6단계)

폴더
`calc/`

방식
개별 실습

결과물
tests/test_calculator.py
테스트 4개 · 4 passed

프롬프트
calc/prompts.md · Day 1 · 5교시

#	어디서	할 일	완료 기준	슬라이드
1	VS Code	tests/test_calculator.py 에 테스트 1개 작성	저장	87
2	터미널 ②	pytest -q --tb=short	RED · 실패 이유가 '아직 없음'	87
3	Gemini	GREEN 프롬프트 (테스트 이름만 바꿔 붙여넣기)	—	88
4	Gemini	diff 읽기 → src/ 만 바뀌면 승인 · tests/ 면 '아니요'	src/ 만 변경	88
5	터미널 ②	pytest 로 전체 확인	R1 1 · R2 2 · R4 4 passed	88–92
6	계속 진행	다음 R 로	R1 ~ R4 완료	89–92

TIP R1 은 13:00 강사 시연(add)과 같은 과정이다. RED 가 import 오류라면 src/calculator.py 의 스텁(NotImplementedError)이 있는지 먼저 확인.

실습 · 직접 하기 Day 1 · 5교시 | 실습 1

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

14:00–14:50

실습 1 — 문자열 덧셈 계산기

add("1,2") → 3 처럼, 쉼표로 구분된 숫자 문자열의 합을 돌려주는 함수.

번호	요구사항	예	테스트 이름
R1	빈 문자열이면 0	add("") → 0	test_empty_string_returns_zero
R2	숫자 하나면 그 숫자	add("7") → 7	test_single_number_returns_itself
R3	쉼표로 구분된 두 숫자의 합	add("1,2") → 3	test_two_numbers_separated_by_comma
R4	숫자가 여러 개여도 합	add("1,2,3,4") → 10	test_many_numbers

시작 상태 — src/calculator.py

```
def add(numbers: str) -> int:
    raise NotImplementedError
```

진행 규칙

- 요구사항은 한 번에 하나씩 — R1이 GREEN이 되기 전에 R2 테스트를 쓰지 않는다
- AI 결과가 '너무 단순'해 보여도 괜찮다 — 다음 테스트가 더 나은 코드를 요구한다

원칙 13:00 강사 시연과 같은 과정으로 R1 부터 한 사이클씩 진행한다. 각 사이클 끝에 "방금 RED 는 올바른 이유로 실패했나?"를 확인한다.

지금 할 일 calc/ 에서 R1 부터 한 줄씩: 테스트 → RED → 프롬프트 → GREEN

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R1 · 빈 문자열이면 0 — 테스트부터

① 직접 작성— tests/test_calculator.py

```python
from calculator import add


def test_empty_string_returns_zero():
    assert add("") == 0
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
F  [100%]
=== FAILURES ===
___ test_empty_string_returns_zero ___
tests/test_calculator.py:5: in test_empty_string_returns_zero
    assert add("") == 0
           ^^^^^^^
src/calculator.py:2: in add
    raise NotImplementedError
E   NotImplementedError
=== short test summary info ===
FAILED tests/test_calculator.py::test_empty_string_returns_zero
1 failed in 0.02s
```

핵심 NotImplementedError로 실패 = '아직 구현이 없어서' 실패. 올바른 이유의 RED.

실습 · 직접 하기 Day 1 · 5교시 | 실습 1 · R1

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R1 · AI에게 최소 구현을 맡긴다

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
test_empty_string_returns_zero 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ test_empty_string_returns_zero만 통과하도록
  최소 구현을 하겠다.
  Edit src/calculator.py
-     raise NotImplementedError
+     return 0
✦ 테스트 파일은 수정하지 않았다.
  1 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
.  [100%]
1 passed in 0.02s
```

핵심 'return 0'은 속임수가 아니라 정답. 지금 테스트가 요구하는 것은 그것뿐이고, 다음 테스트(R2)가 더 많은 것을 요구한다.

지금 할 일 Gemini: 프롬프트 붙여넣기 ▸ diff 읽고 승인 (tests/ 수정이면 '아니요') ▸ 터미널: 1 passed 확인

실습 · 직접 하기 Day 1 · 5교시 | 실습 1 · R2

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R2 · 숫자 하나 입력 — 해당 숫자 반환

① 직접 작성— tests/test_calculator.py

```
from calculator import add


def test_empty_string_returns_zero():
    assert add("") == 0


def test_single_number_returns_itself():
    assert add("7") == 7
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
.F  [100%]
=== FAILURES ===
___ test_single_number_returns_itself ___
tests/test_calculator.py:9: in test_single_number_returns_itself
    assert add("7") == 7
E   AssertionError: assert 0 == 7
E    +  where 0 = add('7')
=== short test summary info ===
FAILED tests/test_calculator.py::test_single_number_returns_itself
1 failed, 1 passed in 0.02s
```

핵심 R1 테스트는 여전히 통과(.), R2만 실패(F). 'assert 0 == 7' — return 0이 이제는 부족하다는 뜻이다.

지금 할 일 VS Code: R2 테스트 추가 ▸ 터미널: pytest ▸ assert 0 == 7 로 1 failed, 1 passed 확인

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R2 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
test_single_number_returns_itself 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 빈 문자열은 0, 그 외에는 정수로 변환하겠다.
  Edit src/calculator.py
-     return 0
+     if numbers == "":
+         return 0
+     return int(numbers)
✦ 2 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
..  [100%]
2 passed in 0.02s
```

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R3·R4 · 쉼표로 구분된 여러 숫자

① 직접 작성— tests/test_calculator.py

```python
from calculator import add


def test_empty_string_returns_zero():
    assert add("") == 0


def test_single_number_returns_itself():
    assert add("7") == 7


def test_two_numbers_separated_by_comma():
    assert add("1,2") == 3


def test_many_numbers():
    assert add("1,2,3,4") == 10
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
..FF  [100%]
=== FAILURES ===
___ test_two_numbers_separated_by_comma ___
tests/test_calculator.py:13: in test_two_numbers_separated_by_comma
    assert add("1,2") == 3
           ^^^^^^^^^^
src/calculator.py:4: in add
    return int(numbers)
…
=== short test summary info ===
FAILED tests/test_calculator.py::test_two_numbers_separated_by_comma
FAILED tests/test_calculator.py::test_many_numbers - ValueError...
2 failed, 2 passed in 0.02s
```

핵심 같은 규칙(쉼표 구분)의 두 예이어서, 두 테스트를 함께 쓴다 —실패 이유: int('1,2')는 숫자가 아니다.

실습 · 직접 하기 Day 1 · 5교시 | 실습 1 · R3·R4

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R3·R4 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
test_two_numbers_separated_by_comma, test_many_numbers 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 쉼표로 나눈 뒤 각 값을 더하겠다.
  Edit src/calculator.py
-     return int(numbers)
+     return sum(int(n) for n in numbers.split(","))
✦ 4 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
....  [100%]
4 passed in 0.02s
```

지금 할 일 Gemini: 프롬프트 ▸ diff 승인 ▸ 터미널: 4 passed ▸

R4 후— 무엇이 달라졌나

src/calculator.py — 지금까지의 결과

```python
def add(numbers: str) -> int:
    if numbers == "":
        return 0
    return sum(int(n) for n in numbers.split(","))
```

테스트 4개 — 매번 전부 다시 실행됐다

터미널

```
> pytest -q --tb=short
....  [100%]
4 passed in 0.02s
```

실험 A와 비교

- 출력 확인: 사람이 숫자를 확인 → 테스트가 기대값과 자동 비교
- 확인 범위: 선택한 예제 하나 → 저장된 모든 테스트
- AI 지시: 막연한 기능 요청 → 테스트를 만족하는 구체적인 요청
- 요구사항 관리: 사람의 기억에 의존 → 중요한 규칙을 테스트로 기록
- 변경 후 확인: 예제를 하나씩 재실행 → pytest로 전체 확인

- 핵심은 테스트가 정답을 스스로 정하는 것이 아니라, 사람이 정한 정답을 저장해 두고 반복해서 자동 확인한다는 것

핵심 점 네 개(....)가 지금까지 확인한 요구사항. 다음 세션에서 요구사항이 바뀔 때, 이 점들이 무엇을 해 주는지 보게 됨.

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

15:00–15:50

실습 2 —진행 내용 (요구사항이 바뀔 때)

폴더
`calc/` (실습 1 이어서)

방식
개인

결과물
R5 · R6 테스트 추가
7 passed

프롬프트
calc/prompts.md · Day 1 · 6교시

#	어디서	할 일	완료 기준	슬라이드
1	VS Code	R5 세미콜론 테스트 추가	저장	96
2	터미널 ②	pytest	2 failed, 4 passed	96
3	Gemini	일부러 짧은 프롬프트 ('평소처럼')	—	97
4	터미널 ②	pytest — 기존 테스트가 깨진 것 확인	3 failed (정상)	97
5	Gemini	회귀 복구 프롬프트	6 passed	98
6	VS Code → Gemini	R6 음수 오류: 테스트 → RED → GREEN	7 passed	99–100

TIP 3단계에서 실패하는 것이 이 실습의 목적. 당황하지 말고 4단계에서 '몇 개가, 왜' 깨졌는지 기록하며 진행.

실습 · 직접 하기 Day 1 · 6교시 | 실습 2

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

15:00–15:50

실습 2 — 요구사항이 바뀔 때

개인 실습. 이번에는 일부러 프롬프트를 '평소처럼' 작성한다.

번호	새 요구사항	예	테스트 이름
R5	세미콜론(;)도 구분자로 쓸 수 있다 · 섞어 써도 된다	add("1;2") → 3 add("1,2;3") → 6	test_semicolon_… test_mixed_delimiters
R6	음수가 있으면 ValueError, 메시지에 음수 목록	add("-1,2,-3") → 오류 '음수는 허용하지 않는다: -1, -3'	test_negative_numbers_…

이번 세션에서 확인할 것

- R5를 짧게 요청하면 AI가 기존 기능을 깨뜨릴 수 있다
- 그때 기존 테스트가 무엇을 하는가
- '회귀(regression)'라는 이름을 붙인다

실험 A와 같은 상황

- A-4에서 '50000원 이하는 할인 안 되게' 요청이 5만 원 경계를 깨뜨렸다
- 그때는 아무도 몰랐다 — 이번에는?

TIP 구분자 처리 코드를 일부러 잘못 바꿔 테스트가 실패한 결과가, 처음부터 테스트 통과하면 split(",")을 split(";")로 바꿔 오류를 만든 뒤, 테스트가 실패하는지 확인한다.

지금 할 일 calc/ 그대로 · 개인 실습 · R5 는 일부러 '평소처럼' 짧은 프롬프트로 시작한다

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R5 · 세미콜론 구분자 — 테스트부터

① 직접 작성— tests/test_calculator.py

```
from calculator import add
…  (R1~R4 테스트 그대로)


def test_semicolon_is_also_a_delimiter():
    assert add("1;2") == 3


def test_mixed_delimiters():
    assert add("1,2;3") == 6
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
....FF  [100%]
=== FAILURES ===
___ test_semicolon_is_also_a_delimiter ___
tests/test_calculator.py:21: in test_semicolon_is_also_a_delimiter
    assert add("1;2") == 3
           ^^^^^^^^^^
…
=== short test summary info ===
FAILED tests/test_calculator.py::test_semicolon_is_also_a_delimiter
FAILED tests/test_calculator.py::test_mixed_delimiters - ValueE...
2 failed, 4 passed in 0.02s
```

핵심 기존 4개는 통과(....), 새 2개만 실패(FF). 올바른 RED.

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

평소처럼 짧게 요청하면 — AI가 기존 기능을 깨뜨린다

▶ 프롬프트 — 일부러 짧게

@src/calculator.py 세미콜론도 구분자로 쓸 수 있게 바꿔줘.

결과 예시 자주 나오는 실패 유형

Gemini CLI

```
✦ 구분자를 세미콜론으로 변경했다.
  Edit src/calculator.py
-     return sum(int(n) for n in numbers.split(","))
+     return sum(int(n) for n in numbers.split(";"))
```

pytest가 바로 알려 준다

터미널

```
> pytest -q --tb=short
..FF.F  [100%]
=== FAILURES ===
___ test_two_numbers_separated_by_comma ___
tests/test_calculator.py:13: in test_two_numbers_separated_by_comma
…  (실패 3건 상세 생략)
=== short test summary info ===
FAILED tests/test_calculator.py::test_two_numbers_separated_by_comma
FAILED tests/test_calculator.py::test_many_numbers - ValueError...
FAILED tests/test_calculator.py::test_mixed_delimiters - ValueE...
3 failed, 3 passed in 0.02s
```

주의 R5 테스트 중 하나(1;2)는 통과했지만, 어제까지 잘 되던 R3·R4가 깨졌다. 이것이 회귀(regression) — 새 변경이 기존 기능을 되돌려 놓은 것.

핵심 실험 A였다면? 새 기능(1;2 → 3)만 눈으로 확인하고 '완성!'이라고 했을 것.

실습 · 직접 하기 Day 1 · 6교시 | 실습 2 · R5

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

회귀를 고친다 — 기준은 '모든 테스트'

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
방금 변경으로 기존 테스트 3개가 실패해. 쉼표와 세미콜론을 모두 구분자로 지원해서 모든 테스트를 통과시켜줘. 테스트 파일은 수정하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 쉼표와 세미콜론 둘 다로 나누도록
  정규식을 쓰겠다.
  Edit src/calculator.py
+ import re
-     return sum(int(n) for n in numbers.split(";"))
+     return sum(int(n) for n in
+                re.split(r"[,;]", numbers))
✦ 6 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
......  [100%]
6 passed in 0.02s
```

핵심 실패 메시지를 그대로 Context로 주면 AI가 무엇을 고쳐야 할지 정확히 알 수 있다. '모든 테스트 통과'가 완료 기준.

지금 할 일 Gemini: 회귀 복구 프롬프트 ▸ diff 승인 ▸ 터미널: 6 passed

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R6 · 음수는 오류 — 예외도 테스트한다

① 직접 작성— tests/test_calculator.py

```
import pytest
…  (기존 테스트 그대로)


def test_negative_numbers_raise_error():
    message = "음수는 허용하지 않습니다: -1, -3"
    with pytest.raises(ValueError, match=message):
        add("-1,2,-3")
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
......F  [100%]
=== FAILURES ===
___ test_negative_numbers_raise_error ___
tests/test_calculator.py:31: in test_negative_numbers_raise_error
    with pytest.raises(ValueError, match=message):
E   Failed: DID NOT RAISE ValueError
=== short test summary info ===
FAILED tests/test_calculator.py::test_negative_numbers_raise_error
1 failed, 6 passed in 0.02s
```

핵심 DID NOT RAISE — 오류가 나야 하는데 안 났다는 뜻. 예외를 검사할 때는 pytest.raises와 match(메시지)를 쓴다.

실습 · 직접 하기 Day 1 · 6교시 | 실습 2 · R6

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

R6 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
test_negative_numbers_raise_error 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 값을 먼저 정수 목록으로 만든 뒤 음수를 모아
  메시지에 넣겠다.
  Edit src/calculator.py
+     if numbers == "":
+         return 0
+     values = [int(n) for n in re.split(r"[,;]", numbers)]
+     negatives = [v for v in values if v < 0]
+     if negatives:
+         listed = ", ".join(map(str, negatives))
+         raise ValueError(f"음수는 허용하지 않습니다: {listed}")
+     return sum(values)
✦ 7 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
.......  [100%]
7 passed in 0.02s
```

지금 할 일 Gemini: 프롬프트 ▸ diff 승인 ▸ 터미널: 7 passed

설명 · 듣고 이해 Day 1 · 6교시 | 정리

회귀 안전망 — 테스트는 쌓인다

R1 빈 문자열	→	test 1
R2 숫자 하나	→	test 2
R3 쉼표 두 개	→	test 3
R4 여러 개	→	test 4
R5 세미콜론	→	test 5
R6 음수 오류	→	test 6

새 요구사항이 들어올 때마다

새 테스트 1개를 추가하고
기존 테스트는 모두 남겨 둔다

pytest 한 줄 = 지금까지의 모든 요구사항 재확인
AI가 무엇을 바꿔도, 깨지면 즉시 빨간불

원칙 Day 1에서 기억할 한 문장: "AI가 코드를 빨리 만들기 때문에 오히려 테스트라는 안전망이 필요하다."

참고 실험 A의 A-4(5만 원 경계)도 테스트가 있었다면 바로 잡혔을까? — 내일 실험 B에서 직접 확인.

AAA 패턴 — 준비 · 실행 · 검증을 나눈다

✗ 한 줄에 다 넣으면 무엇을 검사하는지 흐려진다

```
def test_add():
    assert add("1,2") == 3 and add("") == 0 \
        and add("1;2") == 3
```

단계	할 일
Arrange	입력과 상태를 준비
Act	검사할 함수를 한 번 호출
Assert	결과를 하나의 규칙으로 확인

✓ Arrange · Act · Assert

```
def test_mixed_delimiters():
    # Arrange — 준비
    numbers = "1,2;3"
    # Act — 실행
    result = add(numbers)
    # Assert — 검증
    assert result == 6
```

테스트 이름은 문장으로

- ✗ test_add2, test_case3
- ✓ test_negative_numbers_raise_error

핵심 실패했을 때 이름만 보고 '무슨 규칙이 깨졌는지' 알 수 있어야 좋은 테스트. 한 테스트에는 한 규칙만.

이어서 사용 C:\DEV\tdd-course\calc
막히면 시작점 checkpoints\1.5_실습2완료\calc

AI에게 테스트를 리뷰받는다 — plan 모드, 수정 없이

▶ 프롬프트 — REVIEW 템플릿

@tests/test_calculator.py 테스트를 검토만 해줘. 파일은 수정하지 마.
관점: 테스트 이름, AAA 구조, 한 테스트에 한 규칙, 빠진 경계값.
표로: 테스트 | 문제 | 제안

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI (plan 모드)

```
✦ 전반적으로 이름이 규칙을 잘 설명한다. 제안 세 가지이다.
  1. test_many_numbers
     문제: '많은'이 몇 개인지 모호  →  제안: test_four_numbers_are_summed
  2. test_negative_numbers_raise_error
     문제: 음수가 하나뿐인 경우가 없음  →  제안: add("-5") 케이스 추가
  3. (없음)
     문제: 공백만 있는 입력이 정의되지 않음  →  제안: 요구사항 확인 후 테스트
```

참고 리뷰 결과를 그대로 반영하지 않는다. '공백만 있는 입력'처럼 요구사항이 정해지지 않은 것은 먼저 사람이 결정

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

점검 실습 — R7 혼자서 한 사이클

R7 공백만 있는 문자열도 0으로 본다. `add("   ") → 0`

5분 테스트 작성 → RED 확인 (왜 실패하는지 한 줄로 적기)

5분 GREEN 템플릿으로 프롬프트 → diff 읽고 승인

5분 전체 테스트 GREEN 확인 → 짝과 결과 비교

TIP 막히면 오늘 슬라이드의 R1~R6 과정을 그대로 따라 한다. 내일 확인 실습도 똑같은 순서.

참고 풀이는 다음 장에. 15분 뒤에 함께 본다.

R7 풀이 — RED에서 GREEN까지

① 테스트 추가

```
def test_whitespace_only_returns_zero():
    assert add("   ") == 0
```

② RED — 실패 이유: int(' ')

터미널

```
> pytest -q --tb=short
.......F  [100%]
=== FAILURES ===
___ test_whitespace_only_returns_zero ___
… (상세 생략)
=== short test summary info ===
FAILED tests/test_calculator.py::test_whitespace_only_returns_zero
1 failed, 7 passed in 0.02s
```

▶ 프롬프트 — Gemini CLI

@tests/test_calculator.py @src/calculator.py
test_whitespace_only_returns_zero 만 통과하는 최소 구현을 해줘. 테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시

Gemini CLI

```
✦ 공백을 지운 뒤 비어 있는지 확인하겠다.
  Edit src/calculator.py
-     if numbers == "":
+     if numbers.strip() == "":
```

③ GREEN

```
> pytest -q --tb=short
........  [100%]
8 passed in 0.02s
```

핵심 한 줄 바뀌었지만 테스트 8개가 모두 다시 확인 됨. 이 감각이 내일 장바구니에서도 그대로 쓰인다.

골든 마스터 — '지금 동작'을 통째로 고정하는 안전망

R1~R7 테스트가 '이래야 한다'라면, 골든 마스터는 '지금은 이렇다'를 파일로 저장해 둔다.

1 입력 목록을 정한다
빈 값 · 경계 · 오류 입력까지 넉넉하게

2 실행해서 저장한다
결과를 골든 파일(JSON)로 저장

3 사람이 읽고 승인한다
diff 를 읽고 '이게 맞는 현재 동작인가' 판단

4 이후 매번 비교한다
결과가 달라지면 즉시 FAIL

	명세 테스트 (R1 ~ R7)	골든 마스터 (특성화 테스트)
기대값의 출처	요구사항 · 예시 표	지금 코드의 실제 출력
의미	"이래야 한다"	"지금은 이렇다" — 옳은지는 별개
쓰는 때	새 기능을 TDD 로 만들 때	리팩터링 · 큰 변경(AI 수정) 직전의 안전망
갱신	요구사항이 바뀔 때	사람이 diff 를 읽고 승인할 때만

주의 골든은 정답이 아니라 '현재 답'이다 — 버그도 함께 굳는다. 예) add("1,2,") 가 오류인 것이 의도인지는 사람이 결정.

핵심 오늘은 연습, 내일은 실전 — 2일차 장바구니·확인 실습에서 리팩터링 직전마다 골든 마스터를 먼저 만든다.

이어서 사용 C:\DEV\tdd-course\calc
막히면 시작점 checkpoints\1.7_점검실습완료\calc

16:00–16:50

골든 마스터 작성 프롬프트 — 입력 목록과 비교 테스트를 AI 에게

▶ 프롬프트 — Gemini CLI

@src/calculator.py @tests/test_calculator.py
add() 의 '현재 동작'을 골든 마스터로 고정하는 테스트를 tests/test_golden_master.py 에 만들어줘.
- 입력 16개: 빈 문자열 · 공백만 · 숫자 하나 · 쉼표 · 세미콜론 · 혼합 · 음수 · 잘못된 값(a,b) · 빈 항목(1,,2 / 1,2,)
- 각 입력의 결과(정수) 또는 오류 종류(ValueError)를 tests/golden/add.golden.json 에 저장
- 골든 파일이 있으면 비교만, UPDATE_GOLDEN=1 일 때만 갱신. 골든이 없으면 FAIL
- src/ 와 기존 테스트는 수정하지 마. 끝나면 !pytest -q 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

터미널

```
> pytest -q --tb=line
E   Failed: 골든 파일이 없다 — UPDATE_GOLDEN=1 로 만든 뒤 …
FAILED tests/test_golden_master.py::test_golden_master
1 failed, 8 passed in 0.12s

> UPDATE_GOLDEN=1 pytest -q tests/test_golden_master.py
1 passed in 0.02s                  # PowerShell: $env:UPDATE_GOLDEN=1

> pytest -q
9 passed in 0.02s

✦ 골든 파일을 읽고 승인한 뒤 커밋한다  (git add tests/golden)
```

읽을 것

```
// tests/golden/add.golden.json (일부)
"":        {"result": 0},
"   ":     {"result": 0},
"1,2;3":   {"result": 6},
"-1,2,-3": {"error": "ValueError"},
"a,b":     {"error": "ValueError"},
"1,,2":    {"error": "ValueError"},   # ← 의도?
"1,2,":    {"error": "ValueError"},   # ← 의도?
...
```

골든 파일을 '읽는 일'이 핵심: 빈 항목(1,,2)이 오류인 것은 요구사항인가, 우연인가? 결정은 사람이 한다.

실습 · 직접 하기 Day 1 · 7교시 | 골든 마스터

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

16:00–16:50

골든 마스터가 일하는 장면 — 일부러 망가뜨려 보기

실습: 코드를 한 줄 망가뜨려 테스트가 잡는지 본다 (확인 후 원래대로 되돌림).

src/calculator.py 한 줄 수정

diff

```
-     if not numbers.strip():
+     if not numbers:
```

무엇이 달라지나

- R7 단위 테스트 1개 → "공백만 있으면 0" 한 가지만 알려 준다
- 골든 마스터 → 16개 입력 중 바뀐 입력을 모두 보여 준다
- diff 한 줄로 어떤 입력이 어떻게 변했는지 바로 보인다
- 골든은 R1~R7 이 미처 쓰지 못한 입력(1,,2 · a,b …)의 변화까지 지킨다

터미널 — 실제 출력

터미널

```
> pytest -q --tb=short
.......FF  [100%]
=== FAILURES ===
___ test_whitespace_only_returns_zero ___
E   ValueError: invalid literal for int() with base 10: '    '
___ test_golden_master ___
E   AssertionError: assert {'': {'result...ult': 7}, ...} == {...}
E     Differing items:
E     {'    ': {'error': 'ValueError'}} != {'    ': {'result': 0}}
=== short test summary info ===
FAILED tests/test_calculator.py::test_whitespace_only_returns_zero
FAILED tests/test_golden_master.py::test_golden_master
2 failed, 7 passed in 0.82s

✦ 되돌린 뒤:
> pytest -q
9 passed in 0.02s
```

핵심 골든 마스터 = 입력을 많이 넣은 '결과 사진'. 내일 리팩터링 · 확인 실습 전에 이 안전망부터 만든다.

지금 할 일 VS Code: if not numbers.strip() → if not numbers ▸ pytest → 2 failed ▸ 되돌리기 ▸ 9 passed

실습 · 직접 하기 Day 1 · 7교시 | 보너스

이어서 사용 C:\DEV\tdd-course\calc
위에서 연 폴더 그대로 · 프롬프트 calc\prompts.md

보너스 — GREEN 상태에서 리팩터링 맛보기

시간이 남는 사람만. 내일 14:00 (6교시 리팩터링)에 정식으로 진행.

▶ 프롬프트 — REFACTOR 템플릿

@src/calculator.py 모든 테스트가 통과하는 상태야. 동작은 바꾸지 말고 함수를 읽기 쉽게 작은 함수로 나눠줘.
테스트 파일은 수정하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

```
import re

DELIMITERS = r"[,;]"


def add(numbers: str) -> int:
    if not numbers.strip():
        return 0
    values = _parse(numbers)
    _reject_negatives(values)
    return sum(values)


def _parse(numbers: str) -> list[int]:
    return [int(n) for n in re.split(DELIMITERS, numbers)]


def _reject_negatives(values: list[int]) -> None:
    negatives = [v for v in values if v < 0]
    if negatives:
        listed = ", ".join(map(str, negatives))
        raise ValueError(f"음수는 허용하지 않습니다: {listed}")
```

터미널

```
> pytest -q --tb=short
.........  [100%]
9 passed in 0.02s
```

확인할 것

- 테스트 9개(골든 마스터 포함)가 그대로 통과
- 테스트 파일은 바뀌지 않았다
- 새 기능이 추가되지 않았다

지금 할 일 (선택) Gemini: REFACTOR 프롬프트 ▸ 터미널: 9 passed 유지 · tests/ 가 바뀌지 않았는지 확인

Day 1 회고 — 한 문장으로

"테스트가 오늘 나를 구한 순간은?"

오늘 한 것

- 실험 A — 눈으로 확인의 한계
- RED → GREEN 7바퀴 (R1~R7) · 골든 마스터
- 회귀를 테스트가 잡는 장면

오늘 쓴 도구

- pytest -q --tb=short
- @파일 · !명령 · plan 모드
- GREEN · REFACTOR · REVIEW 템플릿

내일 할 것

- 애매한 요구사항 → 예시 표
- 장바구니 TDD + 경계값
- 실험 B · 확인 실습

참고 과제(선택): 오늘 만든 calc 폴더(골든 마스터 포함)를 git에 커밋해 두세요. 이어지는 복습 퀴즈는 오늘 슬라이드에서만 나온다.

Day 1 복습 퀴즈 — 5문항

Q1 새 테스트를 쓰고 실행했더니 기존 테스트까지 실패했다. 이것은 올바른 RED인가?

Q2 AI에게 GREEN을 맡길 때 프롬프트에 꼭 넣는 '금지' 한 줄은?

Q3 'return 0'처럼 너무 단순한 구현은 잘못된 것인가?

Q4 R5에서 세미콜론을 추가했더니 R3·R4가 깨졌다. 이 현상의 이름은?

Q5 pytest 출력 'E   assert 0 == 7'에서 기대값은 무엇인가?

정답과 한 줄 해설

Q1	**아니다**	기존 테스트까지 실패하면 무언가 깨진 것. 올바른 RED는 '새 테스트만' 실패
Q2	**테스트 파일은 수정하지 마**	AI가 기대값을 바꿔 통과시키는 것을 막는다
Q3	**아니다**	지금 테스트가 요구하는 만큼이면 정답. 다음 테스트가 더 요구한다
Q4	**회귀 (regression)**	새 변경이 기존 기능을 되돌려 놓은 것 — 기존 테스트가 잡아냈다
Q5	**7**	assert 실제 == 기대 → 실제 0, 기대 7

D2

DAY 2

TDD 사용 실습

애매한 요구사항에서 사양을 찾아 예시 표로 정리하고, 장바구니를 TDD로 완성한다. 경계값·AI 규칙 파일·불변식·리팩터링을 거쳐, 어제의 실험을 TDD로 다시 풀고 확인 실습으로 마무리.

08:30 – 17:30

왜 '사양 찾기'가 먼저인가 — AI 는 모르면 추측한다

잘못된 가정으로 코드와 테스트를 함께 작성하면, 테스트로도 오류를 발견하기 어렵다.

① 모호한 요구

- "VIP 고객에게 할인 적용"
- 몇 %? 5만 원 포함? 중복 할인? 순서? 단수?
- 사람마다 머릿속 답이 다르다

② AI 에게 그대로 맡기면

- 물어보지 않고 '그럴듯한 값'을 채워 넣는다
- float 금액 · 반올림 · >= 와 > …
- 코드는 잘 돌아가서 틀린 줄 모른다

③ TDD 에서는

- 테스트를 쓰려면 기대값이 필요하다
- 기대값 = 사양 (코드가 아니라 사양에서)
- 못 쓰겠다 = 사양이 없다는 신호

1 모호한 요구
한 줄짜리 요구사항

2 질문한다
테스트를 쓰다 막힌 곳을 모은다

3 예시 표로 확정
규칙 + 입력 → 기대 결과

4 테스트 → AI 구현
예시 한 줄 = 테스트 하나

핵심 사양 찾기는 코딩 전에 '무엇이 맞는지'를 정하는 일 — 테스트의 기대값은 코드나 AI 의 답이 아니라 사양에서 와야 한다.

원칙 AI 는 질문을 잘 찾아 주지만 결정권은 없다. 결정은 사람(고객 · 기획자)이 하고 예시 표에 남긴다.

사양이 없으면 생기는 일 — 어제 실험 A 에서 이미 겪었다

어제 나온 질문은 모두 사양만으로 판단하기 어려운 내용이었다.

실험 A 에서	숨어 있던 질문	AI 가 몰래 정한 것	사양(예시 표)으로 정하면
A-2 54000.0	금액은 정수인가? 단수는 버림? 반올림?	float 로 계산	C5 원 미만 버림 · 정수
A-4 >= → >	정확히 50,000원도 할인 대상인가?	문장의 '이하'를 글자 그대로	C3 50,000원 포함(이상)
A-3 VIP	VIP 할인은 어느 금액에 먼저 적용?	순서를 임의로	C4 문턱 → VIP 순서
A-1 빈 장바구니	0원인가? 오류인가?	0 으로 처리	C1 0원

참고 A-4 의 프롬프트 "50000원 이하는 할인 안 되게"는 A-2 의 "5만 원 이상 할인"과 정확히 50,000원에서 충돌한다 — 사양이 없으니 아무도 몰랐다.

사양 찾기 3단계 (오늘 1교시)

1 막힌 질문 모으기
테스트를 5분 써 보다가

2 고객에게 묻기
Gemini 가 '고객' 역할

3 예시 표로 확정
C1 ~ C7 + 범위 밖

제공 폴더 C:\DEV\tdd-course\cart
starter\cart 복사 (docs\ 포함) · 프롬프트 cart\prompts.md

08:30–09:20

사양 찾기 — 이렇게 진행한다 (질문 → 예시 표)

폴더
`cart/`

방식
개별 실습

결과물
cart/docs/예시표.md
(C1 ~ C7)

프롬프트
cart/prompts.md · Day 2 · 1교시

#	어디서	할 일	완료 기준	슬라이드
1	VS Code	tests/test_cart.py 에 5분간 테스트 써 보기	막힌 곳 메모	117
2	질문	막힌 질문을 포스트잇에 모아 저장	질문 목록	117
3	Gemini에게 질문	'고객'(Gemini)에게 질문하기	답 받아 적기	118
4	VS Code	답을 docs/예시표.md 로 정리	C1 ~ C7	119
5	Gemini(plan)	예시 표를 붙여넣고 빈틈 질문 받기	질문 목록	120
6	모둠	결정 → 예시 표에 한 줄 추가	표 갱신	120

TIP '고객' 역할 Gemini. 묻지 않은 것은 알려 주지 않으니 질문을 구체적으로 만든다.

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

"VIP 고객에게 할인 적용" — 이것만으로 테스트

요구사항 VIP 고객에게 할인을 준다. 5만 원 이상 사면 할인해 준다.

5분 동안 test_cart.py에 테스트를 써 보면 — 예시 질문들

할인율은?
VIP는 몇 %? 5만 원 할인은 몇 %?

중복 할인?
VIP이면서 5만 원 이상이면 둘 다?

적용 순서?
어느 할인을 먼저? 기준 금액은?

경계는?
정확히 50,000원은 포함인가?

단수 처리?
원 미만은 버림? 반올림?

이상한 입력?
빈 장바구니, 음수 가격은?

핵심 테스트를 못 쓰겠다 = 사양이 없다는 신호. 사양은 처음부터 외우는 것이 아니라, 테스트를 쓰다 막혔을 때 '발견'하는 것.

참고 질문을 모아 포스트잇에 적고, 10분 뒤 Gemini에게 고객 역할을 맡겨 질문한다

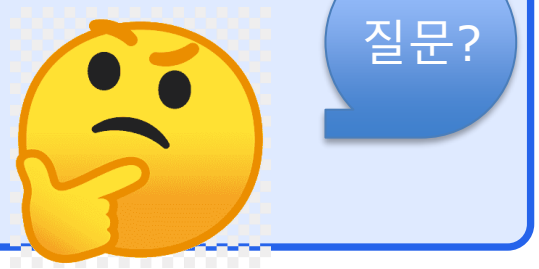
토론 · 짝 활동 Day 2 · 1교시 | 사양 찾기

Gemini에게 고객 역할을 맡겨 질문한다

실제로는 기획자·고객에게 묻는 일. 지금은 Gemini가 '정답을 정해 둔 고객' 역할을 한다.

▶ 프롬프트 — 고객 역할극 (강사가 미리 준비)

지금부터 너는 온라인 쇼핑몰 운영자야. 내가 할인 규칙을 물어보면
아래 정답만 근거로 짧게 대답해. 묻지 않은 것은 먼저 말하지 마.
정답: 합계 50,000원 이상이면 10% 할인(50,000 포함).
VIP는 그 다음에 남은 금액에서 5% 추가 할인. 원 미만은 버림.
빈 장바구니는 0원. 가격·수량이 음수면 오류.
첫 질문을 기다려.



- '정답' 부분을 참조하지 않고 질문만 한다
- 질문을 안 하면 답도 없다 — 질문 목록이 곧 사양

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI — 대화

```
> 5만 원 할인은 몇 %예요?
✦ 10%이다.
> 정확히 50,000원도 할인되나요?
✦ 네, 50,000원부터 할인된다.
> VIP이면서 5만 원 넘으면 둘 다 받나요?
✦ 네. 10% 할인 후 남은 금액에서 5% 더 할인한다.
> 원 미만은요?
✦ 버린다.
> 쿠폰도 있나요?
✦ 그건 이번 범위가 아니다.
```

핵심 '쿠폰은 범위 밖' 같은 대답도 사양. 하지 않을 것을 정해 두면 AI가 엉뚱한 기능을 덧붙이는 것을 막을 수 있다.

지금 할 일 모둠 질문을 강사 화면의 '고객'(Gemini)에게 묻기 ▸ 답을 받아 적기 — 묻지 않은 것은 알려 주지 않는다

실습 · 직접 하기 Day 2 · 1교시 | 예시 표

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

질문의 답을 예시 표로 — 이것이 우리의 사양

규칙 문장 + 입력 → 기대 결과 예시. 예시 한 줄이 곧 테스트 하나가 된다.

ID	규칙	입력 (장바구니 · VIP)	기대 결과
C1	빈 장바구니는 0원	[] · 일반	0
C2	합계 = Σ 가격 × 수량	12,000×3 + 24,000×1 · 일반	합계 60,000
C3	50,000원 이상이면 10% 할인 (포함)	49,999 / 50,000 / 50,001	49,999 / 45,000 / 45,000
C4	VIP는 C3 다음에 5% 추가 할인	60,000 · VIP	60,000 → 54,000 → 51,300
C5	원 미만은 버림	50,001 · 일반	45,000 (45,000.9 → 버림)
C6	결제 금액은 0 이상, 합계 이하	어떤 장바구니든	0 ≤ 결제 금액 ≤ 합계
C7	가격·수량이 음수면 오류	-1,000 × 1	ValueError
—	범위 밖: 쿠폰 · 배송비 · 포인트		만들지 않는다

참고 C6은 특정 예가 아니라 '항상 지켜야 하는 성질' — 11:10 장바구니 TDD 2 에서 불변식(invariant)이라는 이름을 붙인다.

지금 할 일 VS Code: 받아 적은 답을 cart/docs/예시표.md 에 C1 ~ C7 형식으로 정리 (예시 한 줄 = 테스트 하나) ▸ 이 파일이 오늘의 기준 사양

실습 · 직접 하기 Day 2 · 1교시 | 예시 표 점검

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

AI에게 예시 표의 빈틈을 묻는다 — 답은 사람이 정한다

▶ 프롬프트 — plan 모드

아래 할인 규칙 예시 표를 검토만 해줘. 새 규칙을 만들지 말고, 빠진 경계값이나 서로 충돌하는 규칙만 질문 형태로 알려줘.
(예시 표 C1~C7 붙여넣기)

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI (plan 모드)

✦ 표가 명확하다. 확인이 필요해 보이는 질문이다.
1. C4: VIP인데 합계가 50,000원 미만이면 5% 할인만 받나요? (예: 40,000 · VIP → 38,000?)
2. C3·C5: 50,001원의 10% 할인 결과 45,000.9를 버리면 45,000 — 표와 일치한다. 다른 금액도 같은 규칙인가요?
3. C7: 수량이 0인 상품은 허용하나요?

결정 예 – 짝 검토 후 결정

- 1 → 예, VIP만 적용 (예시 한 줄 추가)
- 3 → 수량 0은 허용 (합계에 0원)

원칙 AI는 질문을 잘 찾지만 결정권은 없다. 결정한 답은 예시 표에 한 줄 추가.

지금 할 일 Gemini(plan 모드): 예시 표 붙여넣고 질문 받기 ▸ 짝 검토 후 결정 ▸ 예시 표에 한 줄 추가

이어서 사용 C:\DEV\tdd-course\cart
막히면 시작점 checkpoints\2.1_사양찾기완료\cart

09:30–10:20

장바구니 TDD 1 — 진행 방식(L1 · L2)

폴더
`cart/`

방식
개인

결과물
L1 · L2 테스트 · 5 passed
RED 캡처 2장

프롬프트
cart/prompts.md · Day 2 · 2교시

#	어디서	할 일	완료 기준	슬라이드
1	VS Code	docs/예시표.md(1교시 결과) · src/cart.py 스텁 확인	C1~C7 · 함수 3개	122
2	VS Code → 터미널	L1: C1 · C2 테스트 → pytest	2 failed · 캡처	123
3	Gemini	L1 프롬프트 → diff 승인	2 passed	124
4	VS Code → 터미널	L2: parametrize 경계 세 점 → pytest	3 failed · 캡처	126
5	Gemini	L2 프롬프트 → diff 승인	5 passed	127
6	VS Code → 터미널	>= 를 > 로 바꿔 보기 → 되돌리기	50000 만 실패 → 5 passed	128

TIP docs/예시표.md 가 없으면 checkpoints/2.1_사양찾기완료 의 것을 복사한다. 테스트 이름이나 주석에 예시 표 ID(C1 ~ C7)를 적어 두면 실패했을 때 어떤 규칙인지 바로 알 수 있다.

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

09:30–10:20

장바구니 TDD 1 — 오늘의 진행 순서

cart/ 폴더

```
cart/
├ pytest.ini              # 어제와 동일
├ docs/예시표.md          # 1교시에 만든 사양 (C1 ~ C7)
├ src/cart.py             # 스텁으로 시작
└ tests/test_cart.py      # 예시 표 → 테스트
```

루프	예시 표	함수
L1	C1 · C2	`subtotal`
L2	C3 · C5 (경계)	`apply_threshold_discount`
L3	C4 (VIP)	`final_total`
L4	C6 · C7	불변식 · 오류

어제와 달라진 점

- 테스트의 기대값은 예시 표에서 온다 — 코드나 AI의 답에서 오지 않는다
- 테스트 이름이나 주석에 예시 표 ID(C1~C7)를 적어 두면, 실패했을 때 어떤 규칙인지 바로 안다

참고 09:30 에 L1·L2, 11:10 에 L3·L4를 한다. 루프마다 RED 캡처를 남긴다 — 확인 실습에서 같은 방식으로 작업.

실습 · 직접 하기 Day 2 · 2교시 | L1 · C1·C2

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L1 · 합계 — 예시 표 C1·C2를 그대로 테스트로

① 직접 작성— tests/test_cart.py

```
from cart import subtotal


def test_empty_cart_subtotal_is_zero():
    assert subtotal([]) == 0


def test_subtotal_sums_price_times_qty():
    items = [{"price": 12000, "qty": 3},
             {"price": 24000, "qty": 1}]
    assert subtotal(items) == 60000
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
FF  [100%]
=== FAILURES ===
___ test_empty_cart_subtotal_is_zero ___
tests/test_cart.py:5: in test_empty_cart_subtotal_is_zero
    assert subtotal([]) == 0
…
=== short test summary info ===
FAILED tests/test_cart.py::test_empty_cart_subtotal_is_zero - N...
FAILED tests/test_cart.py::test_subtotal_sums_price_times_qty
2 failed in 0.02s
```

핵심 src/cart.py에는 subtotal을 raise NotImplementedError로만 만들어 둔다(스텁). import 오류가 아니라 '구현 없음'으로 실패해야 올바른 RED.

실습 · 직접 하기 Day 2 · 2교시 | L1

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L1 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_cart.py @src/cart.py
test_empty_cart_subtotal_is_zero, test_subtotal_sums_price_times_qty 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 가격 × 수량을 모두 더하겠다.
  Edit src/cart.py
-     raise NotImplementedError
+     return sum(item["price"] * item["qty"]
+                for item in items)
✦ 2 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
..  [100%]
2 passed in 0.02s
```

지금 할 일 Gemini: L1 프롬프트 ▸ diff 승인 ▸ 터미널: 2 passed

경계값 — 50,000원 근처의 세 점

'이상'과 '초과'가 갈리는 곳은 경계 바로 아래 · 정확히 · 바로 위를 짝으로 테스트.



왜 세 점인가

- 49,999 / 50,000 → '>='와 '>'를 구분한다
- 50,001 → 원 미만 버림(C5)까지 함께 확인

실험 A를 기억하나요?

- A-4의 '50000원 이하는 할인 안 되게'가 >= 를 > 로 바꿨다
- 60,000원 예제로는 절대 잡히지 않는다 — 50,000이 필요

핵심 경계값은 사양(예시 표)에서 찾고, 테스트로 '고정'. 이후 누가(사람이든 AI든) 경계를 바꾸면 즉시 빨간불이 켜진다.

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L2 · parametrize로 경계 세 점을 한 번에

① 직접 작성— tests/test_cart.py

```python
import pytest
from cart import subtotal, apply_threshold_discount
…  (L1 테스트 그대로)


@pytest.mark.parametrize("amount, expected", [
    (49999, 49999),   # 문턱 바로 아래: 할인 없음
    (50000, 45000),   # 문턱 정확히: 10% 할인
    (50001, 45000),   # 문턱 바로 위: 10% 할인 (원 단위 버림)
])
def test_threshold_discount_boundary(amount, expected):
    assert apply_threshold_discount(amount) == expected
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
..FFF  [100%]
=== FAILURES ===
___ test_threshold_discount_boundary[49999-49999] ___
tests/test_cart.py:20: in test_threshold_discount_boundary
    assert apply_threshold_discount(amount) == expected
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
…
=== short test summary info ===
FAILED tests/test_cart.py::test_threshold_discount_boundary[49999-
49999]
FAILED tests/test_cart.py::test_threshold_discount_boundary[50000-
45000]
FAILED tests/test_cart.py::test_threshold_discount_boundary[50001-
45000]
3 failed, 2 passed in 0.02s
```

핵심 @pytest.mark.parametrize는 같은 규칙의 여러 예시를 한 함수로 쓴다. 실행 결과에는 세 개의 테스트로 따로 나타난다.

실습 · 직접 하기 Day 2 · 2교시 | L2

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L2 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_cart.py @src/cart.py
test_threshold_discount_boundary 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ 50,000원 이상이면 90%로, 원 미만은
  정수 나눗셈으로 버리겠다.
  Edit src/cart.py
+ def apply_threshold_discount(amount):
+     if amount >= 50000:
+         return amount * 90 // 100
+     return amount
✦ 5 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
.....  [100%]
5 passed in 0.02s
```

핵심 amount * 90 // 100 — 정수 나눗셈(//)이 '원 미만 버림'(C5). 실험 A처럼 0.9를 곱하면 소수가 생긴다.

지금 할 일 Gemini: L2 프롬프트 ▸ diff 승인 ▸ 터미널: 5 passed

실습 · 직접 하기 Day 2 · 2교시 | L2 · 확인

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

경계 테스트가 일하는 장면 — >= 를 > 로 바꿔 보기

실습: 일부러 코드를 망가뜨려 테스트가 잡는지 확인한다 (확인 후 원래대로 되돌림).

src/cart.py 한 글자 수정

```
-     if amount >= 50000:
+     if amount > 50000:
```

- 49,999와 50,001은 그대로 통과
- 50,000만 실패 — 정확히 바뀐 곳을 가리킨다

터미널 – 실제 출력

```
> pytest -q --tb=short
...F.  [100%]
=== FAILURES ===
___ test_threshold_discount_boundary[50000-45000] ___
tests/test_cart.py:20: in test_threshold_discount_boundary
    assert apply_threshold_discount(amount) == expected
E   assert 50000 == 45000
E    +  where 50000 = apply_threshold_discount(50000)
=== short test summary info ===
FAILED tests/test_cart.py::test_threshold_discount_boundary[50000-45000]
1 failed, 4 passed in 0.02s
```

핵심 실험 A의 A-4에서 아무도 못 본 버그를 테스트 한 줄이 1초 만에 잡았다. 확인했으면 > 를 다시 >= 로 되돌리고 GREEN을 확인.

지금 할 일 VS Code: >= 를 > 로 바꾸기 ▸ 터미널: 50000 케이스만 실패 확인 ▸ 다시 >= 로 되돌리고 5 passed

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

10:30–11:00

GEMINI.md — 이렇게 진행한다 (테스트 수정 막기)

폴더
cart/

방식
강사 시연 → 따라 하기

결과물
cart/GEMINI.md
(docs/예시표.md 는 1교시 결과 사용)

프롬프트
cart/prompts.md · Day 2 · 3교시

#	어디서	할 일	완료 기준	슬라이드
1	강사 시연	나쁜 프롬프트 → AI 가 테스트를 고치는 장면	보기만	130
2	Gemini	같은 나쁜 프롬프트 실행 (각자)	tests/ diff 는 승인 안 함	130
3	VS Code	GEMINI.md 다섯 줄 저장 (예시표.md 는 1교시 결과)	프로젝트 최상단	131
4	Gemini	/memory refresh → /memory show	규칙이 보임	131
5	Gemini	같은 나쁜 프롬프트 다시	AI 가 멈추고 질문	132

TIP 규칙 파일은 '부탁'이지 잠금장치가 아니다. GEMINI.md 가 있어도 diff 를 읽고 승인하는 것은 계속 사람의 몫.

실습 · 직접 하기 Day 2 · 3교시 | AI 규칙 파일

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

10:30–11:00

AI가 테스트를 고쳐서 통과시키는 순간

실습 중 실제로 자주 생긴다. 오늘은 일부러 재현해 본다 — '테스트 수정 금지' 없이, 결과만 요구하면?

▶ 프롬프트 — 나쁜 예

@tests/test_cart.py @src/cart.py 테스트가 다 통과하게 고쳐줘. (← 금지 줄이 없다)

결과 예시 이런 결과가 나올 수 있다

Gemini CLI

```
✦ 50,000원은 할인 대상이 아닌 것으로 보여
  테스트 기대값을 수정했다.
  Edit tests/test_cart.py
-     (50000, 45000),   # 문턱 정확히: 10% 할인
+     (50000, 50000),   # 문턱 정확히: 할인 없음
✦ 5 passed
```

무엇이 문제인가

- GREEN이 됐지만 사양(C3)이 바뀌었다
- AI는 코드와 테스트 중 '고치기 쉬운 쪽'을 고친다
- diff를 읽지 않고 승인하면 모른다
- 예시 >

```
cd C:\DEV\tdd-course\cart
git init
git add .
git commit -m "실습 시작 상태"
git diff -- tests/test_cart.py
```

주의 Edit tests/… 가 보이면 무조건 '아니요'. 그런데 매번 프롬프트에 금지를 쓰는 것은 잊기 쉽다 → 규칙을 파일로 고정.

지금 할 일 Gemini: 나쁜 프롬프트 실행 ▸ tests/ 를 고치는 diff 가 나오면 승인하지 말고 무엇을 바꾸려 했는지 기록

실습 · 직접 하기 Day 2 · 3교시 | GEMINI.md

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

GEMINI.md 다섯 줄 — 매 세션 자동으로 읽히는 규칙

cart/GEMINI.md (프로젝트 루트에 저장)

```
# 이 프로젝트의 규칙
- tests/ 아래 파일은 절대 수정하지 않는다.
- 요청받은 테스트를 통과하는 데 필요한 코드만 쓴다.
- 요구사항은 docs/예시표.md 가 기준이다.
- 변경 후에는 pytest -q --tb=short 결과를 보여준다.
- 테스트가 틀려 보이면 고치지 말고 나에게 질문한다.
```

적용 확인 (세션 안에서)

```
Gemini CLI

> /memory show
✦ 현재 로드된 메모리:
  --- Context from: GEMINI.md ---
  # 이 프로젝트의 규칙
  - tests/ 아래 파일은 절대 수정하지 않는다.
  …
```

만드는 법

- VS Code에서 새 파일 GEMINI.md 혹은 /init → 위 다섯 줄 붙여넣기
- gemini를 다시 시작하거나 /memory refresh

주의

- 규칙 파일은 '지침'이지 잠금장치가 아니다
- diff 승인(사람의 확인)은 계속 필요하다

참고 1교시에 저장한 docs/예시표.md(C1~C7)를 GEMINI.md 가 '기준'으로 가리킨다. AI가 '50,000원은 할인 대상이 아닌 것 같다'고 추측하는 대신 이 파일을 근거로 삼는다.

지금 할 일 VS Code: cart/GEMINI.md 저장 (docs/예시표.md 는 1교시에 저장됨) ▸ Gemini: /memory refresh → /memory show 로 확인

실습 · 직접 하기 Day 2 · 3교시 | GEMINI.md

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

앞에서 사용한 위험한 프롬프트를 다시 입력해 본다 — 이번에는 AI가 테스트를 수정하지 않는다

▶ 프롬프트 — 나쁜 예

@tests/test_cart.py @src/cart.py 테스트가 다 통과하게 고쳐줘.

결과 예시 GEMINI.md 적용 후 결과 예시

Gemini CLI

```
✦ GEMINI.md 규칙에 따라 tests/ 파일은 수정하지 않는다.
  현재 실패: test_threshold_discount_boundary[50000-45000] — 실제 50000, 기대 45000
  docs/예시표.md C3에 따르면 50,000원은 할인 대상(이상)이므로 구현을 수정하겠다.
  Edit src/cart.py
-     if amount > 50000:
+     if amount >= 50000:
✦ 5 passed
```

핵심 규칙 파일 + 예시 표 + 테스트 = AI가 '무엇이 맞는지'를 추측하지 않고 근거로 판단하는 환경. 이것이 하네스의 가장 작은 형태.

원칙 그래도 결과가 다를 수 있다. 끝까지 지켜야 할 사람의 일: Edit tests/... 가 보이면 거절, 테스트를 고칠 일이 생기면 사람이 사양부터 확인.

지금 할 일 Gemini: 같은 나쁜 프롬프트 다시 ▸ AI 가 tests/ 수정을 멈추고 질문하는지 확인

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

11:10–12:00

장바구니 TDD 2 —진행 방식 (L3 · 불변식 · L4)

폴더

`cart/`

방식

개인

결과물

19 passed (골든 마스터 포함)
RED 캡처 2장

프롬프트

cart/prompts.md · Day 2 · 4교시

#	어디서	할 일	완료 기준	슬라이드
1	VS Code → 터미널	L3: VIP 테스트 2개 → pytest	2 failed, 5 passed	134
2	Gemini	L3 프롬프트 → diff 승인	7 passed	134
3	VS Code → 터미널	불변식 테스트 추가 → pytest -q -k between	10 passed (이미 충족)	135–138
4	VS Code → 터미널	L4: 음수 가격 테스트 → pytest	1 failed, 17 passed	139
5	Gemini	L4 프롬프트 → diff 승인	18 passed	140
6	Gemini → 터미널	골든 마스터 프롬프트 → UPDATE_GOLDEN=1 → 골든 읽고 승인	19 passed · 커밋	141–142

TIP 불변식 테스트가 바로 통과하는 것은 정상. 가짜 RED 를 만들지 말고 '이미 충족'으로 기록.

지금 할 일 cart 폴더(앞에서 연 것)에서 ▸ 아래 표를 1단계부터 차례로 · 프롬프트는 폴더의 prompts.md 에서 복사 · 막히면 손을 들어 주세요

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L3 · VIP 추가 할인 — 순서까지 테스트에 적는다

① 테스트 추가 (C4) — tests/test_cart.py

```
def test_vip_gets_extra_5_percent_after_threshold():
    items = [{"price": 12000, "qty": 3},
             {"price": 24000, "qty": 1}]         # 합계 60,000
    # 60,000 → 54,000(문턱 10%) → 51,300(VIP 5%)
    assert final_total(items, is_vip=True) == 51300


def test_non_vip_gets_threshold_discount_only():
    items = [{"price": 25000, "qty": 2}]                # 50,000
    assert final_total(items) == 45000
```

② RED

터미널

```
> pytest -q --tb=short
.....FF  [100%]
=== FAILURES ===
___ test_vip_gets_extra_5_percent_after_threshold ___
… (상세 생략)
=== short test summary info ===
FAILED
tests/test_cart.py::test_vip_gets_extra_5_percent_after_threshold
FAILED tests/test_cart.py::test_non_vip_gets_threshold_discount_only
2 failed, 5 passed in 0.02s
```

▶ 프롬프트 — Gemini CLI

@tests/test_cart.py @src/cart.py @docs/예시표.md

C3와 C4를 기준으로 다음 테스트를 통과하는 최소 구현을 해줘.
- test_vip_gets_extra_5_percent_after_threshold
- test_non_vip_gets_threshold_discount_only

기존 테스트도 계속 통과해야 해.
테스트 파일은 수정하지 마.
문서에 없는 기능은 추가하지 마.
끝나면 pytest -q --tb=short 결과를 보여줘.

결과 예시

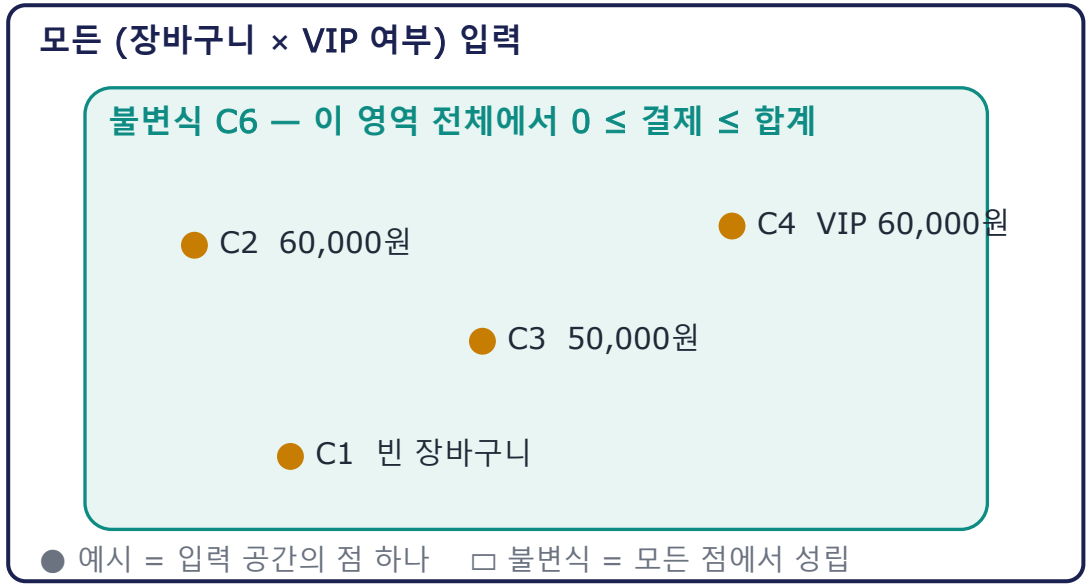
Gemini CLI — ③ GREEN

```
+ def final_total(items, is_vip=False):
+     amount = apply_threshold_discount(subtotal(items))
+     if is_vip:
+         amount = amount * 95 // 100
+     return amount
✦ 7 passed
```

참고 기대값 51,300에 순서가 들어 있다: 60,000 → 54,000(문턱) → 51,300(VIP). 순서를 바꾸면 57,000 → 51,300 … 같아 보이지만 금액에 따라 달라진다(예: 52,000원).

불변식 한 번 더 — 예시는 점 하나, 불변식은 모든 점

1장 1.3 에서 배운 Example 과 Property 를 장바구니로 다시 본다.



Example (예시)

- 입력 하나 → 기대 결과 하나
- 60,000원 · VIP → 51,300원

Property · Invariant (성질 · 불변식)

- 어떤 입력에서도 '항상' 참인 약속
- 0 ≤ 결제 금액 ≤ 합계 (C6)

비유 예시는 '시험 문제 한 문제', 불변식은 '시험 규칙'(점수는 0 미만도, 만점 초과도 없다). 문제를 바꿔도 규칙은 안 바뀐다.

핵심 불변식 테스트는 처음부터 GREEN 이어도 정상(이미 충족) — 나중에 누군가 깨뜨릴 때 빨간불을 켜는 감시 장치다.

불변식을 찾는 세 가지 질문 — 장바구니의 불변식 후보

Q1 항상 참인 것은?
- 입력이 무엇이든 성립하는 범위 · 부호
- 결제 ≥ 0, 합계 ≥ 0

Q2 바뀌면 안 되는 것은?
- 할인이 합계를 넘어설 수 없다
- VIP 가 일반보다 더 내지 않는다

Q3 같은 답이어야 하는 것은?
- 같은 입력 → 항상 같은 결과
- 항목 순서를 바꿔도 같은 합계

ID	불변식 (항상 지킬 약속)	검증 방법	상태
INV-1	0 ≤ 결제 금액 ≤ 합계 (C6)	SAMPLES × VIP 10조합 (오늘)	오늘 테스트로 추가
INV-2	VIP 결제 금액 ≤ 일반 결제 금액	같은 장바구니를 두 번 계산해 비교	부록 C 속성 테스트
INV-3	같은 입력 → 같은 결과 (결정성)	골든 마스터 (이어지는 슬라이드)	골든 마스터
함정	"금액이 늘면 결제 금액도 는다"	49,999 → 49,999 인데 50,000 → 45,000	불변식이 아니다!

참고 1장 1.3 '불변식 분석'(슬라이드 31)과 부록 C 속성 테스트(슬라이드 284~)에서 같은 개념을 다시 만난다.

원칙 불변식은 사양에서 찾는다 — 코드를 읽고 '지금 이렇다'를 적으면 그것은 불변식이 아니라 골든 마스터다.

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

불변식 — '항상' 지켜야 하는 성질

장바구니 예제로 보는 TDD 전 검토 사항

검토 항목	핵심 질문	장바구니 예시	Test로 구체화
요구사항 (Requirement)	사용자가 원하는 것은 무엇인가?	장바구니 상품의 총액을 계산한다	가격 × 수량의 합계가 정확한가
기능 (Feature)	시스템이 제공해야 할 기능은 무엇인가?	합계 계산, 금액 할인, VIP 할인	Feature별 대표 Test
행동/사양 (Behavior / Specification)	주어진 입력에 시스템은 정확히 어떻게 동작해야 하는가?	합계가 50,000원 이상이면 10% 할인	50,000 → 45,000
업무 규칙 (Business Rule)	계산·판단에 적용되는 업무 규칙은 무엇인가?	금액 할인 후 VIP 5% 추가 할인	60,000 → 54,000 → 51,300
적용 조건 (Condition)	어떤 조건에서 해당 규칙이 적용되는가?	amount >= 50000, is_vip=True	조건 만족 / 불만족 Case
경계값 (Boundary Value)	결과가 달라지는 경계는 어디인가?	50,000원이 할인 적용 경계	49,999 / 50,000 / 50,001
불변식 (Invariant)	어떤 변경 후에도 반드시 지켜져야 하는 것은 무엇인가?	최종 결제금액은 음수가 될 수 없다	final_amount >= 0
예외/오류 (Exception / Error Case)	비정상 입력은 어떻게 처리해야 하는가?	음수 가격, 음수 수량 등	Exception 발생 여부 확인
데이터 형식 (Data Type / Format)	값의 타입·단위·정밀도는 어떻게 할 것인가?	금액은 정수로 반환	54000.0이 아닌 54000
규칙 조합 (Rule Interaction)	여러 규칙이 동시에 적용되면 어떻게 되는가?	금액 할인 + VIP 할인	복수 할인 적용 순서와 결과 검증
회귀 조건 (Regression)	변경 후에도 기존의 어떤 동작이 유지되어야 하는가?	VIP 추가 후 일반 고객 계산 결과 유지	기존 Test Suite 전체 실행
완료 기준 (Acceptance Criteria)	어떤 상태가 되면 요구사항이 충족됐다고 판단할 것인가?	신규 기능과 기존 기능 모두 정상	신규 + 기존 Test 모두 GREEN

지금 할 일 VS Code: 불변식 테스트 추가 ▸ 터미널: pytest -q -k between → 10 passed ('이미 충족'으로 기록)

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

불변식 — '항상' 지켜야 하는 성질을 테스트로

C6: 결제 금액은 0 이상, 합계 이하. 특정 예가 아니라 모든 장바구니에서 성립해야 한다.

tests/test_cart.py — 여러 장바구니 × VIP 여부 = 10가지 조합

```
SAMPLES = [
    [],
    [{"price": 1000, "qty": 1}],
    [{"price": 49999, "qty": 1}],
    [{"price": 25000, "qty": 2}],
    [{"price": 99000, "qty": 3}],
]


@pytest.mark.parametrize("items", SAMPLES)
@pytest.mark.parametrize("is_vip", [False, True])
def test_final_total_is_between_zero_and_subtotal(items, is_vip):
    total = final_total(items, is_vip)
    assert 0 <= total <= subtotal(items)
```

어떤 입력에서도 반드시 지켜져야 하는 성질(Property), 즉 불변식(Invariant)

assert 0 <= total <= subtotal(items)

할인 정책이나 VIP 여부가 어떻게 변경되더라도 최종 결제금액(final total)은 0 이상이고, 할인 전 상품 총액(subtotal)을 초과해서는 안 된다.

불변식 (Invariant)	의미	검증 조건
Non-negative Total	최종 결제금액은 음수가 될 수 없다	total >= 0
Upper Bound	할인 후 결제금액은 할인 전 금액보다 커질 수 없다	total <= subtotal(items)
Valid Range	최종 결제금액은 0원과 원래 상품금액 사이에 있어야 한다	0 <= total <= subtotal(items)

지금 할 일 VS Code: 불변식 테스트 추가 ▸ 터미널: pytest -q -k between → 10 passed ('이미 충족'으로 기록)

실습 · 직접 하기 Day 2 · 4교시 | L4 · C7

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L4 · 음수 가격은 오류 — 이번에는 진짜 RED

① 직접 작성— tests/test_cart.py

```
def test_negative_price_is_rejected():
    with pytest.raises(ValueError):
        subtotal([{"price": -1000, "qty": 1}])
```

② 실행 → RED 확인

터미널

```
> pytest -q --tb=short
.................F  [100%]
=== FAILURES ===
___ test_negative_price_is_rejected ___
tests/test_cart.py:52: in test_negative_price_is_rejected
    with pytest.raises(ValueError):
E   Failed: DID NOT RAISE ValueError
=== short test summary info ===
FAILED tests/test_cart.py::test_negative_price_is_rejected - Fa...
1 failed, 17 passed in 0.02s
```

핵심 17개 통과, 1개 실패 — 불변식 10개는 통과(이미 충족), 음수 검사만 실패. 'DID NOT RAISE'는 어제 R6과 같은 모양.

지금 할 일 VS Code: 음수 가격 테스트 추가 ▸ 터미널: 1 failed, 17 passed 확인

실습 · 직접 하기 Day 2 · 4교시 | L4

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

L4 · 최소 구현

▶ 프롬프트 — Gemini CLI

@tests/test_cart.py @src/cart.py
test_negative_price_is_rejected 만 통과하는 최소 구현을 해줘.
테스트 파일은 수정하지 마. 다른 기능은 추가하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI

```
✦ subtotal에서 음수 가격·수량을 검사하겠다.
  Edit src/cart.py
+     for item in items:
+         if item["price"] < 0 or item["qty"] < 0:
+             raise ValueError("가격과 수량은 0 이상이어야 합니다")
✦ 18 passed
```

③ 다시 실행 → GREEN

터미널

```
> pytest -q --tb=short
..................  [100%]
18 passed in 0.02s
```

지금 할 일 Gemini: L4 프롬프트 ▸ diff 승인 ▸ 터미널: 18 passed

리팩터링 전에 골든 마스터를 한 번 더 — 이번에는 장바구니

어제 calc 에서 한 것을 cart/ 에서 다시: 18개 테스트가 모르는 입력까지 통째로 고정한다.

1 조합을 정한다
장바구니 9종 × 일반/VIP = 18

2 저장한다
결과를 cart.golden.json 에

3 읽고 승인한다
예시 표(C1~C7)와 맞는지 눈으로

4 리팩터링 후 비교
다르면 즉시 FAIL

장바구니	내용	일반	VIP
empty	[]	0	0
just_below	49,999 × 1	49,999	47,499
exactly_50000	25,000 × 2	45,000	42,750
sample_60000	12,000×3 + 24,000×1	54,000	51,300
zero_qty	10,000 × 0	0	0
negative_price	-1,000 × 1	ValueError	ValueError

왜 18개 테스트로 부족한가

- 테스트는 '우리가 떠올린' 입력만 본다
- 리팩터링은 떠올리지 못한 입력의 결과도 바꿀 수 있다
- 골든은 조합 전체를 한 번에 지킨다
- 변경(>= → >)이 생기면 3곳이 동시에 빨개진다

핵심 순서: 단위 테스트(사양) GREEN → 골든 마스터 생성 · 승인 → 리팩터링 → 둘 다 GREEN. 골든은 '현재 동작'이므로 사양 테스트가 먼저다.

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

골든 마스터 작성 프롬프트 — cart 의 final_total 고정

▶ 프롬프트 — Gemini CLI

@src/cart.py @tests/test_cart.py @docs/예시표.md
final_total() 의 '현재 동작'을 골든 마스터로 고정하는 테스트를 tests/test_golden_master.py 에 만들어줘.
- 장바구니 9종(빈 장바구니 · 1,000원 · 49,999 / 50,000 / 50,001원 · 합계 60,000원 예제 · 99,000×3 · 수량 0 · 음수 가격) × 일반/VIP = 18개 조합
- 결과(정수) 또는 오류 종류(ValueError)를 tests/golden/cart.golden.json 에 저장, UPDATE_GOLDEN=1 일 때만 갱신, 골든이 없으면 FAIL
- src/ 와 기존 테스트는 수정하지 마. 끝나면 !pytest -q 결과를 보여줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

터미널

```
> pytest -q
FAILED tests/test_golden_master.py::test_golden_master
1 failed, 18 passed in 0.11s      # 골든 파일이 아직 없다

> UPDATE_GOLDEN=1 pytest -q tests/test_golden_master.py
1 passed in 0.01s

> pytest -q
19 passed in 0.02s

> git add tests/golden && git commit -m "test: cart 골든 마스터"
```

읽을 것

```
// tests/golden/cart.golden.json (일부)
"just_below/normal":   {"total": 49999},
"exactly_50000/vip":   {"total": 42750},
"zero_qty/normal":     {"total": 0},
"negative_price/vip":  {"error": "ValueError"},
...
```

✦ 읽을 것
zero_qty → 0 : 120 슬라이드에서 '수량 0 허용'으로 결정
negative_price → ValueError : 예시 표 C7 과 일치

03

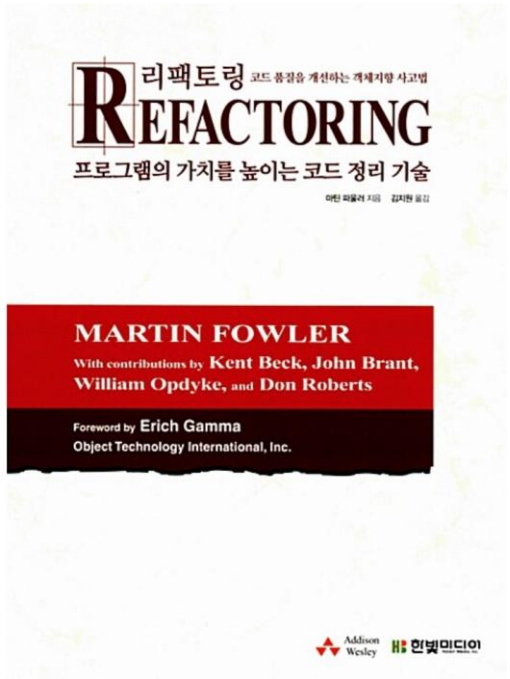
CHAPTER 3 · 이론

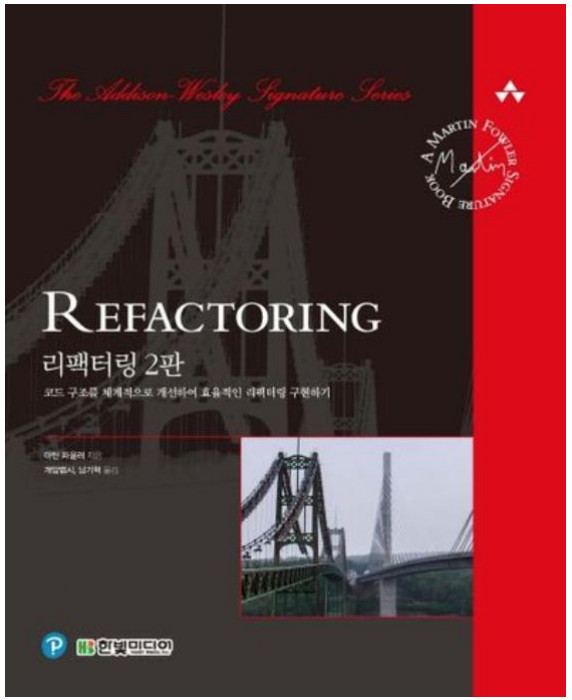
클린 코드와 리팩토링

24가지 코드 냄새 · 리팩토링 기법 · SOLID · 네이밍 · 함수 설계 · 오류 처리 · 정적 분석 · 디자인 패턴 (Python).
외부 동작을 바꾸지 않으면서 내부 구조를 개선한다.

⏱ 13:00–13:50 · 3장 이론

Code Smell & Refactoring — 무엇을, 왜 고치는가





Martin Fowler,『리팩토링』1판·2판



리팩토링이란

외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 것. 테스트가 GREEN인 상태에서만 수행한다.

코드 냄새(Code Smell)란

당장 버그는 아니지만 더 깊은 설계 문제를 암시하는 징후. 냄새를 찾아 이름 붙이면 처방이 따라온다.

이 장의 과정

냄새 24가지 → 3의 법칙 → SOLID → 네이밍·함수·오류 처리 → 정적 분석 → 디자인 패턴 → QA 프롬프트

리팩토링(Refactoring)과 리스트럭처링(Restructuring)

둘 다 '동작은 그대로, 구조만 개선' — 다루는 범위가 다르다.

Refactoring (리팩토링)

- 코드의 내부 구조를 개선하되 외부 동작(기능)은 그대로 유지
- 범위: 한 함수 · 한 클래스 안
- 예) 중복 제거, 이름 바꾸기, 함수 추출

Restructuring (리스트럭처링)

- 시스템 · 모듈의 전체 구조 · 설계를 재편하는 더 큰 범위의 변경
- 범위: 여러 클래스 · 모듈 · 아키텍처
- 예) 책임 분리, 클래스 이동, 패키지 구조 변경

핵심 두 경우 모두 '테스트가 GREEN 인 상태에서, 동작을 바꾸지 않고' 한다. 안전망(테스트 · 골든 마스터)이 없으면 구분 이전에 시작하지 않는다.

리팩토링 예제 — 작은 수준의 개선 (중복 제거)

전: 같은 계산이 두 갈래에 중복

```
def calculate_discount(price, is_vip):
    if is_vip:
        return price - (price * 10 // 100)
    else:
        return price - (price * 5 // 100)
```

후: 중복 제거 — 비율만 고른다

```
def calculate_discount(price, is_vip):
    rate = 10 if is_vip else 5
    return price - (price * rate // 100)
```

핵심 기능은 동일하지만 코드가 간결하고 유지보수가 쉽다 — 입력 → 결과가 같다는 것은 테스트로 확인한다.

TIP dp_refactor_vs_restructure/ 의 test_same.py 가 전 · 후의 결과가 같은지(10개 조합) 확인한다.

리스트럭처링 예제 — 구조적 재조정 (책임 이동)

전: 할인 로직이 Order 에 섞여 있다

```
class Order:
    def __init__(self, price, is_vip):
        self.price = price
        self.is_vip = is_vip
    def discounted_price(self):
        if self.is_vip:
            return self.price - (self.price * 10 // 100)
        else:
            return self.price - (self.price * 5 // 100)
```

후: 할인 책임을 Customer 에게 위임

```
class Customer:
    def __init__(self, is_vip):
        self.is_vip = is_vip
    def get_discount(self, price):
        rate = 10 if self.is_vip else 5
        return price - (price * rate // 100)

class Order:
    def __init__(self, price, customer):
        self.price = price
        self.customer = customer
    def discounted_price(self):
        return self.customer.get_discount(self.price)
```

핵심 책임 분리 — Customer 가 할인을 알고, Order 는 가격만 안다. 구조적 분리로 유연성과 재사용성이 높아진다.

참고 고객 유형이 늘어도 Customer 쪽만 고치면 된다 — 이 방향의 끝이 3.9 의 전략(Strategy) 패턴이다.

리팩토링 vs 리스트럭처링 — 한눈에 비교

구분	리팩토링	리스트럭처링
범위	코드 내부 한 함수 · 클래스	시스템 또는 아키텍처 수준
목적	코드 품질 · 가독성 · 유지보수성 향상	구조 재편성 · 책임 분리 · 모듈화
예시	중복 제거, 변수 이름 변경	클래스 분리, 패키지 구조 이동

참고 이 과정에서는 '리팩토링'이라고 부를 때 리스트럭처링(클래스 분리 · 책임 이동)까지 포함해서 다룬다 — 부록 B 의 클래스 분리가 그 예.

리팩토링 vs 리스트럭처링 vs 리엔지니어링

구분	리팩토링	리스트럭처링	리엔지니어링
범위	코드 내부	모듈 / 시스템 구조	전체 시스템 (코드 · 문서 · 설계)
목적	가독성 향상 · 중복 제거	책임 분리 · 설계 개선	노후 시스템 재생 · 유지보수 가능하게
기능 변화	없음	거의 없음	있을 수도 있음
대표 작업	코드 정리	클래스 분리 / 합치기	레거시 재설계 · 문서화 · 테스트 추가
예시	중복 제거	모듈 구조 개선	COBOL 시스템을 Python 서비스로 이관

참고 리엔지니어링: 기존 시스템을 분석 · 이해한 뒤, 기능을 바꾸지 않거나 최소화하며 코드 · 구조 · 문서를 다시 만들어 유지보수성과 확장성을 높이는 작업. 시작은 늘 테스트(골든 마스터) 추가.

24가지 코드 냄새 (Fowler)

1. 이해하기 힘든 이름
2. 중복 코드
3. 긴 함수
4. 긴 매개변수 목록
5. 전역 데이터
6. 가변 데이터
7. 뒤엉킨 변경
8. 산탄총 수술
9. 기능 편애
10. 데이터 뭉치
11. 기본형 집착
12. 반복되는 switch
13. 반복문
14. 성의 없는 요소
15. 추측성 일반화
16. 임시 필드
17. 메시지 체인
18. 중재자
19. 내부자 거래
20. 거대한 클래스
21. 서로 다른 인터페이스의 대안 클래스들
22. 데이터 클래스
23. 상속 포기
24. 주석

① 중복 코드 — 같은 규칙이 두 곳 이상에 있다

증상 같은 코드 두 곳 이상 **처방** 함수 추출 · 공통 함수로 모은다

✗ Before

```
def vip_price(price):
    if price >= 50000:
        return int(price * 0.9 * 0.95)
    return int(price * 0.95)


def normal_price(price):
    if price >= 50000:
        return int(price * 0.9)
    return int(price)
```

✓ After

```
def threshold(price):
    return price * 0.9 if price >= 50000 else price


def vip_price(price):
    return int(threshold(price) * 0.95)


def normal_price(price):
    return int(threshold(price))
```

동작이 같은지 확인

```
> pytest -q  # Before == After
........
[100%]
8 passed in 0.01s
```

핵심 할인 규칙(50,000원 · 10%)이 두 함수에 복사되어 있다 — 한 곳만 고치면 두 함수의 결과가 달라진다.

② 긴 함수 — 한 함수가 여러 일을 한다

증상 20줄 초과 · 여러 단계가 섞임 **처방** 단계별 함수로 분리 (Extract Function)

✗ Before

```
def checkout(items, is_vip):
    total = 0
    for it in items:
        if it["qty"] < 1:
            raise ValueError("qty")
        total += it["price"] * it["qty"]
    if total >= 50000:
        total = total * 0.9
    if is_vip:
        total = total * 0.95
    receipt = f"합계 {int(total):,}원"
    return int(total), receipt
```

✓ After

```
def subtotal(items):
    if any(it["qty"] < 1 for it in items):
        raise ValueError("qty")
    return sum(it["price"] * it["qty"] for it in items)


def apply_discounts(amount, is_vip):
    if amount >= 50000:
        amount *= 0.9
    return amount * 0.95 if is_vip else amount


def checkout(items, is_vip):
    total = int(apply_discounts(subtotal(items), is_vip))
    return total, f"합계 {total:,}원"
```

동작이 같은지 확인

```
> pytest -q  # Before == After
...
[100%]
3 passed in 0.01s
```

핵심 검증 · 합계 · 할인 · 영수증 문자열이 한 함수에 — 나누면 각 단계를 따로 테스트할 수 있다.

③ 거대한 클래스 — 한 클래스가 너무 많은 책임을 진다

증상 계산 · 내보내기 · 알림이 한 클래스에 **처방** SRP 적용 — 책임별 클래스로 분리

✗ Before

```
class Order:
    def __init__(self, items):
        self.items = items

    def total(self):
        return sum(p * q for p, q in self.items)

    def to_csv(self):
        return "\n".join(f"{p},{q}" for p, q in self.items)

    def email_body(self):
        return f"주문 금액: {self.total()}원"
```

✓ After

```
class Order:                          # 계산만 책임진다
    def __init__(self, items):
        self.items = items

    def total(self):
        return sum(p * q for p, q in self.items)


class OrderCsv:                       # 내보내기 책임
    @staticmethod
    def render(order):
        return "\n".join(f"{p},{q}" for p, q in order.items)


class OrderMail:                      # 알림 책임
    @staticmethod
    def body(order):
        return f"주문 금액: {order.total()}원"
```

동작이 같은지 확인

```
> pytest -q  # Before == After
.
[100%]
1 passed in 0.01s
```

핵심 CSV 형식이 바뀌어도, 메일 문구가 바뀌어도 Order 를 고쳐야 했다 — 변경 이유가 하나가 되도록 나눈다.

④ 긴 매개변수 목록 — 인자가 너무 많다

증상 매개변수 3~4개 초과 **처방** 매개변수 객체 도입 (dataclass)

✗ Before

```
def create_user(name, email, phone, street, city, zip_code):
    return f"{name} <{email}> {phone} / {street}, {city} {zip_code}"
```

✓ After

```
from dataclasses import dataclass


@dataclass
class Address:
    street: str
    city: str
    zip_code: str


def create_user(name, email, phone, address: Address):
    a = address
    return f"{name} <{email}> {phone} / {a.street}, {a.city} {a.zip_code}"
```

동작이 같은지 확인

```
> pytest -q  # Before == After
.
[100%]
1 passed in 0.01s
```

핵심 주소 세 값은 늘 함께 다닌다 — Address 하나로 묶으면 순서를 헷갈려 street 와 city 를 바꿔 넣는 실수가 사라진다.

⑤ 죽은 코드 — 아무도 쓰지 않는 코드

증상 미사용 변수 · 함수 · 닿지 않는 코드 **처방** 즉시 삭제 (기록은 Git 이 기억한다)

✗ Before

```
def shipping_fee(amount):
    old_rate = 0.05                 # 쓰이지 않는 변수
    if amount >= 30000:
        return 0
    return 3000
    print("never")                  # 닿지 않는 코드


def legacy_fee(amount):             # 아무도 부르지 않는 함수
    return amount * 0.05
```

✓ After

```
def shipping_fee(amount):
    if amount >= 30000:
        return 0
    return 3000
```

동작이 같은지 확인

```
> pytest -q  # Before == After
...
[100%]
3 passed in 0.01s
```

핵심 ruff check 가 미사용 변수(F841)를 알려 준다. '나중에 쓸지도'는 Git 히스토리로 충분하다.

⑥ 매직 넘버 — 의미를 알 수 없는 숫자

증상 50000 · 90 · 2 같은 맨 숫자 **처방** 이름 있는 상수로 추출

✗ Before

```
def final_price(price, grade):
    if price >= 50000:
        price = price * 90 // 100
    if grade == 2:
        price = price * 95 // 100
    return price
```

✓ After

```
DISCOUNT_THRESHOLD = 50_000
THRESHOLD_PERCENT = 90
VIP_PERCENT = 95
VIP = 2


def final_price(price, grade):
    if price >= DISCOUNT_THRESHOLD:
        price = price * THRESHOLD_PERCENT // 100
    if grade == VIP:
        price = price * VIP_PERCENT // 100
    return price
```

동작이 같은지 확인

```
> pytest -q  # Before == After
....
[100%]
4 passed in 0.01s
```

핵심 grade == 2 가 VIP 라는 것은 코드를 쓴 사람만 안다 — 상수 이름이 곧 문서가 된다.

⑦ 깊은 중첩 — if 가 계단처럼 깊다

증상 중첩 3단계 이상 **처방** Early Return (보호절) · 함수 추출

✗ Before

```
def can_ship(order):
    if order is not None:
        if order["paid"]:
            if order["items"]:
                if order["address"]:
                    return True
    return False
```

✓ After

```
def can_ship(order):
    if order is None:
        return False
    if not order["paid"]:
        return False
    if not order["items"]:
        return False
    return bool(order["address"])
```

동작이 같은지 확인

```
> pytest -q  # Before == After
.....
[100%]
5 passed in 0.01s
```

핵심 실패 조건을 먼저 돌려보내면 '정상 경로'가 왼쪽 끝 한 줄로 보인다.

⑧ 부적절한 이름 — 이름이 의도를 숨긴다

증상 d · t · tmp · x · calc **처방** 의도를 드러내는 이름으로 (Rename Symbol F2)

✗ Before

```
def calc(d, t):
    tmp = 0
    for x in d:
        tmp += x[0] * x[1]
    return tmp * (1 - t)
```

✓ After

```
def total_after_tax(order_lines, tax_rate):
    subtotal = sum(price * qty for price, qty in order_lines)
    return subtotal * (1 - tax_rate)
```

동작이 같은지 확인

```
> pytest -q  # Before == After
.
[100%]
1 passed in 0.01s
```

핵심 d 와 t 가 무엇인지 알려면 함수를 끝까지 읽어야 했다 — 이름만 보고 무엇을 하는지 알 수 있게 바꾼다.

3의 법칙 (Rule of Three) — Python 예제

돈 로버츠(Don Roberts): ① 처음엔 그냥 한다 ② 두 번째 중복은 일단 진행한다 ③ 세 번째 반복되면 리팩토링한다.
참고 : https://eoinnoble.com/posts/origins-of-the-rule-of-three/



✗ 세 번째 중복 (냄새)

```
class UserService:
    def get_user_full_name(self, first, last):
        return f'{first} {last}'

    def get_admin_full_name(self, first, last):
        return f'{first} {last}'   # 중복

    def get_guest_full_name(self, first, last):
        return f'{first} {last}'   # 세 번째!
```

✓ 공통 로직 추출

```
def format_full_name(first, last):
    return f'{first} {last}'

class UserService:
    def get_user_full_name(self, first, last):
        return format_full_name(first, last)

    def get_admin_full_name(self, first, last):
        return format_full_name(first, last)

    def get_guest_full_name(self, first, last):
        return format_full_name(first, last)
    # 이름 규칙이 바뀌면 format_full_name 한 곳만 고친다
```

디미터 법칙 — 최소 지식 원칙 (Python)

Law of Demeter :

- 낯선 이에게 말 걸지 않는다 (Don't talk to strangers)-1987년 말 미국 노스이스턴 대학교(Northeastern University)의 이안 홀랜드(Ian Holland)와 동료들이 진행하던 '디미터 프로젝트(Demeter Project) ' 에서 사용한 공식 모토(Motto)
- 연쇄 호출은 기차(Train wreck)이자 결합도의 냄새(Smell)다 - 마틴 파울러(Martin Fowler)의 저서 《리팩터링(Refactoring)》과 로버트 C. 마틴(Uncle Bob)의 저서 《클린 코드(Clean Code)》를 통해 소개

✗ 나쁜 예 (위반)

```
city = order.customer \
            .address \
            .city
```

- order→customer→address→city 내부 구조를 다 안다
- 구조가 바뀌면 호출부가 전부 깨진다

✓ 좋은 예 (준수)

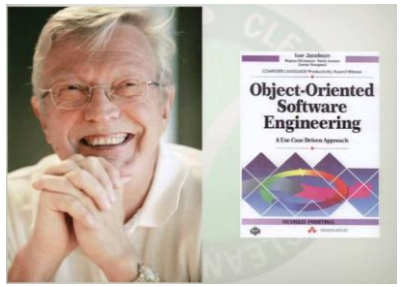
```
city = order.customer_city()
# Order 내부에서 위임 처리:
#   return self._customer.address.city
```

- 외부는 Order 하나만 알면 된다
- 내부 구조 변경이 호출부에 새어 나가지 않는다

원칙 위임 메서드로 감싸 내부 구조를 숨긴다 — 객체는 자신의 직접 이웃하고만 대화한다.

객체 지향 설계 5원칙 — SOLID

로버트 마틴(Robert C. Martin, 별칭 Uncle Bob) 2002년 저서《Agile Software Development, Principles, Patterns, and Practices》에서 정립. 2004년경 마이클 페더스(Michael Feathers)가 SOLID라는 두문자어(Acronym)로 명명.



S

단일 책임 (SRP)

하나의 클래스는 하나의 책임만 가진다

2002년 저서《Agile Software Development, Principles, Patterns, and Practices》에서 정립

O

개방-폐쇄 (OCP)

확장에는 열려 있고 변경에는 닫혀 있다

1988년 바트랑 마이어(Bertrand Meyer)가 처음 제안한 개념으로, 이후 로버트 마틴이 객체 지향 관점으로 재정립

L

리스코프 치환 (LSP)

파생 클래스는 기반 클래스를 대체할 수 있어야 한다

1987년 바바라 리스코프(Barbara Liskov)가 발표한 강연/논문 내용을 1996년 로버트 마틴이 원칙으로 정리

I

인터페이스 분리 (ISP)

쓰지 않는 메서드에 의존하지 않는다

1996년 로버트 마틴이 제록스(Xerox) 컨설팅 당시 체계화한 원칙

D

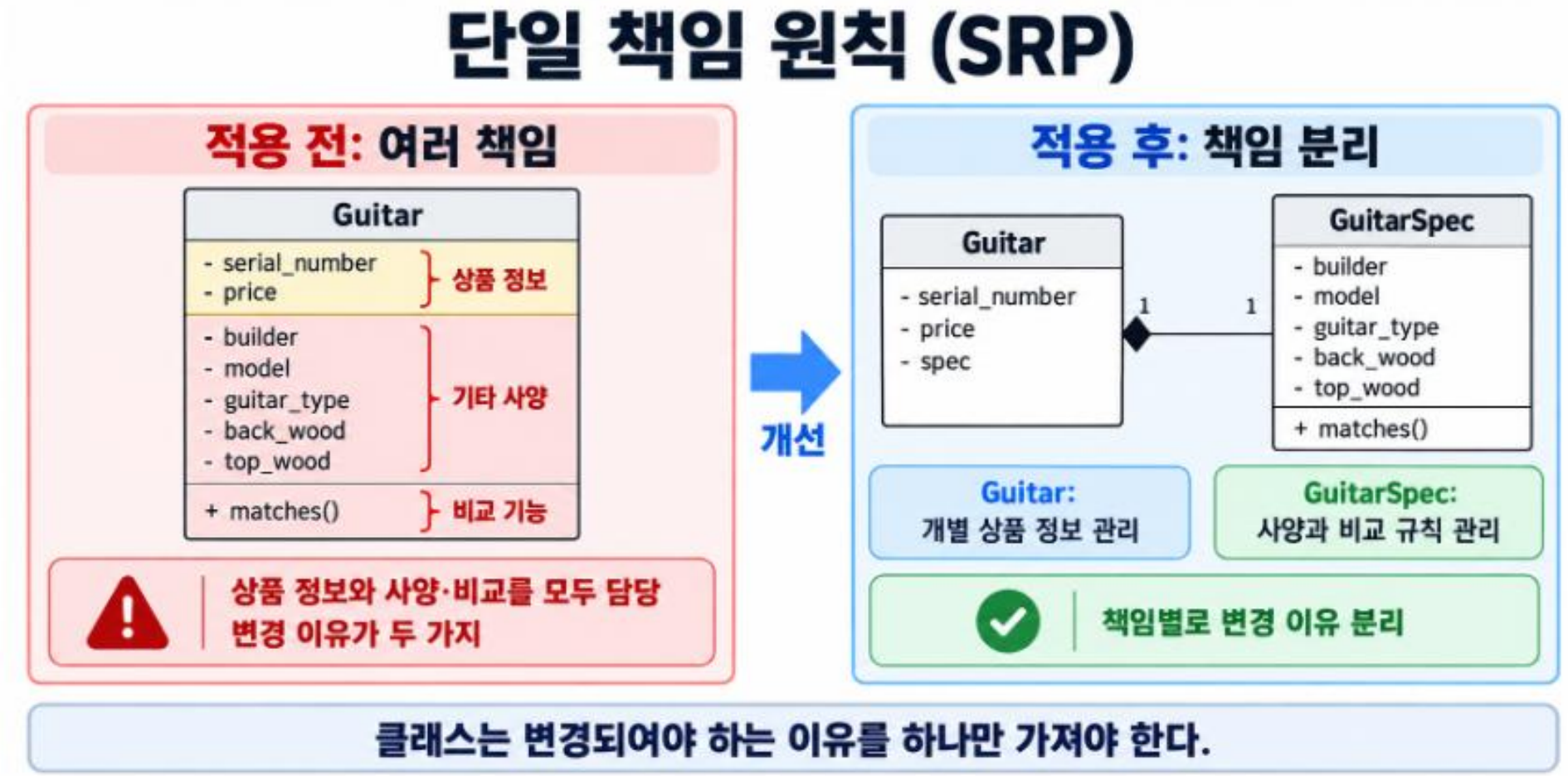
의존 역전 (DIP)

추상화에 의존하고 구체적인 것에 의존하지 않는다

1996년 로버트 마틴이 발표한 논문《The Dependency Inversion Principle》에서 처음 발표

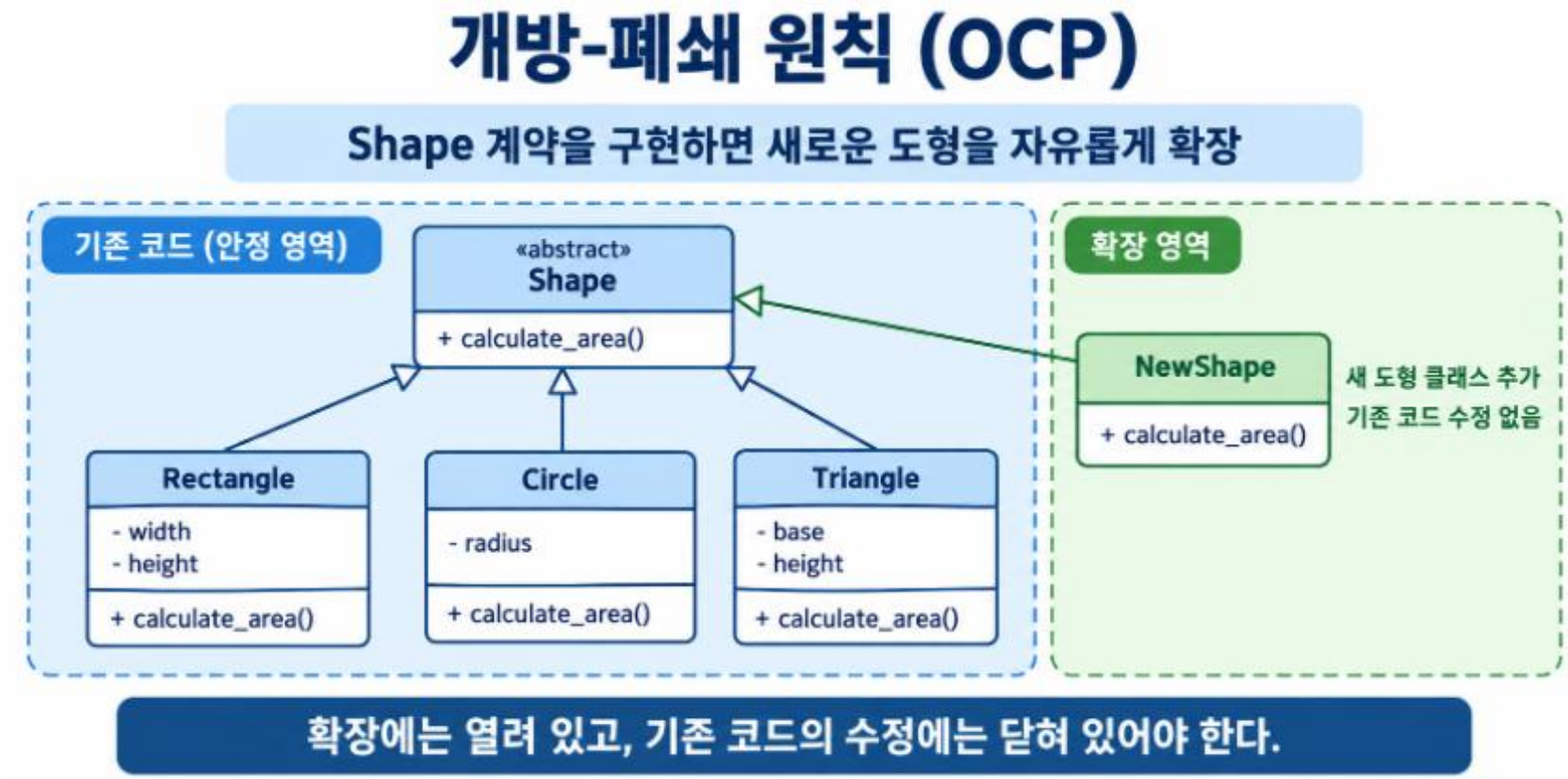
SRP 예시 — 책임을 분리한다 (Python)

클래스는 단 하나의 책임만 가진다 — 변경의 이유가 하나여야 한다. 직렬화·표현 책임은 별도로 분리한다.



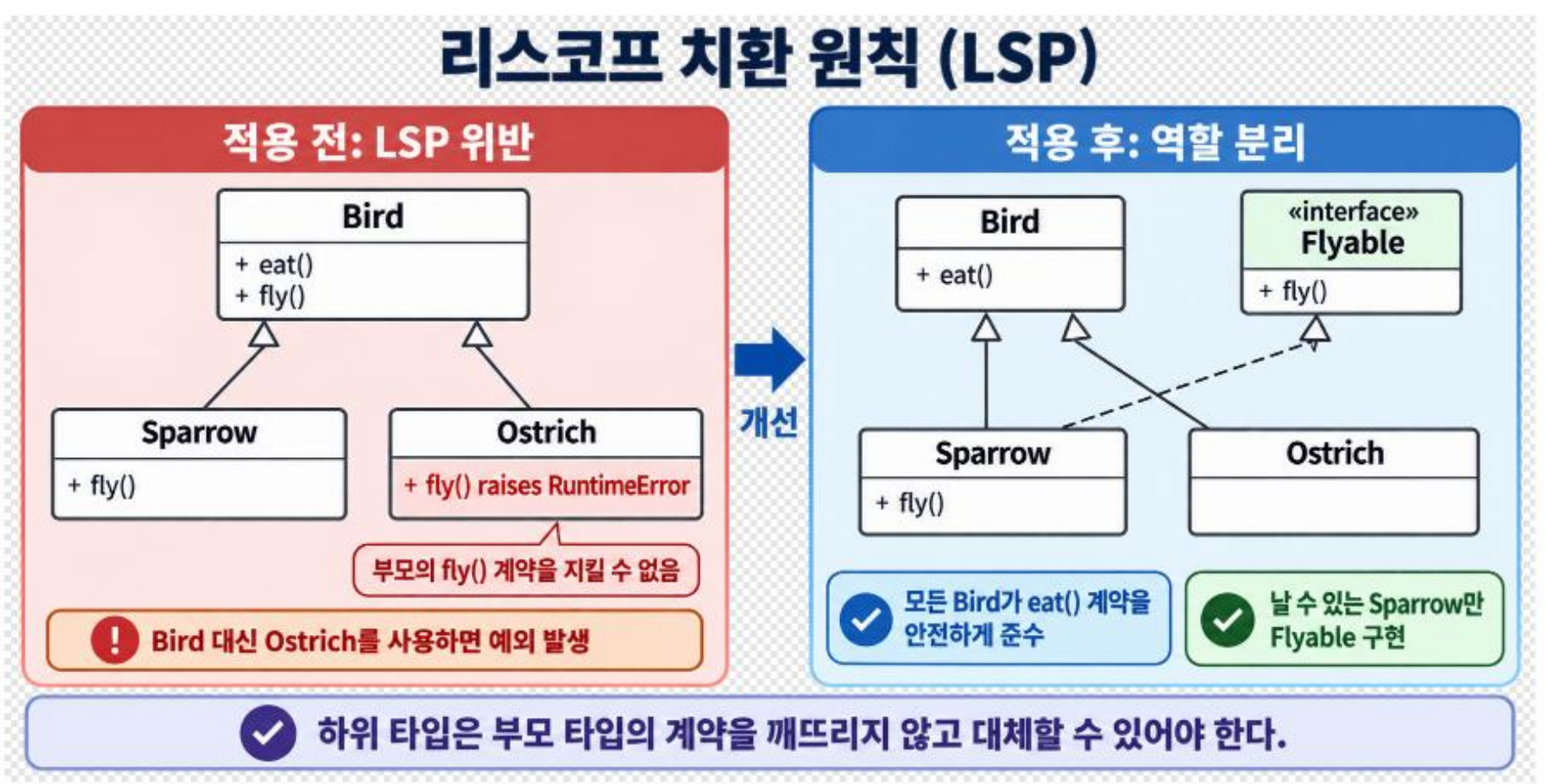
OCP · LSP · ISP · DIP — Python 적용 요약

OCP	새 유형마다 if/elif 추가	추상 기반 클래스(ABC:Abstract Base Class)·Protocol + 딕셔너리 디스패치



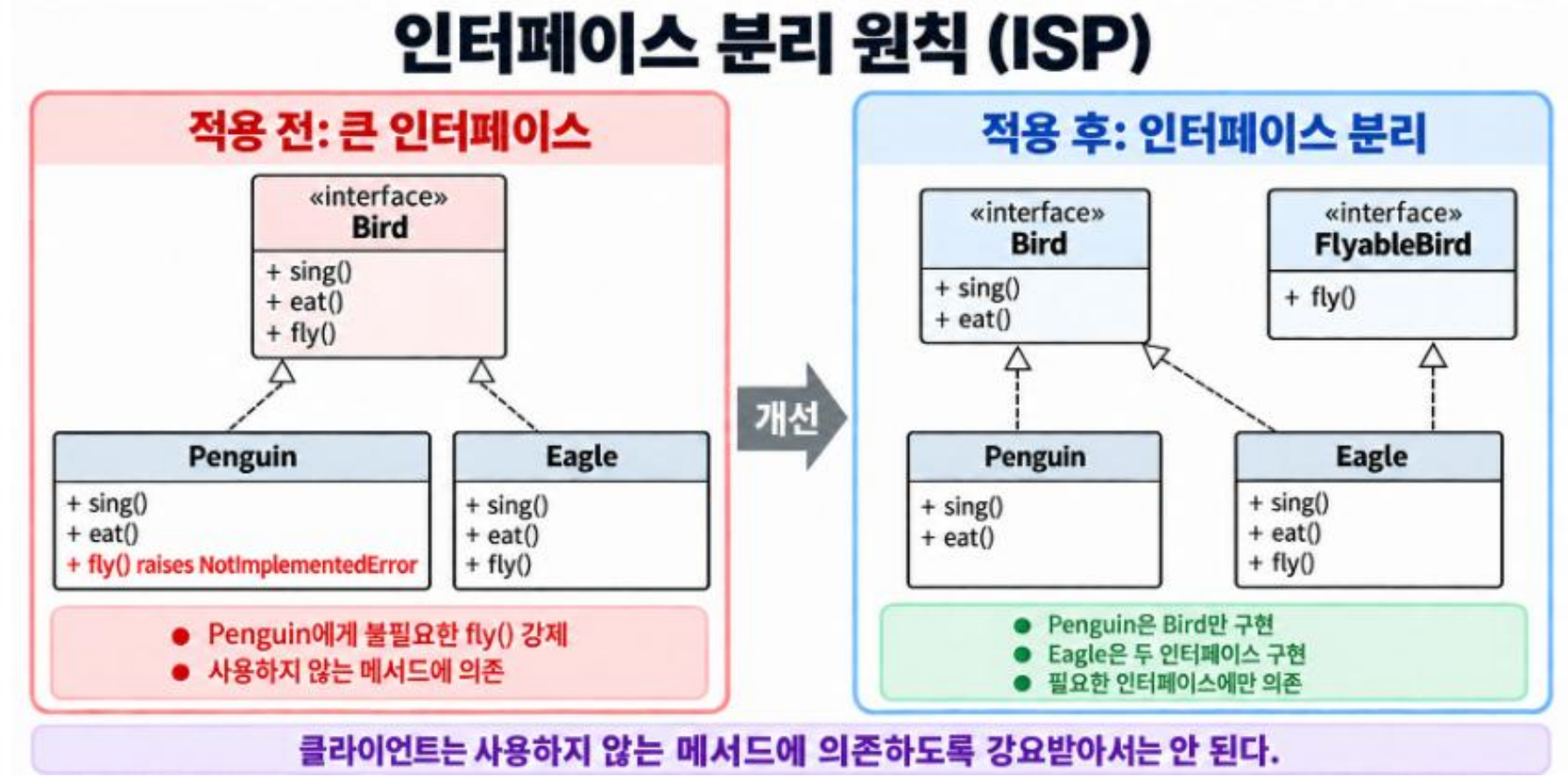
OCP · LSP · ISP · DIP — Python 적용 요약

LSP	자식이 부모 계약을 깸 (fly 못하는 Ostrich)	공통은 Bird, 능력은 Flyable 인터페이스로 분리



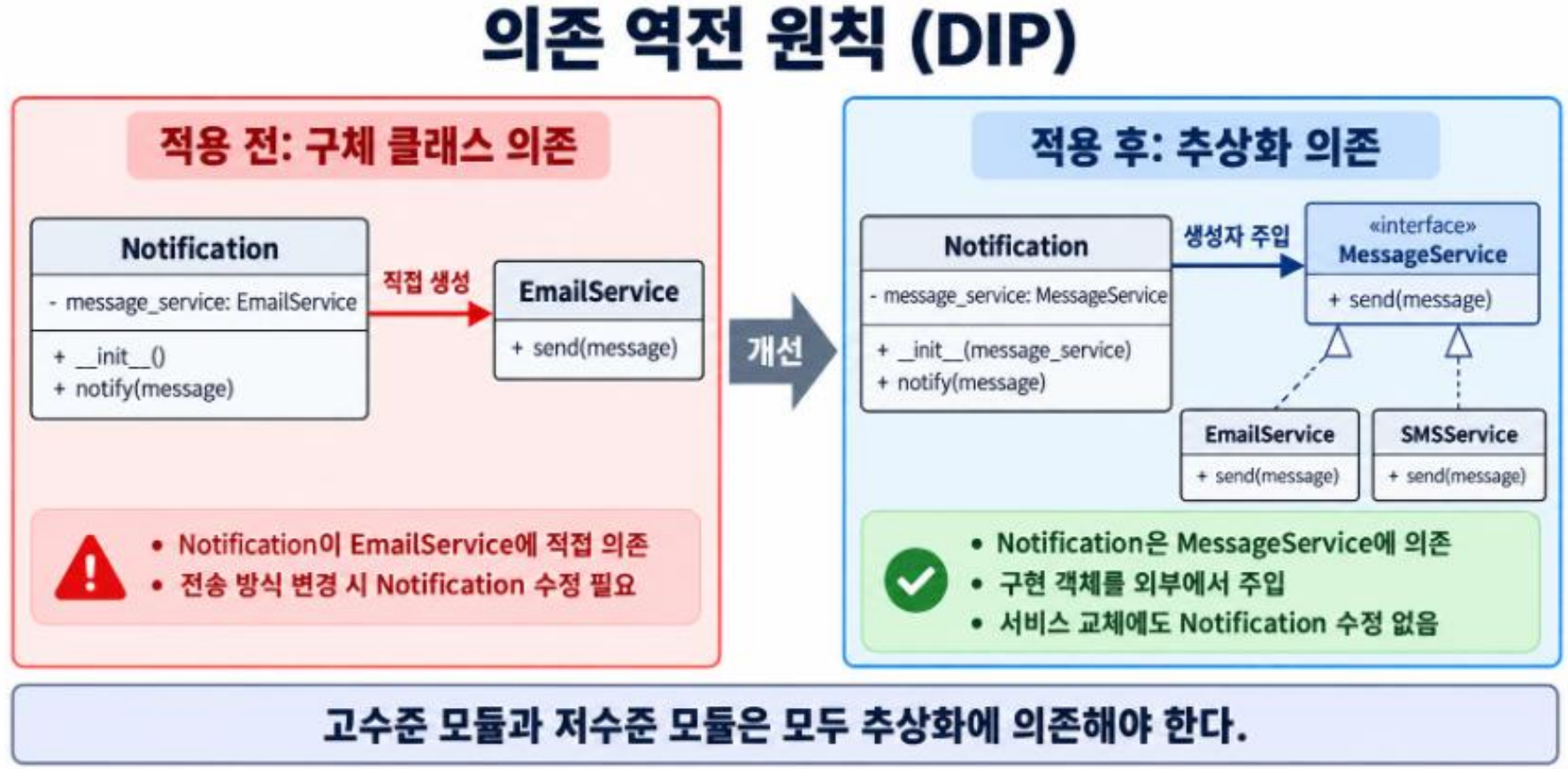
OCP · LSP · ISP · DIP — Python 적용 요약

ISP	쓰지 않는 메서드 강제 구현 (Penguin.fly)	Bird / FlyableBird로 인터페이스 분리



OCP · LSP · ISP · DIP — Python 적용 요약

DIP	구체 클래스에 직접 의존 (EmailService)	MessageService 추상화 + 생성자 주입(DI)



좋은 네이밍의 원칙 (Python)

의도를 표현하라

✗ `d  # 경과 일수`

✓ `elapsed_days`

그릇된 정보 피하기

✗ `account_list  # List 아닐 수도`

✓ `accounts`

의미 있는 구분

✗ `a1, a2 / product_info`

✓ `source, destination`

검색 가능한 이름

✗ `if s == 5:`

✓ `WORK_DAYS = 5`

클래스 = 명사

✗ `class Manager / Processor`

✓ `class Customer / Account`

함수 = 동사

✗ `name() / age()`

✓ `get_name() / is_valid()`

설명 · 듣고 이해 3장 · 3.5

Python 네이밍 & 스타일 — PEP 8

구분	규칙 (PEP 8)	예시
클래스	PascalCase	`CalculatorService`
함수 / 메서드	snake_case	`calculate_total()`
변수	snake_case	`elapsed_time`
상수	UPPER_SNAKE_CASE	`MAX_SIZE`
비공개 멤버	선행 밑줄	`_internal_state`
모듈 / 패키지	짧은 소문자	`cart, utils.math_utils`
들여쓰기	스페이스 4칸	탭 대신 스페이스
최대 라인 길이	79자 (docstring 72자)	`black / ruff` 로 자동 정렬

핵심 VS Code: ruff / black / flake8 로 스타일을 자동 강제할 수 있다 — 사람이 아니라 도구가 스타일을 지킨다.

함수 설계 — 작게, 하나만, 한 수준으로

1 작게 만들어라

20줄 이내가 이상적. 들여쓰기 1~2단계 이내

2 한 가지만 해라

함수 이름으로 설명되는 한 가지 작업만

3 추상화 수준 통일

고수준(비즈니스)과 저수준(I/O)을 섞지 말 것

4 서술적 이름

길고 서술적인 이름이 짧고 의미 없는 이름보다 낫다

5 인수를 최소화

이상 0개, 최대 3개. 초과 시 매개변수 객체 고려

6 부수 효과 없애기

숨겨진 변경 금지. 명령과 조회를 분리(CQS)

Clean Error Handling (Python)

원칙	나쁜 코드 ✗	좋은 코드 ✓
오류 코드 대신 예외	`if delete_page(p) == E_OK:`	`try: delete_page(p) / except ...:`
None 반환하지 않기	`return None  # 책임 떠넘김`	빈 리스트 · 명시적 예외
None 전달하지 않기	`do_something(None)`	가드 절 · Optional 명시
예외에 의미 담기	`raise Exception("error")`	`raise StorageError(...) from e`

```
class StorageError(Exception):
    """저장소 계층의 의미 있는 예외"""

# 예외 체이닝으로 원인 보존 (raise ... from e)
def read_config(path):
    try:
        with open(path, encoding='utf-8') as f:
            return f.read()
    except OSError as e:
        raise StorageError(f'설정 읽기 실패: {path}') from e
```

정적 분석으로 코드 스멜을 자동 탐지 (Python)

정적 분석은 실행 없이 소스코드를 분석해 버그·스타일 위반·복잡도 문제를 찾아낸다.

도구	탐지 내용	연동 방법
Pylance	실시간 타입·문법 경고	VS Code Python 확장 내장
ruff	스타일·버그·복잡도 (초고속)	ruff check . / pre-commit
flake8	코딩 규칙 (PEP 8·복잡도)	flake8 + 설정 파일
mypy	정적 타입 검사	mypy src
bandit	보안 취약 패턴	bandit -r src
pytest-cov	테스트 커버리지 측정	pytest --cov=src

```
$ ruff check . && mypy src && pytest --cov=src    # 한 파이프라인으로 정적 분석 자동 실행
```

디자인 패턴이란? — 자주 만나는 설계 문제의 전형적인 해결책

건축에서 시작 (1977)

- 크리스토퍼 알렉산더『A Pattern Language』
- 건축에서 반복되는 설계 문제와 해결책을 정리
- 소프트웨어 공학에 영감을 줌

GoF 의 정립 (1994)

- Gamma · Helm · Johnson · Vlissides
- 『Design Patterns』— 객체지향 설계의 23개 패턴
- 생성 · 구조 · 행동 3분류

이후의 발전 (2000년대~)

- MVC · 의존성 주입 · 마이크로서비스 패턴 등
- 새 도메인마다 새 패턴 등장
- '모범 사례를 이름 붙여 공유'하는 문화

핵심 패턴 = 재사용 가능한 설계 템플릿(Best Practice). 외우는 것이 아니라 '냄새를 만났을 때 꺼내 쓰는 처방'으로 익힌다.

주의 패턴을 먼저 정하고 코드를 끼워 맞추지 않는다 — 테스트 GREEN 에서 냄새를 확인하고 필요한 패턴만 적용한다.

GoF 23개 패턴 — 생성 · 구조 · 행동

생성 (Creational)	구조 (Structural)	행동 (Behavioral)
Singleton	Adapter	Command
Abstract Factory	Composite	Interpreter
Factory Method ★	Decorator	Iterator
Builder	Facade	Mediator
Prototype	Flyweight	Memento
	Proxy	Observer
		State
		Strategy ★
		Template Method ★
		Visitor ★

참고 ★ = 이 장에서 Python 예제로 다루는 패턴 (Command · Template Method · Factory Method · Strategy · Visitor).

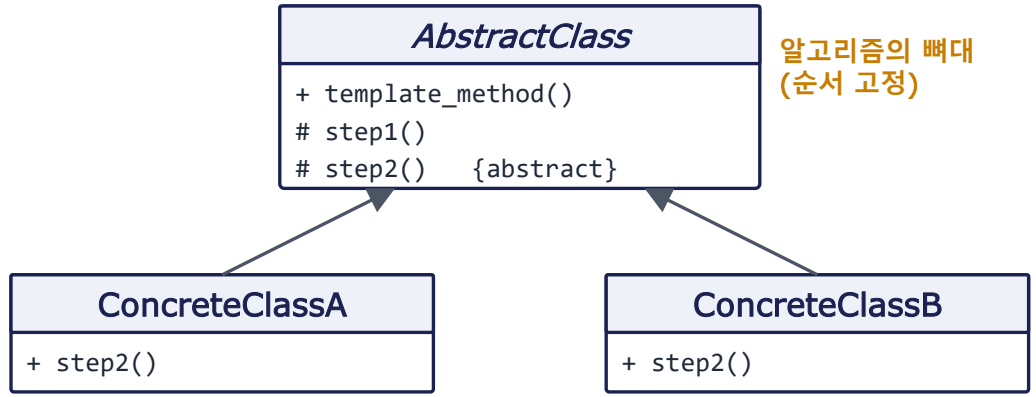
예시 1 · 중복 코드 → Pull Up 과 템플릿 메서드

냄새: 중복 코드(DUPLICATED CODE) — 형제 클래스에 같은 코드가 반복된다.

템플릿 메서드 패턴 (Template Method)

- 알고리즘의 뼈대는 상위 클래스가 정의
- 달라지는 단계만 하위 클래스가 구현(재정의)
- 행동(Behavioral) 패턴 — 구조는 같고 세부만 다를 때

클래스 다이어그램



핵심 Pull Up Field · Pull Up Method — 형제 클래스에 같은 필드 · 메서드가 있으면 부모로 올리고, 다른 부분만 남겨 둔다.

TIP IDE 지원: VS Code 에서는 Pylance 의 Move Symbol · 추출(Extract) 리팩터링, 또는 코딩 에이전트에 '‘Pull Up 해줘’' 로 요청한다. 결과는 반드시 테스트로 확인.

Pull Up Field · Pull Up Method — Developer · Manager

✗ Before — name · get_details 가 두 클래스에 중복

```
class Developer:
    def __init__(self, name, language):
        self.name = name
        self.language = language
    def get_details(self):
        return (f"{self.name} (Developer, "
                f"Language: {self.language})")

class Manager:
    def __init__(self, name, team_size):
        self.name = name
        self.team_size = team_size
    def get_details(self):
        return (f"{self.name} (Manager, "
                f"Team Size: {self.team_size})")
```

✓ After — 부모 Employee 로 올리고 다른 부분만 자식이

```
class Employee(ABC):
    def __init__(self, name):
        self.name = name                    # Pull Up Field
    def get_details(self):                  # Pull Up Method
        return f"{self.name} ({self.role()}, {self.extra()})"
    @abstractmethod
    def role(self): ...
    @abstractmethod
    def extra(self): ...

class Developer(Employee):
    def __init__(self, name, language):
        super().__init__(name)
        self.language = language
    def role(self): return "Developer"
    def extra(self): return f"Language: {self.language}"
# Manager 도 같은 모양 (role · extra 만 다름)
```

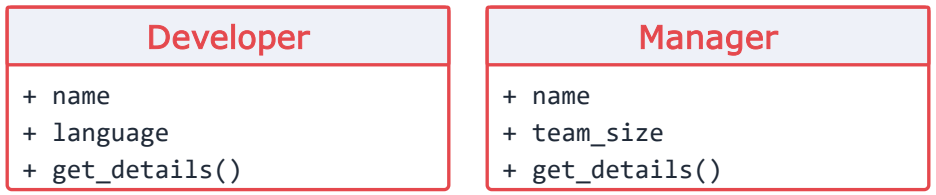
핵심 name 필드와 get_details 뼈대를 Employee 로 올렸다 — Developer · Manager 는 role() · extra() 만 구현.

Pull Up — 클래스 다이어그램 Before / After

같은 필드 · 메서드를 형제 클래스마다 두지 않고 부모로 올린다.

✗ Before — 두 클래스에 같은 name · get_details 가 중복

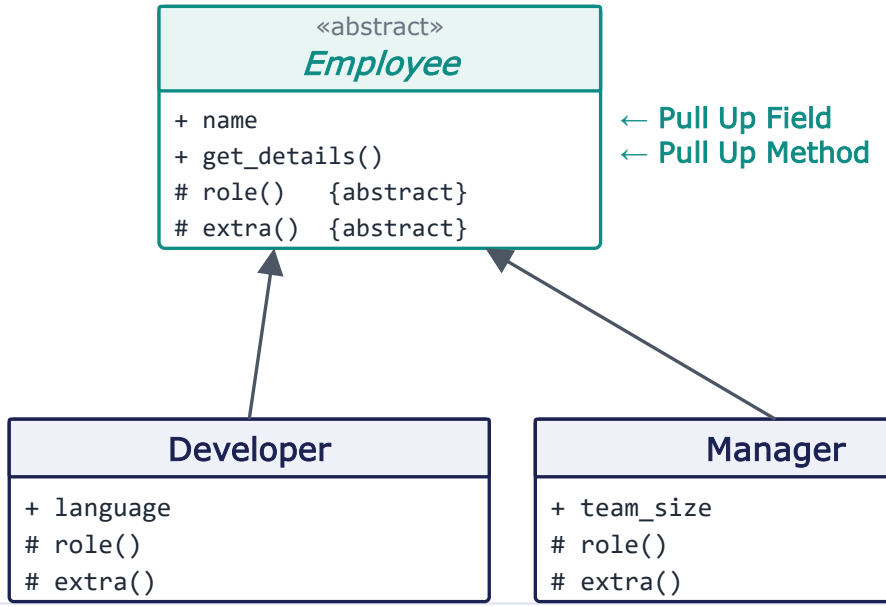
name · get_details 가 똑같이 반복된다



- 중복 → 한쪽만 고치면 서로 달라진다
- 공통 부모가 없어 새 직군마다 복사한다

Pull Up

✓ After — 부모 Employee 로 올리고 다른 부분만 자식이



핵심 공통부(name · get_details 뼈대)는 부모에, 차이(role · extra)만 자식에 — 템플릿 메서드와 같은 모양이 된다.

긴 함수의 중복 → 템플릿 메서드 — 보고서 예제

냄새: 긴 함수(LONG FUNCTION) + 중복 — 머리말·꼬리말이 함수마다 복사되어 있다.

✗ Before — 두 함수가 같은 틀을 복사

```python
def sales_report(rows):
    lines = ["=== 보고서 ===", "제목: 매출"]
    for name, amount in rows:
        lines.append(f"- {name}: {amount:,}원")
    lines.append(f"합계: {sum(a for _, a in rows):,}원")
    lines.append("=== 끝 ===")
    return "\n".join(lines)

def inventory_report(rows):    # ← 같은 틀을 복사
    lines = ["=== 보고서 ===", "제목: 재고"]
    for name, qty in rows:
        lines.append(f"- {name}: {qty}개")
    lines.append(f"합계: {sum(q for _, q in rows)}개")
    lines.append("=== 끝 ===")
    return "\n".join(lines)
```

✓ After — render() 가 뼈대, 하위가 행·합계만

```python
class Report(ABC):
    title = ""
    def render(self, rows):                # 템플릿 메서드
        lines = ["=== 보고서 ===", f"제목: {self.title}"]
        lines += [self.format_row(n, v) for n, v in rows]
        lines.append(self.format_total(rows))
        lines.append("=== 끝 ===")
        return "\n".join(lines)
    @abstractmethod
    def format_row(self, name, value): ...
    @abstractmethod
    def format_total(self, rows): ...

class SalesReport(Report):
    title = "매출"
    def format_row(self, name, value):
        return f"- {name}: {value:,}원"
    def format_total(self, rows):
        return f"합계: {sum(v for _, v in rows):,}원"
```

TIP tests/test_same.py 가 전 · 후 출력 문자열이 같음을 확인한다 — 구조만 바뀌고 결과는 같다.

예시 2 · 함수를 명령으로 바꾸기 (Replace Function with Command)

냄새: 긴 함수 — 여러 단계의 계산이 한 함수에 몰려 있다.

Command 패턴

- 행동(요청)을 객체로 캡슐화
- 요청을 매개변수화 · 저장 · 로깅 가능, 호출자(Invoker)와 수신자 분리
- 구성: Command · Invoker · Receiver
- undo · 여러 단계 메서드로 쪼개기가 쉬워진다

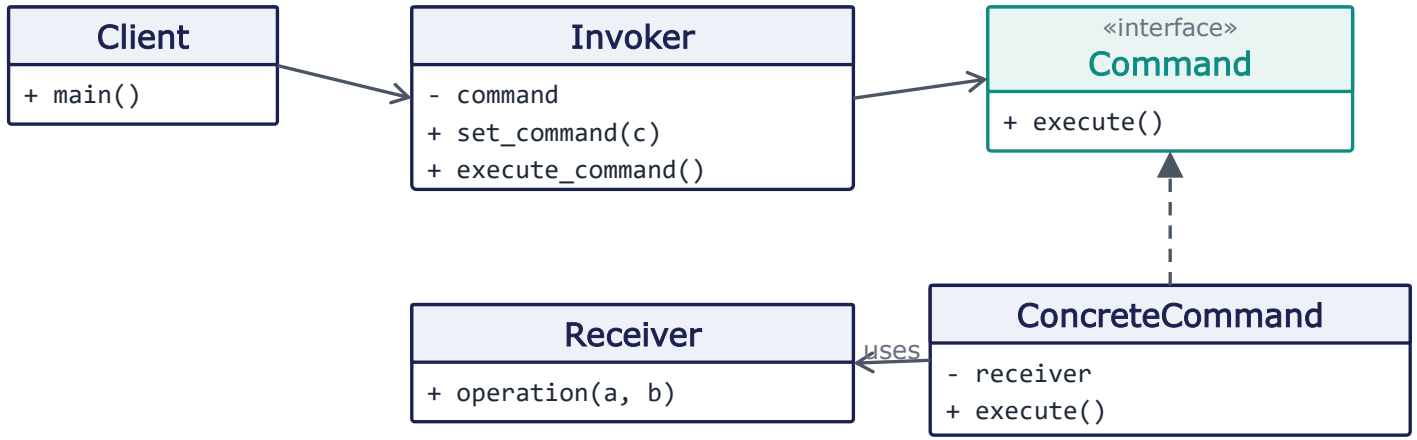
실생활 비유 — 식당 주문서

- 손님의 주문이 종이(주문서)로 적혀 요리사에게 전달된다.
- 주문서가 '커맨드' — 누가 언제 요리하든 같은 주문서로 같은 요리가 나온다.
- 웨이터(Invoker)는 요리 방법을 몰라도 주문서만 전달하면 된다.

주의 대부분은 '커맨드'보다 '함수'가 낫다. 함수로 해결되지 않을 때(되돌리기 · 큐잉 · 여러 단계 상태)만 Command 로 바꾼다.

Command 패턴 구조 — 클래스 다이어그램

호출자(Invoker)는 명령(Command)의 execute() 만 안다. 실제 일은 수신자(Receiver)가 한다.



우리 예제와 대응

역할	코드
Invoker	Order.calculate_total_price()
Command	TotalPriceCalculator
Receiver	order.items (데이터)
Client	Order(items) 를 쓰는 쪽

실생활 비유 — 식당 주문서 (종이 주문서가 Command)

손님
주문을 말한다

→

주문서
Command · 요청을 객체로

→

웨이터
Invoker · 전달만

→

요리사
Receiver · 실제 요리

참고 주문서(Command)가 있으니 웨이터는 요리를 몰라도 되고, 주문서를 모아 두거나(큐잉) 취소(undo)할 수도 있다.

함수를 명령으로 바꾸기 — Before / After

✗ Before — Order 가 합계 · 할인을 모두 계산

```
class Order:
    def __init__(self, items):
        self.items = items
    def calculate_total_price(self):
        total = 0
        for _, price, qty in self.items:
            total += price * qty
        if total > 1000:
            total = total * 9 // 10   # 할인
        return total
```

✓ After — 계산은 TotalPriceCalculator(Command), Order 는 호출만

```
class TotalPriceCalculator:           # Command
    def __init__(self, order):
        self.order = order
    def execute(self):
        return self._apply_discount(self._subtotal())
    def _subtotal(self):
        return sum(p * q for _, p, q in self.order.items)
    def _apply_discount(self, total):
        return total * 9 // 10 if total > 1000 else total

class Order:                           # Invoker
    def __init__(self, items):
        self.items = items
    def calculate_total_price(self):
        return TotalPriceCalculator(self).execute()
```

핵심 Order(Invoker)는 계산을 Command 에 위임 — 단일 책임(SRP). 할인 정책을 바꿀 때 Command 만 고치면 된다.

TIP 순서: ① 테스트 GREEN ② 클래스 추출(Extract Class) ③ 메서드 추출 ④ 다시 GREEN.

함수를 명령으로 바꾸기 — 실행 결과와 장점

Python

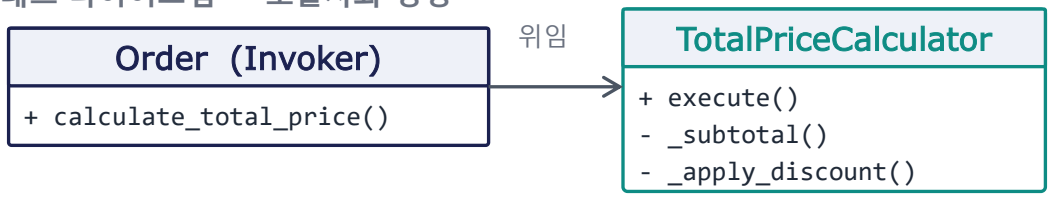
```
>>> from after_mod import Order
>>> items = [("노트북", 1200, 1), ("마우스", 50, 2)]
>>> Order(items).calculate_total_price()
1170                    # 1300 > 1000 → 10% 할인

> pytest -q
6 passed in 0.07s       # 전 · 후 동일
```

리팩토링 후의 장점

- 단일 책임 원칙(SRP): 계산 로직과 Order 의 역할이 분리
- 새 할인 정책은 Command 객체만 수정
- 재사용: TotalPriceCalculator 를 다른 곳에서도 사용
- 가독성: Order 코드가 간결해지고 역할이 분명

클래스 다이어그램 — 호출자와 명령



원칙 리팩토링 확인의 순서 — 전 · 후 같은 입력을 넣어 같은 결과가 나오는지(test_same.py) → 전체 테스트 GREEN.

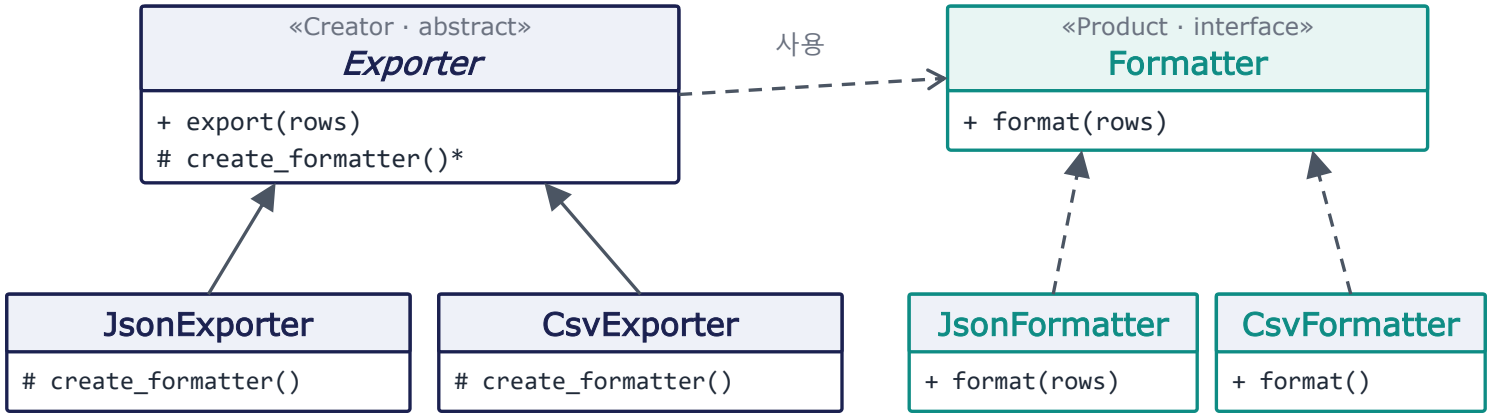
예시 3 · 팩토리 메서드 패턴 (Factory Method)

객체 생성을 하위 클래스에 맡겨, 만드는 쪽과 쓰는 쪽을 분리한다 — 생성(Creational) 패턴.

목적

- 객체 생성 코드를 클라이언트에서 분리
- 생성 방식을 바꿔도 클라이언트는 그대로
- 다형성으로 '무엇을 만들지'를 위임

클래스 다이어그램 — Exporter / Formatter



Exporter 하위 클래스가 해당 Formatter 를 생성한다 (팩토리 메서드)

핵심 if kind == "json" … elif kind == "csv" … 분기가 새 형식마다 늘어나면 Factory Method 를 떠올린다.

참고 Python 에서는 클래스를 값으로 넘기거나 dict 에 등록하는 '간단한 팩토리'로도 충분한 경우가 많다.

팩토리 메서드 예시 — 내보내기(Exporter)

✗ Before — 형식마다 if 분기

```
def export(rows, kind):
    if kind == "json":
        return json.dumps(rows, ensure_ascii=False)
    if kind == "csv":
        buf = io.StringIO()
        csv.writer(buf, lineterminator="\n"
                   ).writerows(rows)
        return buf.getvalue()
    raise ValueError(kind)
```

✓ After — 하위 클래스가 만들 객체를 결정

```
class Formatter(ABC):                       # Product
    @abstractmethod
    def format(self, rows): ...

class JsonFormatter(Formatter):             # ConcreteProduct
    def format(self, rows):
        return json.dumps(rows, ensure_ascii=False)
# CsvFormatter 도 같은 모양

class Exporter(ABC):                        # Creator
    def export(self, rows):
        return self.create_formatter().format(rows)
    @abstractmethod
    def create_formatter(self): ...         # 팩토리 메서드

class JsonExporter(Exporter):               # ConcreteCreator
    def create_formatter(self):
        return JsonFormatter()
```

TIP 새 형식(XML) = Formatter 하나 + Exporter 하나 추가. 기존 코드는 한 줄도 바꾸지 않는다(OCP).

냄새 · 기능 편애(FEATURE ENVY) — 남의 데이터를 너무 많이 쓴다

무엇이 문제인가

- 메서드가 자기 클래스보다 다른 클래스의 데이터를 더 많이 사용
- 응집도 감소 · 결합도 증가 · 유지보수성 저하
- 예) 다른 객체의 getter 를 여러 개 부르는 메서드

처방

- 함수 옮기기(Move Function / Move Method) — 가장 많이 쓰는 클래스로
- 일부분만 부러워하면 함수 추출 후 옮기기
- 데이터와 그 데이터를 쓰는 행동은 같은 곳에

개선 기법 (리팩토링 기법 12가지 중 — 기능 편애는 '메서드 이동')

메서드 추출 (Extract Method)	메서드 상향 (Pull Up Method)	알고리즘 전환 (Substitute Algorithm)	조건식 쪼개기 (Decompose Conditional)
매개변수 세트를 객체로 (Parameter Object)	객체째 통째로 전달 (Preserve Whole Object)	메서드로 매개변수 교체	클래스 추출 (Extract Class)
메서드 이동 (Move Method)	클래스 내용 직접 삽입 (Inline Class)	필드 이동 (Move Field)	서브클래스로 전환 (Type Code → Subclasses)

참고 예외: 데이터와 행동을 일부러 분리하는 설계(전략 · 방문자 패턴)는 기능 편애가 아니라 의도된 구조다 — 이어지는 예시.

기능 편애 리팩토링 — Move Method

✗ Before — Invoice 가 Address 의 데이터만 잔뜩 쓴다

```python
@dataclass
class Address:
    street: str
    city: str
    zip_code: str

class Invoice:
    def customer_info(self, customer):
        a = customer.address   # Address 의 데이터만 잔뜩
        return f"{a.street}, {a.city} {a.zip_code}"
```

✓ After — 포맷 책임을 Address 로 이동

```python
@dataclass
class Address:
    street: str
    city: str
    zip_code: str
    def format(self):                        # Move Method
        return f"{self.street}, {self.city} {self.zip_code}"

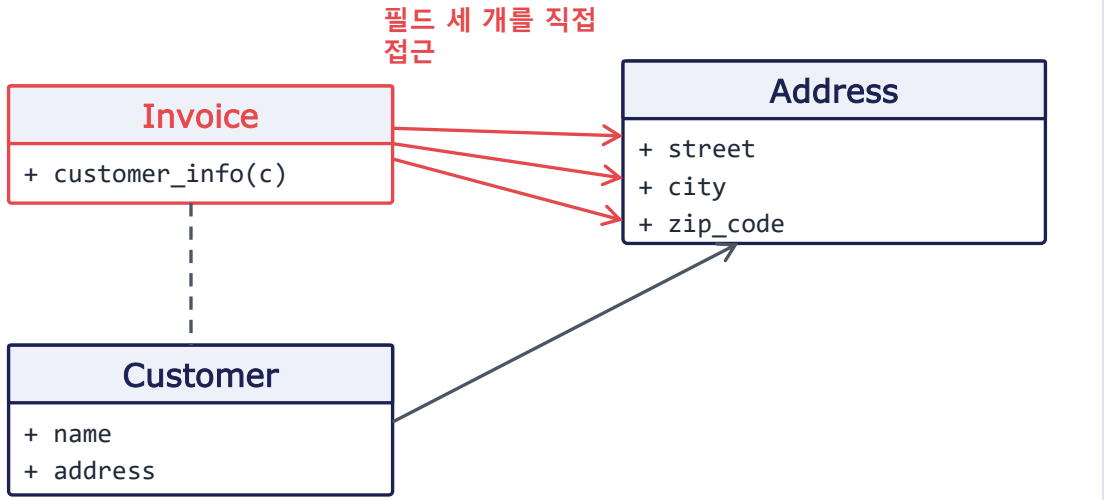
class Invoice:
    def customer_info(self, customer):
        return customer.address.format()   # 메시지만 전달
```

핵심 customer_info() 는 Address 의 street · city · zip_code 만 썼다 → Address.format() 으로 이동, Invoice 는 메시지만 전달.

기능 편애 — 클래스 다이어그램 Before / After

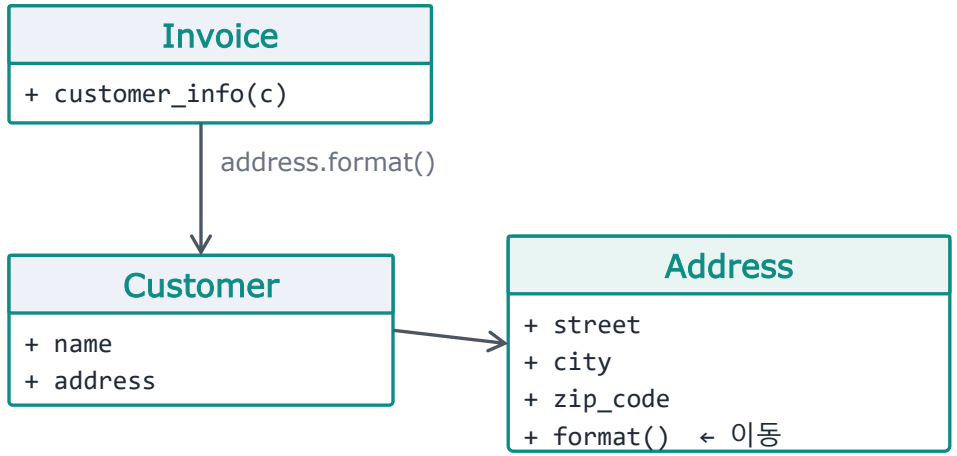
메서드가 데이터가 있는 클래스로 이동하면 의존이 한 줄로 줄어든다.

✗ Before — Invoice 가 Address 의 필드 세 개를 직접 사용



Move Method

✓ After — 포맷 책임을 Address 로 이동



핵심 Invoice 는 Customer 에게 메시지만 전달 — Address 의 내부 구조(필드 세 개)를 몰라도 된다. 응집도 ↑ 결합도 ↓

전략 패턴 (Strategy) — 행위를 캡슐화해 바꿔 끼운다

"무엇을 할지"를 외부에서 주입받고, 런타임에 알고리즘(전략)을 선택한다.

실생활 비유 — 결제 수단

- 전략 A 신용카드 · B 페이팔 · C 간편결제
- 주문(Context)은 결제 방법을 몰라도 된다

언제 쓰면 좋을까?	적합
조건문이 많고 if/elif 로 알고리즘을 분기	YES
행위를 런타임에 바꾸고 싶을 때	YES
같은 인터페이스 아래 다양한 알고리즘이 존재	YES
알고리즘을 분리해 따로 테스트하고 싶을 때	YES

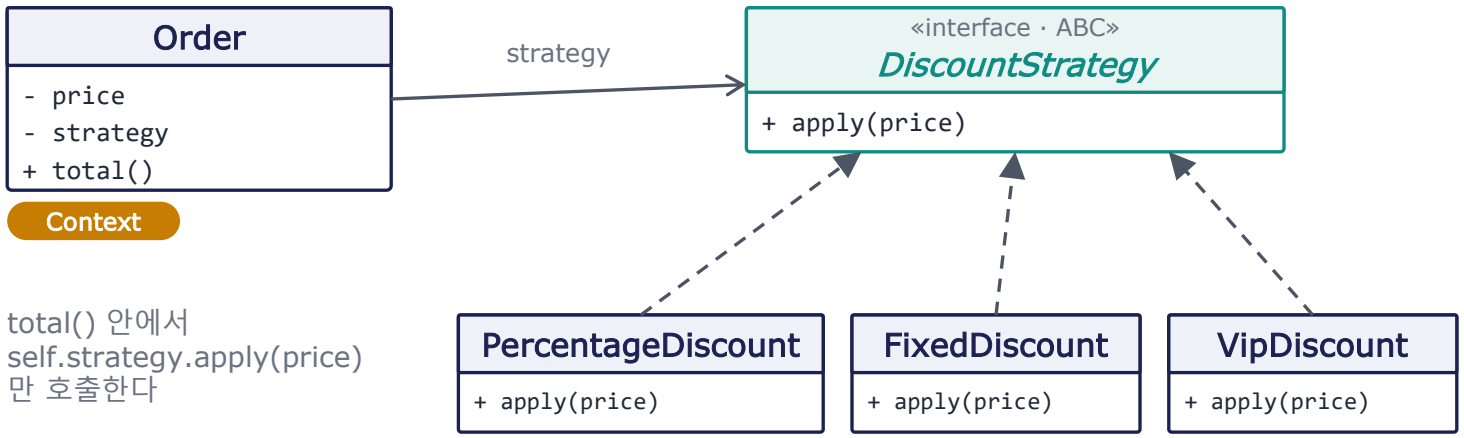
구조

- Strategy(인터페이스) ← ConcreteStrategy A · B · C / Context 가 Strategy 를 보관하고 위임
- Python: ABC 로 인터페이스를 쓰거나, 함수 · 호출 가능 객체를 그대로 전략으로 넘겨도 된다

핵심 새 알고리즘은 새 클래스(또는 함수) 추가만으로 — if/elif 를 고치지 않는다(OCP). 오늘 확인 실습의 요구 5 가 바로 이것이다.

전략 패턴 — 클래스 다이어그램 (할인 정책)

Context(Order)가 Strategy 인터페이스만 알고, 구체 알고리즘은 바꿔 끼운다.



일반 용어 ↔ 우리 코드

역할	코드
Context	Order
Strategy	DiscountStrategy
ConcreteStrategy	Percentage · Fixed · Vip
교체 시점	order.strategy = FixedDiscount()

핵심 새 할인 정책 = DiscountStrategy 를 구현한 클래스 하나 추가. Order 와 기존 전략은 한 줄도 바뀌지 않는다(OCP).

전략 패턴 예시 — 할인 정책 (정률 · 정액 · VIP)

✗ Before — kind 문자열로 if 분기

```
def apply_discount(price, kind):
    if kind == "percent":
        return price * 90 // 100
    if kind == "fixed":
        return price - 1000
    if kind == "vip":
        return price * 85 // 100
    raise ValueError(kind)
```

✓ After — 정책마다 전략 클래스, Order 는 위임

```
class DiscountStrategy(ABC):              # Strategy
    @abstractmethod
    def apply(self, price): ...

class PercentageDiscount(DiscountStrategy):   # 정률 10%
    def apply(self, price):
        return price * 90 // 100
class FixedDiscount(DiscountStrategy):        # 정액 1,000원
    def apply(self, price):
        return price - 1000
class VipDiscount(DiscountStrategy):          # VIP 15%
    def apply(self, price):
        return price * 85 // 100

class Order:                                  # Context
    def __init__(self, price, strategy):
        self.price, self.strategy = price, strategy
    def total(self):
        return self.strategy.apply(self.price)
```

TIP strategy 를 바꿔 끼우면 같은 Order 로 다른 할인이 된다 (test_strategy_can_be_swapped_at_runtime).

Gilded Rose 에 전략 적용 ① — 문제 구조

아이템 종류마다 갱신 방식이 다르다 → 한 함수에 분기가 쌓인다 (OCP 위반).

before_mod.py — update_quality() (요약)

```
def update_quality(items):
    for item in items:
        if item.name == "Aged Brie":
            ...   # 품질 +1, 판매일이 지나면 +2 (최대 50)
        elif item.name == "Sulfuras":
            pass  # 아무 것도 안 함
        elif item.name == "Backstage":
            ...   # 11일 미만 +2, 6일 미만 +3, 공연 후 0
        else:
            ...   # 일반: -1, 판매일이 지나면 -2 (최소 0)
```

무엇이 문제인가

- 이름(문자열)에 따른 if / elif 가 한 함수에 모여 있다
- 새 아이템을 추가할 때마다 update_quality() 가 변경된다 → OCP 위반
- 한 종류의 규칙을 고치다 다른 종류를 깨뜨리기 쉽다
- 그래서 먼저 30일 시뮬레이션 골든 마스터를 만든다

원칙 행위(Behavior)가 아이템 종류에 따라 변한다 → 종류별 규칙을 전략 객체로 캡슐화하는 것이 전략 패턴의 전형적인 적용이다.

Gilded Rose 에 전략 적용 ② — 전략 클래스와 사용

after_mod.py — 전략 클래스들

```python
class QualityUpdateStrategy(ABC):          # Strategy
    @abstractmethod
    def update(self, item): ...

class AgedBrieStrategy(QualityUpdateStrategy):
    def update(self, item):
        if item.quality < 50:
            item.quality += 1
        item.sell_in -= 1
        if item.sell_in < 0 and item.quality < 50:
            item.quality += 1

class SulfurasStrategy(QualityUpdateStrategy):
    def update(self, item):
        pass                               # 아무 것도 안 함

# BackstageStrategy · DefaultStrategy 도 같은 모양
# (전체: dp_strategy_gilded_rose/src/after_mod.py)
```

after_mod.py — 등록과 호출

```python
STRATEGIES = {
    "Aged Brie": AgedBrieStrategy(),
    "Sulfuras": SulfurasStrategy(),
    "Backstage": BackstageStrategy(),
}
DEFAULT = DefaultStrategy()

def update_quality(items):
    for item in items:
        STRATEGIES.get(item.name, DEFAULT).update(item)
```

확인

- test_same.py: 4종 × 8 × 5 조합을 30일 돌려 전 · 후 비교
- 새 아이템 = 전략 클래스 + 등록 한 줄

핵심 update_quality() 는 이제 '전략 고르기 + 호출'만 한다. 부록 B 의 템플릿 메서드 + 팩토리 구조와 같은 목표를 다른 길로 간 것.

방문자 패턴 (Visitor) — 데이터는 그대로, 연산만 확장

객체 구조는 바꾸지 않고, 객체에 수행할 연산을 분리하는 행동(Behavioral) 패턴.

구성

- Element: accept(visitor) 를 가진 데이터 (Book · Fruit)
- Visitor: 종류마다 visit_* 메서드를 선언
- ConcreteVisitor: 연산 하나(가격 · 라벨)

언제 쓰나

- 데이터 클래스는 안정적이고 연산이 자주 늘 때
- if isinstance(...) 분기가 연산마다 반복될 때
- 연산을 모아서 따로 테스트하고 싶을 때

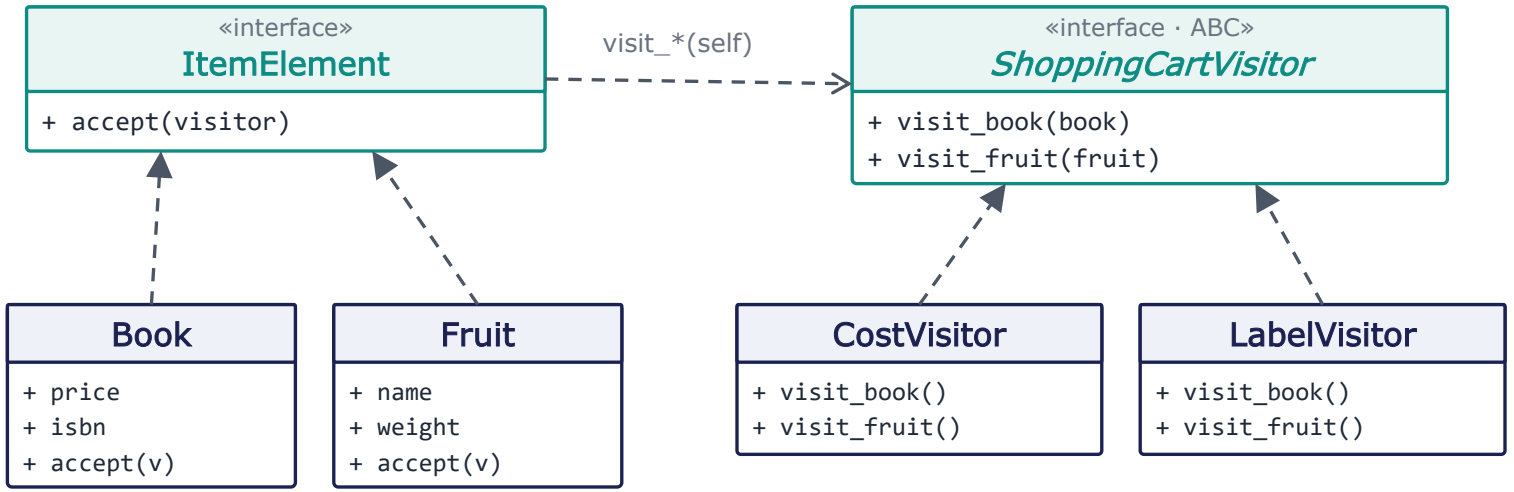
주의

- Element 종류가 자주 늘면 Visitor 를 모두 고쳐야 한다
- Python 은 functools.singledispatch 로 더 간단히도 가능
- 단순하면 그냥 함수 · if 가 낫다

핵심 전략은 '알고리즘 하나를 갈아 끼움', 방문자는 '구조를 돌며 종류별로 연산 수행' — 데이터와 행동을 분리하는 두 가지 방식이다.

방문자 패턴 — 클래스 다이어그램과 호출 순서

Element 는 accept(visitor) 만 가지고, 종류별 연산은 Visitor 가 가진다 (double dispatch).




```
Client:  total = sum(item.accept(CostVisitor()) for item in items)
```

호출 순서 (double dispatch)

① client: item.accept(visitor)

② Book.accept(): visitor.visit_book(self)

③ CostVisitor.visit_book(): 가격 반환

④ Fruit 이면 visit_fruit() 로 — 종류별 분기가 사라진다

새 연산 = 새 Visitor 클래스. Book · Fruit 는 그대로.

주의 Element 종류(Book · Fruit …)가 자주 늘면 모든 Visitor 를 고쳐야 한다 — 데이터가 안정적이고 연산이 자주 늘 때만 쓴다.

방문자 패턴 예시 — 장바구니 항목(Book · Fruit)

✗ Before — 연산마다 isinstance 분기

```
def total_cost(items):
    total = 0
    for item in items:
        if isinstance(item, Book):
            total += item.price
        elif isinstance(item, Fruit):
            total += (item.price_per_kg
                      * item.weight)
    return total
```

✓ After — accept(visitor) · Visitor 에 연산을 모은다

```
class Book:                              # Element
    def accept(self, visitor):
        return visitor.visit_book(self)
class Fruit:
    def accept(self, visitor):
        return visitor.visit_fruit(self)

class ShoppingCartVisitor(ABC):          # Visitor
    @abstractmethod
    def visit_book(self, book): ...
    @abstractmethod
    def visit_fruit(self, fruit): ...

class CostVisitor(ShoppingCartVisitor):  # ConcreteVisitor
    def visit_book(self, book):
        return book.price
    def visit_fruit(self, fruit):
        return fruit.price_per_kg * fruit.weight

total = sum(i.accept(CostVisitor()) for i in items)
```

TIP LabelVisitor 를 추가해도 Book · Fruit 는 한 줄도 바뀌지 않았다 — 새 연산 = 새 Visitor.

냄새 → 리팩토링 → 패턴 — 한 장으로 정리

코드 냄새	리팩토링 기법	관련 패턴	예제 폴더
중복 코드 · 긴 함수	Pull Up Field/Method · 메서드 추출	Template Method	dp_pull_up · dp_template_method
긴 함수 (여러 단계 계산)	함수를 명령으로 바꾸기	Command	dp_command
반복되는 if/switch (생성)	생성 코드를 하위 클래스로	Factory Method	dp_factory_method
기능 편애	함수 옮기기(Move Method)	(전략 · 방문자는 의도된 예외)	dp_feature_envy
반복되는 if/switch (행위)	조건문을 다형성으로	Strategy	dp_strategy · dp_strategy_gilded_rose
종류별 연산 분기 (isinstance)	연산을 Visitor 로 분리	Visitor	dp_visitor

원칙 모든 예제는 '전 · 후 같은 입력 → 같은 결과'(test_same.py)로 확인했다. 패턴 적용도 리팩토링 — 테스트 GREEN 에서만, 한 번에 하나씩.

참고 다음 절(3.10)의 QA 프롬프트로 코딩 에이전트에게 냄새 진단 · 패턴 적용 계획을 맡길 때 같은 순서를 지시한다.

QA를 위한 프롬프트 엔지니어링 — 5요소

막연한 "고쳐줘"는 범위가 폭주한다. QA 프롬프트는 아래 5요소를 명시할수록 결과가 안정적이다.

1

역할/관점

"보안 리뷰어 관점에서"

2

맥락

"calc 함수만, 동작 변경 없이"

3

작업

"스멜을 심각도 순으로 나열"

4

제약

"파일 수정 말고 보고만"

5

출력 형식

"표: 위치 | 문제 | 위험 | 제안"

핵심 출력 형식을 지정하면 결과를 그대로 QA 리포트·커스텀 커맨드 절차로 재활용할 수 있다 — 형식은 자동화의 입력이 된다.

작업별 프롬프트 패턴 — 진단 · 리뷰 · 리팩터 · 테스트

▶ 진단

«대상»의 코드 스멜·중복·복잡도를 찾아줘. 수정 말고 [위치|유형|근거|난이도] 표로, 우선순위 순으로 정리해줘.

▶ 리뷰

방금 변경을 «보안|회귀|성능» 관점으로만 리뷰해줘. 동작이 바뀔 지점과 그걸 잡을 테스트가 없는 부분을 우선순위로.

▶ 리팩터

«함수»의 외부 동작을 바꾸지 말고 내부 구조만 개선해줘. 먼저 계획을 보여주고 승인 후 적용, 매 단계 후 테스트 실행.

▶ 테스트

«함수»의 테스트 갭을 찾아줘(경계·예외·중복). 빠진 케이스를 먼저 보여주고, 합의된 케이스만 테스트로 추가해줘.

원칙 범위 폭주를 막는 한 줄 — "먼저 계획을 보여주고 승인 후 진행. 요청하지 않은 변경은 하지 마."

보기만 C:\DEV\tdd-course\demo
강사 PC 화면을 본다 · 이어지는 실습에서 직접 한다

Code Smell & Refactoring — 진단은 plan, 수정은 한 번에 하나

① 읽기 전용으로 진단

```
$ gemini --approval-mode plan
# 읽기 전용 세션: 파일 수정 불가
```

▶ 진단 (plan 모드)

@src/user_service.py 의 코드 스멜을 찾아줘. 수정하지 말고 [위치 | 스멜 | 근거 | 처방] 표로, 우선순위 순으로 보고해줘.

▶ 리팩터 (default 모드)

1순위 스멜(중복 코드 — 3의 법칙) 하나만 고쳐줘.
format_full_name() 으로 추출하고 외부 동작은 바꾸지 마. 적용 전 계획을 먼저 보여주고, 끝나면 !pytest -q 결과를 보고해.

확인할 것

- pytest가 리팩터 전후 모두 GREEN인가
- 테스트 파일이 수정되지 않았는가
- 한 번에 스멜 하나만 처리했는가 (Tidy First)

체험 탐지(plan)와 실행(default)을 분리하면 "고치다 보니 다른 것도 바꿨다"는 범위 폭주가 사라진다 — 6교시 리팩터링 실습과 부록 B의 기본 리듬이다.

이어서 사용 C:\DEV\tdd-course\cart
막히면 시작점 checkpoints\2.4_TDD2완료\cart

14:00–14:50

리팩터링 · 실험 B — 진행 방식

폴더

`cart/`

방식

리팩터링: 짝
실험 B: 개인 → 짝 비교

결과물

19 passed 유지
기록표 B 열

프롬프트

cart/prompts.md · Day 2 · 6교시

#	어디서	할 일	완료 기준	슬라이드
1	**터미널 ①**	gemini --approval-mode plan → 후보 받기	후보 3개 이하	200
2	**터미널 ①**	gemini (기본 모드) → 1단계 프롬프트	19 passed	201
3	Gemini	2단계 프롬프트	19 passed	201
4	Gemini	실험 B: 어제 A-4 와 같은 요청	테스트가 잡는지 확인	202
5	기록표	A 열과 B 열 비교 → 모둠 공유	차이 한 가지	203

TIP 리팩터링은 GREEN 에서만. 한 단계마다 pytest 를 실행해 19 passed 가 유지되는지 확인한 뒤 다음 단계로 진행.

지금 할 일 cart 폴더(앞에서 연 것)에서 ▸ 아래 표를 1단계부터 차례로 · 프롬프트는 폴더의 prompts.md 에서 복사 · 막히면 손을 들어 주세요

실습 · 직접 하기 Day 2 · 6교시 | 리팩터링

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

14:00–14:50

먼저 계획만 — plan 모드로 리팩터링 후보 선택하기

@src/cart.py 모든 테스트가 통과하는 상태야. 동작은 바꾸지 않고 읽기 쉽게 만들 리팩터링 후보를 작은 것부터 3개 이하로 제안만 해줘. 파일은 수정하지 마.

결과 예시 응답 문구는 실행마다 다를 수 있다

Gemini CLI (plan 모드)

✦ 리팩터링 후보 (작은 것부터)
1. 매직 넘버 50000 · 90 · 95 를 이름 있는 상수로 (THRESHOLD, THRESHOLD_PERCENT, VIP_PERCENT)
2. '금액 × 퍼센트 // 100' 계산이 두 번 반복 → _percent_of(amount, percent) 헬퍼
3. (선택) subtotal의 입력 검사를 별도 함수로
각 단계 후 pytest로 19개 통과(골든 마스터 포함)를 확인하는 것을 권한다.

원칙 한 번에 하나씩. 1번을 적용하고 GREEN 확인 → 커밋 → 2번. 두 개를 한꺼번에 하면 깨졌을 때 원인을 모르게 된다.

지금 할 일 터미널: gemini --approval-mode plan ▸ 리팩터링 후보 받기 (파일 수정 없음) ▸ 짝과 검토

실습 · 직접 하기 Day 2 · 6교시 | 리팩터링

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

후보 1·2를 차례로 적용 — 매번 19 passed

REFACTOR

▶ 프롬프트 — 1단계

@src/cart.py 모든 테스트가 통과하는 상태야.
동작은 바꾸지 말고 매직 넘버 50000, 90, 95 를 이름 있는 상수로 바꾸는 것만 해줘.
테스트 파일은 수정하지 마. 끝나면 !pytest -q --tb=short 결과를 보여줘.

▶ 프롬프트 — 2단계

좋아. 이번에는 '× 퍼센트 // 100' 반복을 _percent_of 헬퍼로 뽑는 것만 해줘. 나머지 조건은 같아.

각 단계 후 터미널

```
> pytest -q --tb=short
..................  [100%]
19 passed in 0.02s
```

결과 예시 src/cart.py — 2단계 후

```python
THRESHOLD = 50_000            # 문턱 할인 기준 금액 (이상)
THRESHOLD_PERCENT = 90        # 문턱 할인 후 남는 비율
VIP_PERCENT = 95              # VIP 추가 할인 후 남는 비율


def subtotal(items):
    for item in items:
        if item["price"] < 0 or item["qty"] < 0:
            raise ValueError("가격과 수량은 0 이상이어야 합니다")
    return sum(item["price"] * item["qty"] for item in items)


def _percent_of(amount, percent):
    return amount * percent // 100


def apply_threshold_discount(amount):
    if amount >= THRESHOLD:
        return _percent_of(amount, THRESHOLD_PERCENT)
    return amount


def final_total(items, is_vip=False):
    amount = apply_threshold_discount(subtotal(items))
    if is_vip:
        amount = _percent_of(amount, VIP_PERCENT)
    return amount
```

지금 할 일 터미널: gemini (기본 모드) ▸ 1단계 프롬프트 → 19 passed ▸ 2단계 프롬프트 → 19 passed

실습 · 직접 하기 Day 2 · 6교시 | 실험 B

이어서 사용 C:\DEV\tdd-course\cart
위에서 연 폴더 그대로 · 프롬프트 cart\prompts.md

실험 B — 어제 A-4 와 같은 요청을 테스트가 있는 상태에서

같은 요청('50000원 이하는 할인 안 되게' 포함)을 지금의 cart/에서 해 본다. AI가 또 >= 를 > 로 바꾼다면?

▶ 프롬프트 — 실험 A-4와 같은 요청

@src/cart.py 코드가 좀 지저분하네. 50000원 이하는 할인 안 되게 해. 깔끔하게 정리해줘. 끝나면 !pytest -q 결과를 보여줘.

결과 예시 AI가 같은 실수를 했을 때

Gemini CLI

```
✦ 조건을 정리했다.
  Edit src/cart.py
-     if amount >= THRESHOLD:
+     if amount > THRESHOLD:
✦ !pytest 결과 — 3개 실패:
FAILED tests/test_cart.py::test_threshold_discount_boundary[50000-45000]
FAILED tests/test_cart.py::test_non_vip_gets_threshold_discount_only
FAILED tests/test_golden_master.py::test_golden_master
3 failed, 16 passed in 0.02s
✦ 경계 조건을 바꾼 것이 원인이다. 되돌리겠다. → 19 passed
```

실험 A에서는

- 5만 원 손님이 할인을 못 받는데 아무도 몰랐다

실험 B에서는

- AI가 같은 실수를 해도 테스트(와 골든 마스터)가 바로 잡고, AI 스스로 되돌렸다

지금 할 일 Gemini(cart/): 실험 A-4 와 같은 요청 ▸ pytest 결과 확인 ▸ 기록표 B 열 작성

토론 · 짝 활동 Day 2 · 6교시 | 비교

실험 A vs 실험 B — 기록표로 비교

아래는 작성 예시. 각자의 기록표로 직접 채우고 짝과 공유하고 검토.

항목	실험 A (AI만)	실험 B (AI + TDD)
요구사항 충족	4개 중 3개 (5만 원 경계 깨짐)	7개 규칙 모두 (C1~C7)
변경 후 깨진 기존 기능	1개 — 발견 못 함	깨졌지만 즉시 발견, 되돌림
AI에게 다시 요청한 횟수	2~3회 (소수점, 정리)	루프마다 1회
문제를 발견한 방법	우연히 / 발견 못 함	테스트 실패 메시지
AI가 몰래 정한 것	float 금액, 반올림, 경계	없음 — 예시 표가 정함
리팩터링 후 자신감 (1~5)	2	5

원칙

테스트가 AI의 실수를 없애 주는 것은 아니다. 실수를 바로 발견해서 잘못된 코드를 그대로 사용하지 않게 해 준다.

- 테스트 없음: AI의 실수를 발견하지 못하고 완료할 수 있음
- 테스트 있음: 같은 실수가 발생해도 실패로 표시되어 수정하게 됨

지금 할 일 각자 기록표 A · B 열 채우기 → 짝과 작업한 내용 하나씩 공유

실습 · 직접 하기 Day 2 · 7교시 | 확인 실습

제공 폴더 C:\DEV\tdd-course\ship
starter\ship 복사 (요구사항 docs\ 포함) · 프롬프트 ship\prompts.md

15:00–16:20

확인 기준 실습 — 배송비 계산기 (개인 80분 + 짝 검토 30분)

Gemini CLI 사용 가능 · 폴더 ship/ (pytest.ini · src/ · tests/ · docs/ 준비됨) · shipping_fee(order_amount) 를 TDD 로

처음부터 주어지는 요구

- 1. 기본 배송비는 3,000원이다.
- 2. 주문 금액이 30,000원 이상이면 배송비는 무료이다.
- 3. 주문 금액이 0원 이하이면 오류(ValueError)이다.

40분 경과 시 강사가 발표

요구 4 (요구사항 변경)

- GOLD 회원은 20,000원 이상이면 무료이다.
- shipping_fee(order_amount, grade="NORMAL")
- 기존 요구 1~3 은 그대로 — 회귀를 확인한다

60분 경과 시 강사가 발표

요구 5 (리팩터링 과제)

- 등급별 규칙을 if / dict 분기가 아닌 전략(Strategy) 객체로 분리
- ShippingPolicy(ABC) + 등급별 클래스, shipping_fee() 는 위임만
- 새 등급(예: SILVER)은 클래스 하나만 추가 — shipping_fee() 수정 0줄
- 동작은 그대로: 골든 마스터 + 전 · 후 GREEN

주의 요구 4 · 5 는 발표 시점에 시작한다. 처음부터 하면 '요구사항 변경 후 회귀'(기준 ④)와 'GREEN 에서만 리팩터링'(기준 ⑤)을 평가할 수 없다.

TIP '전략 패턴으로'가 막막하면 3장 3.9 의 할인 전략 예제(디자인 패턴 · 전략)를 다시 본다 — 같은 모양이다.

실습 · 직접 하기 Day 2 · 7교시 | 확인 실습

이어서 사용 C:\DEV\tdd-course\ship
위에서 연 폴더 그대로 · 프롬프트 ship\prompts.md

15:00–16:20

진행 순서와 확인 사항 — 개인 80분 + 짝 검토 30분

시간	할 일	남길 것
0–10분	① 요구 1~3 → 계약표 (CT · INV · OQ)	docs/계약표.md
10–20분	② 계약표 → 테스트 플랜 → 스켈레톤 코드	docs/테스트플랜.md · 5 skipped
20–40분	요구 1~3 을 하나씩 RED → GREEN	RED 캡처 2장 이상 · 5 passed
40분	요구 4 발표 → 계약표 · 플랜에 한 줄씩 추가	—
40–55분	요구 4 RED → GREEN, 전체 테스트 재실행(회귀)	요구 4 RED 캡처 · 8 passed
55–60분	골든 마스터 생성 · 승인 (리팩터링 전)	test_golden_master · 9 passed
60분	요구 5 발표 — 전략 패턴 리팩터링	—
60–75분	전략 패턴으로 리팩터링 → 전체 테스트 · 골든 마스터	리팩터링 전 · 후 pytest 결과
75–80분	추적성 매트릭스 완성	docs/추적성매트릭스.md
16:20–16:50	짝 검토 · 체크리스트 점검	체크리스트 ✓

제출물

- docs/ 3종 · tests/ · src/
- 기록지: RED 캡처 · Gemini 프롬프트 · 최종 pytest 결과

쓸 수 있는 것

- 오늘 · 어제 슬라이드와 프롬프트 템플릿
- GEMINI.md (cart/ 의 것을 복사해도 됨)
- 작업 중 짝과의 대화는 금지 — 작업 후 짝과의 검토

지금 할 일 확인: docs/계약표 · 테스트플랜 · 추적성매트릭스 · tests/ · src/ · 기록지(RED 캡처 · 프롬프트 · 최종 pytest)

확인 지표 — 5개 중 4개 이상, ①과 ④는 필수

확인 기준	확인 방법	통과 조건
① 요구사항을 실패하는 테스트로 바꾼다 (필수)	RED 캡처	구현 전에 실패한 기록 2건 이상
② AI에게 범위를 정해 요청한다	프롬프트 기록 · 테스트 파일	'이 테스트만 통과' 등 범위 지정, AI가 테스트를 고치지 않음
③ 경계값 양쪽을 테스트한다	테스트 파일	29,999 / 30,000 포함 (요구 4는 19,999 / 20,000)
④ 요구사항 변경 후 회귀를 확인한다 (필수)	최종 pytest 결과	요구 4 추가 후 기존 테스트가 모두 남아 있고 통과
⑤ GREEN 상태에서만 리팩터링한다	기록지	골든 마스터 생성 후, GREEN 에서만 전략 패턴으로 리팩터링 — 리팩터링 전후 전체 테스트 · 골든 통과 기록

실습 · 직접 하기 Day 2 · 7교시 | 확인 실습 ①

이어서 사용 C:\DEV\tdd-course\ship
위에서 연 폴더 그대로 · 프롬프트 ship\prompts.md

15:00–16:20

① 요구사항 → 계약표(Contract) — 테스트를 쓰기 전에

▶ 프롬프트 — 계약표 (읽기 전용 · 모든 코딩 에이전트 공통)

역할: 요구사항 분석가. 아래 요구사항 1~3 을 계약표로 바꿔줘. 파일은 수정하지 마.
열: 계약 ID(CT-n) | 규칙 | 사전조건(입력) | 사후조건(출력) | 오류
별도 표: 불변식(INV-n), 열린 질문(OQ-n). 새 규칙을 만들지 말고, 모호한 곳은 OQ 로만 적어줘.
(요구사항 1~3 붙여넣기)

결과 예시 응답 문구는 실행마다 다를 수 있다

계약 ID	규칙	사전조건 (입력)	사후조건 (출력)	오류
CT-1	기본 배송비 3,000원	0 < order_amount < 30,000	3,000	—
CT-2	30,000원 이상 무료	order_amount ≥ 30,000	0	—
CT-3	0원 이하는 오류	order_amount ≤ 0	—	ValueError
INV-1	배송비는 항상 0 또는 3,000	order_amount > 0	fee ∈ {0, 3000}	—
OQ-1	소수 금액이면? (예: 29,999.5)	미정	미정	사람이 결정

TIP AI 는 빈틈(OQ)을 찾아 주고, 결정은 사람이 한다. 결정한 답은 표에 한 줄 추가한다 — 오늘 예시 표(C1~C7)와 같은 원리.

지금 할 일 Gemini(plan 모드): 계약표 프롬프트 ▸ 열린 질문(OQ)은 내가 결정 ▸ docs/계약표.md 저장

실습 · 직접 하기 Day 2 · 7교시 | 확인 실습 ②

이어서 사용 C:\DEV\tdd-course\ship
위에서 연 폴더 그대로 · 프롬프트 ship\prompts.md

15:00–16:20

② 계약표 → 테스트 플랜 → 스켈레톤 코드

▶ 프롬프트 — 테스트 플랜

@docs/계약표.md 의 계약마다 테스트 케이스(TC-n)를 표로 펼쳐줘.
열: TC | 계약 | 유형(일반·경계·오류) | 입력 | 기대 결과
경계값은 바로 아래 · 정확히 · 바로 위를 짝으로. 파일은 수정하지 마.

▶ 프롬프트 — 스켈레톤 코드

@docs/테스트플랜.md 대로 tests/test_shipping.py 골격만 만들어줘.
함수마다 docstring 에 TC · CT 번호, 본문은 pytest.skip("TODO").
src/shipping.py 는 시그니처 + docstring + NotImplementedError 만.

테스트 플랜 (일부)

TC	계약	유형	입력	기대
TC-01	CT-1	일반	10,000	3,000
TC-02	CT-2	경계	29,999	3,000
TC-03	CT-2	경계	30,000	0
TC-04·05	CT-3	오류	0 · -1	ValueError

스켈레톤 → 실행

tests/test_shipping.py

```
def test_basic_fee_is_3000():
    """TC-01 · CT-1 · 기본 배송비 3,000원"""
    pytest.skip("TODO TC-01")

> pytest -q -rs
SKIPPED [1] tests\test_shipping.py:9: TODO TC-01
5 skipped in 0.01s
```

핵심 skip 은 '아직 안 씀'을 눈에 보이게 한다. 본문을 채우면 skip → RED(올바른 이유로 실패) → GREEN 으로 하나씩 바뀐다.

지금 할 일 Gemini: 테스트 플랜 프롬프트 ▸ 스켈레톤 프롬프트 ▸ pytest → 5 skipped ▸ 본문을 하나씩 채워 RED → GREEN

실습 · 직접 하기 Day 2 · 7교시 | 확인 실습 ③

이어서 사용 C:\DEV\tdd-course\ship
위에서 연 폴더 그대로 · 프롬프트 ship\prompts.md

15:00–16:20

③ 골든 마스터 → ④ 전략 패턴 리팩터링 프롬프트

▶ 프롬프트 — 골든 마스터 (리팩터링 전)

@src/shipping.py @tests/test_shipping.py @docs/계약표.md
shipping_fee() 의 '현재 동작'을 골든 마스터로 고정하는 테스트를 tests/test_golden_master.py 에 만들어줘.
- 등급 NORMAL · GOLD × 금액 -1, 0, 1, 10000, 19999, 20000, 29999, 30000, 30001, 100000 = 20개 조합
- 결과(정수) 또는 오류 종류(ValueError)를 tests/golden/shipping.golden.json 에 저장, UPDATE_GOLDEN=1 일 때만 갱신, 골든이 없으면 FAIL
- src/ 와 기존 테스트는 수정하지 마. 끝나면 !pytest -q 결과를 보여줘.

▶ 프롬프트 — 전략 패턴 리팩터링 (동작 불변)

@src/shipping.py 모든 테스트(골든 마스터 포함)가 통과하는 상태야.
동작은 바꾸지 말고 등급별 배송비 규칙을 전략 패턴으로 리팩터링해줘.
- ShippingPolicy(ABC) 에 fee(order_amount) 추상 메서드, NormalPolicy · GoldPolicy 클래스
- POLICIES 딕셔너리로 등급 → 정책을 연결하고, shipping_fee() 는 검증 후 위임만
- 한 단계씩, 매번 !pytest -q. 테스트와 골든 파일은 수정하지 마.
- 끝나면 if grade 분기가 남았는지, 새 등급을 추가하려면 무엇을 바꾸는지 한 줄로 알려줘.

결과 예시 응답 문구는 실행마다 다를 수 있다

골든 마스터 생성 · 승인

```
> pytest -q
FAILED tests/test_golden_master.py   # 골든 없음
1 failed, 8 passed

> UPDATE_GOLDEN=1 pytest -q tests/test_golden_master.py
1 passed

> pytest -q
9 passed in 0.02s
```

전략 패턴 리팩터링 후

```
✦ ShippingPolicy · NormalPolicy · GoldPolicy 로 분리했다.
> pytest -q
9 passed in 0.02s   # 전 · 후 동일 · tests/ diff 없음
```

주의 골든 마스터는 요구 4 GREEN(8 passed) 뒤, 리팩터링 앞에서만 만든다. 순서를 바꾸면 틀린 동작을 굳히거나 리팩터링 안전망이 없다.

지금 할 일 요구 4 GREEN(8 passed) ▸ 골든 마스터 프롬프트 → 9 passed ▸ 전략 패턴 프롬프트 → 9 passed 유지

이어서 사용 C:\DEV\tdd-course\ship
위에서 연 폴더 그대로 · 프롬프트 ship\prompts.md

⑤ 추적성 매트릭스와 체크리스트 점검 프롬프트

▶ 프롬프트 — 추적성 매트릭스

@docs/계약표.md @docs/테스트플랜.md @tests/ @src/ 로
docs/추적성매트릭스.md 를 만들어줘.
열: 요구 | 계약 | 테스트 케이스 | 테스트 함수 | 구현(함수·클래스) | 상태
- 테스트 함수 이름과 구현 위치는 실제 코드에서 찾아 적고, 찾지 못한 칸은 비우지 말고 '없음'으로 표시
- 요구 없이 존재하는 테스트나 코드가 있으면 표 아래에 따로 나열
- 만드는 파일은 docs/추적성매트릭스.md 하나뿐.

▶ 프롬프트 — 체크리스트 점검 (읽기 전용)

@docs/확인체크리스트.md 의 11개 항목을 현재 저장소 상태로 점검해줘. 파일은 수정하지 마.
표: 항목 | 충족 / 미충족 | 증거(파일 · 실행 명령) | 부족한 것
- pytest -q 를 실행해 통과 수를 직접 확인하고, tests/ 가 바뀌었는지 git diff 로 확인해줘
- 미충족 항목은 '무엇을 하면 충족되는지'를 한 줄로 제안만 해줘 (직접 고치지 마).

기대 산출물 — 추적성 매트릭스 (일부)

요구	계약	TC	구현
요구 1	CT-1	TC-01	NormalPolicy.fee
요구 2	CT-2	TC-02·03	NormalPolicy.fee
요구 3	CT-3	TC-04·05	shipping_fee
요구 4	CT-4·5	TC-06~08	GoldPolicy.fee
요구 5	INV	TC-09	ShippingPolicy

핵심 빈 칸 = 검증 없는 요구 또는 요구 없는 코드. 점검 결과(미충족)는 AI 가 고치게 하지 말고 내가 채운다.

TIP 체크리스트 전체: 다음 슬라이드. 짝 검토에서는 서로의 증거 파일을 열어 본다.

계약표 · 테스트 플랜 — 테스트 코드를 쓰기 전의 산출물

docs/계약표.md (요구 1~4 반영)

ID	규칙	사전조건 (입력)	사후조건	오류
CT-1	기본 배송비 3,000원	0 < order_amount < 30,000	3,000	—
CT-2	30,000원 이상 무료	order_amount ≥ 30,000	0	—
CT-3	0원 이하는 오류	order_amount ≤ 0	—	ValueError
CT-4	GOLD 는 20,000원 이상 무료	grade="GOLD" · order_amount ≥ 20,000	0	—
CT-5	grade 생략 = NORMAL (회귀)	grade 생략	NORMAL 과 동일	—
INV-1·2	배송비 ∈ {0, 3000} · 금액이 늘어도 배송비는 늘지 않는다	order_amount > 0	—	—

docs/테스트플랜.md

TC	계약	유형	입력 (금액 · 등급)	기대	테스트 함수
TC-01	CT-1	일반	10,000 · NORMAL	3,000	test_basic_fee_is_3000
TC-02 · 03	CT-2	경계 짝	29,999 · 30,000	3,000 · 0	test_free_shipping_boundary
TC-04 · 05	CT-3	오류	0 · -1	ValueError	test_non_positive_amount_raises
TC-06 · 07	CT-4	경계 짝	19,999 · 20,000 · GOLD	3,000 · 0	test_gold_free_shipping_boundary
TC-08	CT-5	회귀	20,000 · NORMAL	3,000	test_normal_member_keeps_30000_rule
TC-09	INV	골든	등급 2 × 금액 10	전후 동일	test_golden_master

핵심 요구 4 는 40분 시점에 발표 — 계약표에 CT-4·5, 플랜에 TC-06~08 이 한 줄씩 추가되는 것을 확인한다.

모범 답안 — 요구 1~3 테스트와 GREEN

① 직접 작성— tests/test_shipping.py

```python
import pytest
from shipping import shipping_fee


def test_basic_fee_is_3000():
    assert shipping_fee(10000) == 3000


@pytest.mark.parametrize("amount, expected", [
    (29999, 3000),   # 기준 바로 아래
    (30000, 0),      # 기준 정확히: 무료
])
def test_free_shipping_boundary(amount, expected):
    assert shipping_fee(amount) == expected


@pytest.mark.parametrize("amount", [0, -1])
def test_non_positive_amount_raises(amount):
    with pytest.raises(ValueError):
        shipping_fee(amount)
```

② 구현 후 실행 → GREEN

터미널

```
> pytest -q --tb=short
.....  [100%]
5 passed in 0.02s
```

핵심 요구 2는 경계 두 점(29,999 / 30,000), 요구 3은 0과 음수. 확인 판정의 ③은 이 두 줄이 있는지로 본다.

요구 4 (GOLD) — 기존 테스트가 회귀를 지켜 본다

추가 테스트 (요구 4) — TC-06 ~ 08

tests/test_shipping.py

```python
@pytest.mark.parametrize("amount, expected", [
    (19999, 3000),   # TC-06
    (20000, 0),      # TC-07
])
def test_gold_free_shipping_boundary(amount, expected):
    assert shipping_fee(amount, grade="GOLD") == expected

def test_normal_member_keeps_30000_rule():   # TC-08
    assert shipping_fee(20000, grade="NORMAL") == 3000
```

RED — 기존 5개는 통과 · 새 3개 실패 (이유: grade 인자가 없다)

터미널

```
> pytest -q --tb=short
.....FFF  [100%]
E   TypeError: shipping_fee() got an unexpected keyword
    argument 'grade'              # 3개 모두 같은 이유
FAILED …test_gold_free_shipping_boundary[19999-3000]
FAILED …test_gold_free_shipping_boundary[20000-0]
FAILED …test_normal_member_keeps_30000_rule
3 failed, 5 passed in 0.12s
```

GREEN — 요구 4 구현 (리팩터링 전)

src/shipping.py

```python
BASE_FEE = 3000
def shipping_fee(order_amount, grade="NORMAL"):
    if order_amount <= 0:
        raise ValueError("주문 금액은 0보다 커야 합니다")
    if grade == "GOLD":
        if order_amount >= 20000:
            return 0
    elif order_amount >= 30000:
        return 0
    return BASE_FEE
```

최종 GREEN

터미널

```
> pytest -q --tb=short
........  [100%]
8 passed in 0.02s
```

핵심 요구 4 뒤에도 기존 테스트 5개가 그대로 남아 통과 → 기준 ④ 충족. 이 if grade 분기가 다음 단계(요구 5)의 리팩터링 대상이다.

참고 TC-08(일반 회원은 20,000원에도 3,000원)이 '기존 규칙 유지'를 지키는 회귀 테스트다.

요구 5 준비 — 리팩터링 전에 골든 마스터를 만든다

tests/test_golden_master.py — 등급 2 × 금액 10 = 20개 조합

tests/test_golden_master.py (발췌)

```
GRADES = ["NORMAL", "GOLD"]
AMOUNTS = [-1, 0, 1, 10000, 19999, 20000,
           29999, 30000, 30001, 100000]

def capture(amount, grade):
    try:
        return {"fee": shipping_fee(amount, grade=grade)}
    except ValueError:
        return {"error": "ValueError"}

def test_golden_master():            # TC-09
    actual = {f"{g}/{a}": capture(a, g)
              for g in GRADES for a in AMOUNTS}
    # UPDATE_GOLDEN=1 일 때만 골든 파일 갱신 (발췌)
    expected = json.loads(GOLDEN.read_text())
    assert actual == expected
```

리팩터링 전에 생성 · 승인

터미널

```
> pytest -q
FAILED tests/test_golden_master.py   # 골든 없음
1 failed, 8 passed

> UPDATE_GOLDEN=1 pytest -q tests/test_golden_master.py
1 passed

> pytest -q
9 passed in 0.02s
```

골든 파일을 읽는다: GOLD/20000 → 0 · NORMAL/20000 → 3000 · NORMAL/0 → ValueError … 계약표와 일치하면 승인.

주의 골든은 8 passed(사양 테스트)가 확인된 뒤에 만든다 — 틀린 동작을 굳히지 않기 위해. 승인한 골든은 리팩터링 중 한 줄도 바뀌면 안 된다.

핵심 리팩터링 전 · 후 `pytest` 결과가 9 passed 로 같고 tests/ 가 바뀌지 않았다 → 기준 ⑤ 충족.

요구 5 — if grade 분기를 전략(Strategy)으로 리팩터링

✗ Before — 등급이 늘수록 분기가 늘어난다

src/shipping.py (요구 4 GREEN)

```
BASE_FEE = 3000


def shipping_fee(order_amount, grade="NORMAL"):
    if order_amount <= 0:
        raise ValueError("주문 금액은 0보다 커야 합니다")
    if grade == "GOLD":
        if order_amount >= 20000:
            return 0
    elif order_amount >= 30000:
        return 0
    return BASE_FEE
```

결과

```
> pytest -q   # 전 · 후 동일
9 passed in 0.02s
새 등급 SILVER → 클래스 + 등록 한 줄
shipping_fee() 수정 0줄
```

✓ After — 등급 = 전략 클래스, 함수는 위임만

src/shipping.py (전략 패턴 · import 생략)

```
BASE_FEE = 3000

class ShippingPolicy(ABC):          # Strategy
    @abstractmethod
    def fee(self, order_amount): ...

class NormalPolicy(ShippingPolicy):
    def fee(self, order_amount):
        return 0 if order_amount >= 30000 else BASE_FEE

class GoldPolicy(ShippingPolicy):
    def fee(self, order_amount):
        return 0 if order_amount >= 20000 else BASE_FEE

POLICIES = {"NORMAL": NormalPolicy(),
            "GOLD": GoldPolicy()}

def shipping_fee(order_amount, grade="NORMAL"):
    if order_amount <= 0:
        raise ValueError("주문 금액은 0보다 커야 합니다")
    return POLICIES[grade].fee(order_amount)
```

핵심 전략 패턴 = '등급별 규칙'을 객체로 분리해 분기(if)를 없앤다. 새 등급은 기존 코드를 고치지 않고 추가(OCP).

점검 tests/ 는 그대로 · 골든 마스터 그대로 · if grade == … 가 사라졌는지 · POLICIES 에 등록했는지 확인한다.

추적성 매트릭스 — 요구 ↔ 계약 ↔ 테스트 ↔ 코드, 빈 칸이 없어야 한다

요구	계약	테스트 케이스	테스트 함수	구현 (src/shipping.py)	상태
요구 1 기본 3,000원	CT-1	TC-01	test_basic_fee_is_3000	BASE_FEE · NormalPolicy.fee	GREEN
요구 2 30,000원 이상 무료	CT-2	TC-02 · 03	test_free_shipping_boundary	NormalPolicy.fee	GREEN
요구 3 0원 이하 오류	CT-3	TC-04 · 05	test_non_positive_amount_raises	shipping_fee (ValueError)	GREEN
요구 4 GOLD 20,000원 이상 무료	CT-4 · 5	TC-06 · 07 · 08	test_gold_free_shipping_boundary · test_normal_member_keeps_30000_rule	GoldPolicy.fee · POLICIES	GREEN
요구 5 전략 패턴 리팩터링	INV-1 · 2	TC-09	test_golden_master	ShippingPolicy · NormalPolicy · GoldPolicy	GREEN

점검 표의 어느 칸이라도 비면 — 테스트 없는 요구(검증 누락) 또는 요구 없는 코드(범위 초과)다. 커밋 메시지에 계약 ID 를 적어 두면 추적이 쉽다.

참고 오늘의 CT · INV · OQ 번호가 후속 과정(L5 '계약과 추적성')의 계약 ID 로 이어진다.

토론 · 짝 활동 Day 2 · 7교시 | 확인 체크리스트 16:20–16:50

확인 실습 체크리스트 — 내가 한 일을 스스로 점검한다

#	확인 항목	증거 (파일 · 명령)	✓
1	요구사항 분석 → 계약표 작성	docs/계약표.md — CT · INV · OQ	☐
2	테스트 플랜 작성	docs/테스트플랜.md — TC ↔ CT, 경계는 짝	☐
3	스켈레톤 코드 작성	src 스텁 + tests 골격 → 5 skipped	☐
4	RED — 올바른 이유로 실패	캡처 2장 이상 (NotImplementedError · TypeError)	☐
5	GREEN — 요구 1~3	5 passed	☐
6	요구 4 변경 후 회귀 확인	기존 5개 그대로 → 8 passed	☐
7	골든 마스터 테스트	리팩터링 전 생성 · 승인 → 9 passed	☐
8	리팩터링은 GREEN 에서만	전 · 후 pytest 결과 (tests/ 변경 없음)	☐
9	전략 패턴 적용 여부	if grade 분기 제거 · 등급별 클래스 · 새 등급은 클래스 추가만	☐
10	추적성 매트릭스 작성	docs/추적성매트릭스.md — 빈 칸 없음	☐
11	AI 사용 기록	프롬프트 · diff 검토 · tests/ 수정 승인 안 함	☐

지금 할 일 체크리스트를 위에서부터 ✓ ▸ 빠진 증거는 지금 채우기 ▸ 짝과 서로의 증거 파일 열어 보기

오늘 배운 것이 현업에서는 이렇게 확장된다

후속 과정에서 다루게 되는 내용

L3

레거시 코드와 테스트

- 테스트가 없는 코드부터 시작
- 골든 마스터를 레거시 규모로 확장
- Gilded Rose 카타로 연습

L4

하네스 엔지니어링

- GEMINI.md를 팀 규칙으로 확장
- 커스텀 슬래시 커맨드(/tdd-red)
- 승인 정책 · 외부 도구(MCP)

L5

계약과 추적성

- 오늘의 계약표(CT·INV·OQ) → 계약 ID (INV · E · AC)
- 추적성 매트릭스를 자동화
- 커밋 메시지에 계약 ID

핵심 오늘의 예시 표(C1~C7)·계약표(CT)가 L5의 계약 ID가 되고, 오늘의 GEMINI.md 다섯 줄이 L4의 하네스가 된다

04

CHAPTER 4 · 적용 과제

Gilded Rose — 레거시 코드 QA 프롬프팅

테스트 없는 레거시 코드에 계약을 입히고, 골든 마스터로 현재 동작을 고정한 뒤 안전하게 리팩터링하고 새 기능(Conjured)을 더한다. Gemini CLI + VS Code · Python 3 · pytest

⏱ 과정 후 적용 과제 (선택) · 부록 A · B 와 함께

실습 개요 & 아이템 규칙 (Python · Gemini CLI)

실습 개요

- 프로젝트: 기존 Gilded Rose 저장소 사용 (폴더 복사)
- 브랜치: git init → baseline 커밋 → feature/qa-prompting
- 기술 스택: Python 3 + pytest + pytest-cov + approvaltests
- 규칙 파일: GEMINI.md (프로젝트 루트, 자동 로딩)

절대 규칙 (GEMINI.md)

- Item 클래스는 절대 수정 금지
- quality 범위 0~50 (Sulfuras 제외)
- sellIn 음수 허용 (기한 초과 상태)
- 테스트 GREEN 상태에서만 리팩토링 진행

일반 아이템
매일 quality -1 | 기한 후 -2 | 최솟값 0

Aged Brie
오래될수록 quality ↑ | 기한 후 +2 | 최댓값 50

Backstage Pass
sellIn < 11 → +2, < 6 → +3 | 기한 후 quality = 0

Sulfuras
quality 항상 80 고정 | sellIn 변화 없음

Conjured ★ 신규
매일 -2 | 기한 후 -4 (일반의 2배) ← 구현할 기능

저장소 분석 — 실제 구조와 테스트 실행법

항목	실제 저장소
경로	python/ — gilded_rose.py · texttest_fixture.py · tests/
버전관리	git 저장소가 아님 → git init 필요
요구사항	GildedRoseRequirements.md · README.md
테스트	tests/test_gilded_rose.py (unittest) · test_gilded_rose_approvals.py (approvaltests)

실행 방법

```
$ cd python
$ python -m unittest            # test_foo 1건 FAIL
$ python -m unittest tests.test_gilded_rose
$ pip install pytest approvaltests && python -m pytest -q
```

저장소 분석 — 요약

- 구현은 이미 있다 — python/gilded_rose.py의 중첩 조건문
- 실행 가능한 계약은 없다 — docs/PRD.md·INV/E/AC 없음, 테스트는 실패하는 test_foo 1건과 빈 승인 파일뿐
- 의도된 규칙은 GildedRoseRequirements.md(한국어판 _kr.md)와 README에 흩어져 있다
- 빠진 기능은 Conjured 하나, 제약은 'Item 수정 금지'

A

APPENDIX A

부록 A — 테스트 안전망 직접 만들기

4장 Gilded Rose 를 에이전트 없이 손으로 따라간다. 요구사항 → 레거시 읽기 → 경계값 테스트 → 상수 정리 → 승인 테스트(골든 마스터) → Conjured TDD. 단계마다 코딩 에이전트 공통 프롬프트 예시를 붙였고, 마지막 장에 테스트 함정과 점검 포인트를 모았다.

복습 · 과제용 — pytest

Gilded Rose 요구사항과 비즈니스 규칙

레거시 코드를 고치기 전에 요구사항부터 완전히 이해한다.

배경 여관 Gilded Rose는 상품의 quality를 매일 갱신하는 시스템을 운영한다. 레거시 코드를 해치지 않고 'Conjured' 아이템을 추가하는 것이 과제다.

일반 아이템

- 매일 sellIn -1, quality -1
- 판매일이 지나면 quality 2배 감소
- quality 최솟값 0

Aged Brie

- 오래될수록 quality +1/일
- 판매일이 지나면 +2/일
- quality 최댓값 50

Sulfuras

- 전설 아이템 — 팔지 않는다
- sellIn·quality 변화 없음
- quality는 항상 80

Backstage Passes

- 11일 이상 남음: +1/일
- 10~6일: +2/일 · 5~1일: +3/일
- 콘서트가 지나면 quality = 0

Conjured ★ 신규

- 일반의 2배 속도로 감소 (-2/일)
- 판매일이 지나면 -4/일
- 이번에 TDD로 구현할 기능

경계값의 핵심

quality는 0 미만·50 초과가 될 수 없다 (Sulfuras 제외)

테스트는 이 경계의 양쪽을 먼저 겨눈다

TIP Backstage 경계는 '10일 이상'이 아니라 sellIn ≤ 10부터 +2이다 — 11과 10, 6과 5, 1과 0을 짝으로 테스트한다.

제공 폴더 C:\DEV\tdd-course\gilded-rose
Gilded Rose 카타의 python 폴더 복사 (L3 starter\gilded-rose 로 대신 가능)

요구사항 분석 프롬프트 — 문서에서 규칙표 뽑기

코드는 아직 보지 않는다. 문서만 읽고 규칙표(R-n)와 열린 질문(OQ-n)을 만든다.

▶ 프롬프트 — 요구사항 분석 (읽기 전용)

[역할] 레거시 코드 QA 분석가
[입력] GildedRoseRequirements.md · README.md (코드는 보지 마)
[작업] 아이템 종류별(일반 · Aged Brie · Sulfuras · Backstage · Conjured) 규칙을 표로 정리해줘.
열: 아이템 | 규칙 ID(R-n) | 규칙 | 경계 조건 | 출처(문서 위치)
모호하거나 서로 충돌하는 곳은 표 아래 '열린 질문(OQ-n)'으로만 적어줘.
[제약] 파일 수정 금지. 요구사항에 없는 규칙을 만들지 마.
[출력] 마크다운 표 2개 (규칙표, OQ 표)

기대 산출물 — 규칙표 (일부)

ID	아이템	규칙	경계
R-1	일반	매일 quality -1	최소 0
R-2	일반	판매일 후 -2	sell_in < 0
R-4	Aged Brie	오래될수록 +1 (판매일 후 +2)	최대 50
R-6	Sulfuras	sell_in · quality 불변	quality 80
R-7	Backstage	11일↑ +1 · 10~6일 +2 · 5~1일 +3	공연 후 0
R-8	Conjured	일반의 2배 (-2 / -4)	신규 · 미구현

열린 질문 (OQ) — 사람이 결정

ID	질문
OQ-1	Backstage: sell_in 이 0 인 날의 quality 는 0 인가, 갱신 전 값인가?
OQ-2	Sulfuras 는 quality 80 을 항상 유지하는가, 입력 그대로인가?

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

레거시 코드 이해와 문제점 파악

테스트 없이 고치지 않는다 — 먼저 읽고, 테스트로 보호막을 만든다.

python/gilded_rose.py — Item 클래스 (수정 금지 구역)

```
class Item:
    def __init__(self, name, sell_in, quality):
        self.name = name
        self.sell_in = sell_in    # 판매까지 남은 일수
        self.quality = quality    # 품질 (0~50)
```

레거시 코드의 문제점

- if 5단계 중첩 — 과정을 따라가기 어렵다
- 매직 스트링("Aged Brie" 등)이 여러 곳에 반복된다
- update_quality() 하나에 모든 규칙이 섞여 있다
- 테스트가 없어 안전하게 고칠 수 없다
- Conjured를 어디에 넣어야 할지 알 수 없다

1 이해 | **2 테스트로 보호** | **3 리팩터링** | **4 기능 추가**

순서를 거꾸로 하면 반드시 회귀 버그가 난다 — 이 부록 A는 1·2단계, 부록 B는 3·4단계를 다룬다.

참고 Item은 다른 팀 소유라는 카타의 제약 때문에 수정 금지 — 데이터(Item)와 동작(갱신 규칙)을 분리하는 근거가 된다.

레거시 update_quality() — ① 판매일 전 갱신

python/gilded_rose.py — 레거시 원본

```
class GildedRose:
    def __init__(self, items):
        self.items = items

    def update_quality(self):
        for item in self.items:
            if item.name != 'Aged Brie' and \
               item.name != 'Backstage passes to a TAFKAL80ETC concert':
                if item.quality > 0:
                    if item.name != 'Sulfuras, Hand of Ragnaros':
                        item.quality = item.quality - 1
            else:
                if item.quality < 50:
                    item.quality = item.quality + 1
                    if item.name == 'Backstage passes to a TAFKAL80ETC concert':
                        if item.sell_in < 11:
                            if item.quality < 50:
                                item.quality = item.quality + 1
                        if item.sell_in < 6:
                            if item.quality < 50:
                                item.quality = item.quality + 1
```

레거시 update_quality() — ② sellIn 감소와 판매일 후 처리

python/gilded_rose.py — 이어서 (for 루프 안)

```
            if item.name != 'Sulfuras, Hand of Ragnaros':
                item.sell_in = item.sell_in - 1

            if item.sell_in < 0:
                if item.name != 'Aged Brie':
                    if item.name != 'Backstage passes to a TAFKAL80ETC concert':
                        if item.quality > 0:
                            if item.name != 'Sulfuras, Hand of Ragnaros':
                                item.quality = item.quality - 1
                    else:
                        item.quality = item.quality - item.quality  # 0
                else:
                    if item.quality < 50:
                        item.quality = item.quality + 1
```

핵심 sellIn을 먼저 줄인 뒤 'sell_in < 0'으로 판매일 후를 판단. 리팩터링에서 감소 위치를 옮기면 이 경계가 'sell_in < 1'로 바뀐다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

레거시 코드 읽기 프롬프트 — 분기 지도와 위험 지점

코드를 고치기 전에 '무슨 일을 하는지'부터 지도로 만든다. 수정은 하지 않는다.

▶ 프롬프트 — 코드 읽기 (읽기 전용)

```
[역할] 레거시 코드 리뷰어
@gilded_rose.py 의 update_quality() 를 읽고, 수정 없이 아래를 만들어줘.
1) 분기 지도: 조건 | 하는 일 | 대응하는 규칙(R-n)
2) 규칙표에 대응하지 않는 분기는 '의도 불명'으로 표시
3) 위험 지점 5개 이내: 위치 · 이유 · 리팩터링 전에 필요한 테스트
[제약] 코드와 Item 클래스는 수정 금지. 추측은 '추정'이라고 적어줘.
[출력] 표 3개
```

기대 산출물 — 분기 지도 (일부)

조건	하는 일	규칙
name ∉ {Brie, Backstage}, quality > 0, ≠ Sulfuras	quality -1	R-1
name == Aged Brie, quality < 50	quality +1	R-4
name == Backstage, sell_in < 11 / < 6	추가 +1 / +1	R-7
name ≠ Sulfuras	sell_in -1	—
sell_in < 0 (판매일 후)	추가 -1 / +1 / 0	R-2·5·7

위험 지점 (리팩터링 전에 테스트 필요)

- sell_in 감소 위치를 옮기면 판매일 후 경계가 `< 0` → `< 1` 로 바뀐다
- Backstage 의 `quality < 50` 검사가 3곳에 흩어져 있다

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

경계값 분석과 테스트 케이스 설계

CORRECT의 Range · Boundary · Error 조건을 먼저 표로 정의한다.

아이템	경계 조건	예상 결과	우선순위
일반	quality = 0에서 더 감소하지 않음 · sellIn = 0이면 2배 감소	quality ≥ 0 · 10 → 8	★★★
Aged Brie	quality = 50에서 더 증가하지 않음 · sellIn = 0이면 +2	quality ≤ 50	★★★
Backstage	sellIn 11 → +1 · 10 → +2 · 5 → +3 · 0 → quality 0	구간 경계마다 +1/+2/+3, 콘서트 후 0	★★★
Sulfuras	sellIn·quality 불변 · quality 80 고정	갱신 후에도 동일	★★☆
Conjured	quality 1에서 -2 → 0 · sellIn 0이면 -4	quality ≥ 0, 0에서 멈춤	★★★ 미구현

핵심 경계는 항상 양쪽을 짝으로 — 11/10, 6/5, 1/0. 한쪽만 테스트하면 '≥'와 '>'를 바꾸는 실수를 잡지 못한다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

테스트 케이스 설계 프롬프트 — 경계값을 표로 먼저

테스트 코드를 쓰기 전에 '무엇을 확인할지'를 표로 합의한다 (CORRECT · Right-BICEP 관점).

▶ 프롬프트 — 테스트 케이스 설계 (읽기 전용)

[역할] 테스트 설계자
[입력] 규칙표(R-n) + @gilded_rose.py
[작업] 규칙마다 테스트 케이스를 표로 설계해줘.
  열: TC | 규칙 | 유형(일반·경계·오류) | 입력(name, sell_in, quality) | 기대 결과 | 우선순위(★)
  경계는 바로 아래 · 정확히 · 바로 위를 짝으로 (Backstage 11/10 · 6/5 · 1/0, quality 0/49/50)
[제약] 기대값은 요구사항에서만 가져와. 현재 코드의 출력을 베끼지 마. 어긋나면 OQ 로.
[출력] 표 1개 + ★★★ 케이스 목록

기대 산출물 — 테스트 케이스 표 (일부)

TC	규칙	유형	입력 (name · sell_in · quality)	기대 quality
TC-01	R-3	경계	Normal · 5 · 0	0 (더 줄지 않음)
TC-02	R-2	경계	Normal · 0 · 10	8
TC-03	R-5	경계	Aged Brie · 5 · 50	50
TC-04	R-7	경계 짝	Backstage · 11 / 10 · 20	21 / 22
TC-05	R-7	경계 짝	Backstage · 6 / 5 · 20	22 / 23
TC-06	R-7	경계 짝	Backstage · 1 / 0 · 20	23 / 0
TC-07	R-6	불변	Sulfuras · 0 · 80	80

★★★ 먼저: 경계 짝(TC-04~06) · 상·하한(TC-01·03)

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

경계값 테스트 코드 (pytest)

test_gilded_rose.py

```
# 경계값: quality는 0 미만이 되지 않는다
def test_quality_never_negative():
    items = [Item('Normal Item', 5, 0)]
    GildedRose(items).update_quality()
    assert items[0].quality == 0

# 경계값: quality는 50을 넘지 않는다
def test_aged_brie_quality_max_50():
    items = [Item('Aged Brie', 5, 50)]
    GildedRose(items).update_quality()
    assert items[0].quality == 50

# 경계값: 판매일이 지나면 2배 감소
def test_normal_item_degrades_twice_after_sell_date():
    items = [Item('Normal Item', 0, 10)]
    GildedRose(items).update_quality()
    assert items[0].quality == 8   # 10 - 2
```

Backstage 경계값 — pytest.mark.parametrize

sellIn > 10	6 ≤ sellIn ≤ 10	1 ≤ sellIn ≤ 5	sellIn = 0
+1 / 일	+2 / 일	+3 / 일	콘서트 후 → 0

← 시간이 흐를수록 sellIn 감소

test_gilded_rose.py — Backstage passes

```
import pytest

@pytest.mark.parametrize('sell_in,init_q,expected_q', [
    (15, 20, 21), (11, 20, 21),   # > 10: +1 · 경계 11
    (10, 20, 22), ( 6, 20, 22),   # +2 시작 10 · 경계 6
    ( 5, 20, 23), ( 1, 20, 23),   # +3 시작 5 · 경계 1
    ( 0, 20,  0),                 # 콘서트 후 0
    ( 5, 50, 50), ( 0, 50,  0),   # 상한 50 · 0으로
])
def test_backstage_pass_quality(sell_in, init_q, expected_q):
    items = [Item('Backstage passes to a TAFKAL80ETC concert', sell_in, init_q)]
    GildedRose(items).update_quality()
    assert items[0].quality == expected_q
```

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

테스트 코드 작성 프롬프트 — 표를 pytest 로

표(TC)가 합의되면 코딩 에이전트에게 코드를 맡긴다. 프로덕션 코드는 건드리지 않는다.

▶ 프롬프트 — 테스트 코드 작성

@docs/테스트케이스.md 의 ★★★ 케이스를 tests/test_gilded_rose.py 에 pytest 로 작성해줘.
- 규칙마다 함수 1개, 같은 규칙의 경계는 @pytest.mark.parametrize 로
- 함수 이름은 규칙을 말하게, docstring 에 TC · R 번호
- Item 클래스와 gilded_rose.py 는 수정하지 마
- 끝나면 !pytest -q 결과를 보여줘. 레거시에서 통과해야 정상(특성화) — 실패하면 먼저 기대값과 OQ 를 의심해줘

기대 결과

tests/test_gilded_rose.py

```
def test_backstage_quality_by_sell_in(
        sell_in, init_q, expected_q):
    """TC-04~06 · R-7"""
    items = [Item(BACKSTAGE_PASS, sell_in, init_q)]
    GildedRose(items).update_quality()
    assert items[0].quality == expected_q

> pytest -q
............  [100%]
12 passed in 0.03s     # 레거시에서 이미 통과
```

실패하면 먼저 의심할 것

- 코드가 아니라 기대값(표) · OQ
- '현재 코드에 맞춰' 기대값을 고치지 않는다

TIP 규칙 · 경계 · 범위를 주면 AI 가 알아서 하는 것처럼 보이지만, 기대값의 출처를 '표'로 고정하는 한 줄이 품질을 결정한다.

상수 리팩터링 — 매직 스트링을 이름 있는 상수로

✗ Before — 문자열이 곳곳에 흩어짐

```
def update_quality(self):
    for item in self.items:
        if item.name != 'Aged Brie' and \
           item.name != 'Backstage passes to …':
            if item.name != 'Sulfuras, Hand of …':
                item.quality -= 1
        # 'Aged Brie'가 여러 곳에 반복
        # → 하나만 오타가 나도 버그
```

✓ After — constants.py로 추출

```
# constants.py
AGED_BRIE = 'Aged Brie'
BACKSTAGE_PASS = ('Backstage passes to a '
                  'TAFKAL80ETC concert')
SULFURAS = 'Sulfuras, Hand of Ragnaros'
CONJURED = 'Conjured Mana Cake'
MAX_QUALITY, MIN_QUALITY = 50, 0
SULFURAS_QUALITY = 80
```

gilded_rose.py — 상수로 바꾼 뒤 (동작 불변 → 기존 테스트 전부 GREEN)

```
from constants import AGED_BRIE, BACKSTAGE_PASS, SULFURAS, MIN_QUALITY
if item.name not in (AGED_BRIE, BACKSTAGE_PASS):
    if item.name != SULFURAS and item.quality > MIN_QUALITY:
        item.quality -= 1
```

상수 정리 뒤의 확인 — 무엇이 안전망인가

test_constants.py — 보조 확인

```
import constants

def test_quality_bounds_defined():
    assert constants.MAX_QUALITY == 50
    assert constants.MIN_QUALITY == 0
    assert constants.SULFURAS_QUALITY == 80
```

이 테스트의 한계

- 상수 값만 확인한다 — 동작이 바뀌어도 통과할 수 있다
- 진짜 안전망은 A.3의 동작 테스트와 A.5의 승인 테스트

매직 스트링 → 상수 | **전체 테스트 실행** | **GREEN이면 커밋** | **다음 정리로**

- 상수 추출은 구조 변경(동작 불변) — 테스트 코드는 그대로 두고 통과만 확인.
- Python에서는 모듈 상수나 Enum으로 표현하고, VS Code의 Rename Symbol(F2)로 일괄 치환.

원칙 Tidy First — 구조 변경 커밋과 동작 변경 커밋을 나눈다. 상수 정리와 Conjured 추가가 한 커밋에 섞이면 되돌릴 수 없다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

상수 리팩터링 프롬프트 — 매직 스트링을 이름으로

첫 리팩터링은 가장 안전한 것부터: 값만 이름 붙이고 구조는 그대로.

▶ 프롬프트 — 상수 추출 (동작 불변)

@gilded_rose.py 모든 테스트가 통과하는 상태야.
동작은 바꾸지 말고, 아이템 이름 문자열과 매직 넘버(0 · 50 · 80)를
constants.py 의 이름 있는 상수로 바꾸는 것만 해줘.
- Item 클래스 · 테스트 수정 금지, 구조 변경(메서드 추출 등) 금지
- 이름 하나를 바꿀 때마다 pytest 를 실행해줘
- 끝나면 pytest -q 결과와 git diff --stat 을 보여줘

기대 결과 — constants.py

constants.py

```
# constants.py
AGED_BRIE = 'Aged Brie'
BACKSTAGE_PASS = ('Backstage passes to a '
                  'TAFKAL80ETC concert')
SULFURAS = 'Sulfuras, Hand of Ragnaros'
CONJURED = 'Conjured Mana Cake'
MAX_QUALITY, MIN_QUALITY = 50, 0
SULFURAS_QUALITY = 80
```

확인 (구조 변경 = 동작 불변)

- pytest -q 가 변경 전과 같은 결과
- tests/ · Item 클래스 diff 없음
- 상수 추출 커밋과 기능 커밋을 나눈다 (Tidy First)

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

승인 테스트(ApprovalTests) — 레거시 보호막

여러 아이템 × 30일 시뮬레이션 결과를 골든 파일로 고정한다 (골든 마스터).

1. 첫 실행 → .received.txt 생성 (approved가 없어 FAIL)
2. 사람이 결과를 검토 → .approved.txt로 승인
3. 이후 실행마다 approved와 자동 비교
4. 불일치 → FAIL, diff로 변경점 확인

터미널 · test_gilded_rose_approval.py

```
$ pip install approvaltests

from approvaltests import verify

def test_thirty_day_simulation():
    # 처음에는 approved 파일이 없어 반드시 FAIL
    # → received 파일을 읽고 승인해야 보호막이 완성된다
    verify(simulate_30_days(make_items()))
```

핵심 L3 과정 Gilded Rose 실습의 골든 마스터와 같은 원리. 승인(approved 갱신)은 언제나 사람이 diff를 읽은 뒤에 한다.

30일 시뮬레이션과 승인 테스트 코드

test_gilded_rose_approval.py

```
def make_items():
    return [
        Item('+5 Dexterity Vest', 10, 20),
        Item('Aged Brie', 2, 0),
        Item('Elixir of the Mongoose', 5, 7),
        Item('Sulfuras, Hand of Ragnaros', 0, 80),
        Item('Sulfuras, Hand of Ragnaros', -1, 80),
        Item('Backstage passes to a TAFKAL80ETC concert', 15, 20),
        # Conjured는 넣지 않는다 — 곧 동작을 바꿀 대상
    ]

def simulate_30_days(items):
    gr, lines = GildedRose(items), []
    for day in range(31):
        lines.append(f'-------- day {day} --------')
        lines += [f'{i.name}, {i.sell_in}, {i.quality}' for i in gr.items]
        if day < 30:
            gr.update_quality()
    return '\n'.join(lines)
```

주의 골든에 Conjured를 넣으면 구현하는 순간 골든이 깨진다. 변경 대상은 골든에서 빼고 명세 테스트(A.6)로만 검증.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

골든 마스터 작성 프롬프트 — 30일 시뮬레이션 고정

아이템 × 30일 결과를 통째로 저장해 두고, 이후 모든 변경과 비교한다.

▶ 프롬프트 — 승인 테스트(골든 마스터)

@gilded_rose.py @texttest_fixture.py
update_quality() 의 현재 동작을 골든 마스터로 고정하는 테스트를 만들어줘.
- 아이템 6종(+5 Dexterity Vest · Aged Brie · Elixir · Sulfuras×2 · Backstage) × 30일. Conjured 는 넣지 마 (곧 바꿀 대상)
- tests/test_gilded_rose_approval.py: approvaltests 의 verify(), 또는 도구 없이 JSON 비교
- 승인 파일은 만들어 두되, 내가 읽고 승인하기 전에는 커밋하지 마
- 같은 코드를 두 번 돌려 결과가 같은지(결정성)도 확인해줘. 코드는 수정하지 마

진행 — 승인 테스트 4단계

#	무엇을
1	첫 실행 → received 파일 생성, approved 가 없어 FAIL
2	사람이 received 를 읽는다 (day 0 · day 30 · Sulfuras 80 · Backstage 0)
3	맞는 현재 동작이면 approved 로 승인 · 커밋
4	이후 실행마다 자동 비교 — 불일치는 diff 로

주의 AI 에게 'approved 파일도 알아서 갱신해'라고 맡기지 않는다 — 승인은 사람의 일.

확인

- 같은 코드로 두 번 실행해 결과가 같다 (결정성)

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

Conjured를 TDD로 추가 — ① RED

RED — 실패 테스트 먼저 | GREEN — 최소 코드로 통과 | REFACTOR — 동작 불변 정리

test_conjured.py

```
def test_conjured_degrades_twice_as_fast():
    items = [Item('Conjured Mana Cake', 10, 20)]
    GildedRose(items).update_quality()
    assert items[0].quality == 18   # 레거시는 19 → 올바른 이유로 FAIL

def test_conjured_degrades_four_after_sell_date():
    items = [Item('Conjured Mana Cake', 0, 10)]
    GildedRose(items).update_quality()
    assert items[0].quality == 6    # 레거시는 8 → FAIL

def test_conjured_quality_never_negative():
    items = [Item('Conjured Mana Cake', 5, 1)]
    GildedRose(items).update_quality()
    assert items[0].quality == 0    # 레거시도 1-1=0 → 이미 PASS
```

주의 세 번째 테스트는 추가하자마자 통과 — '가짜 RED'로 꾸미지 말고 "기존 구현으로 이미 충족"이라고 기록

Conjured — ② GREEN과 ③ REFACTOR

gilded_rose.py — ② GREEN: 최소 코드

```
def update_quality(self):
    for item in self.items:
        if item.name == CONJURED:
            degrade_by = 2 if item.sell_in > 0 else 4
            item.quality = max(MIN_QUALITY, item.quality - degrade_by)
            item.sell_in -= 1
            continue
        ...  # 기존 로직은 그대로
```

gilded_rose.py — ③ REFACTOR: 상·하한 처리를 헬퍼로

```
def _degrade(item, amount):
    item.quality = max(MIN_QUALITY, item.quality - amount)

def _upgrade(item, amount):
    item.quality = min(MAX_QUALITY, item.quality + amount)
```

점검 점검: 승인 파일(approved)은 한 줄도 바뀌지 않아야 한다. 이 부록은 판별을 정확 일치로 단순화

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

Conjured TDD 프롬프트 — RED → GREEN → REFACTOR

한 번에 하나씩, 단계마다 따로 요청한다. 하나의 프롬프트로 세 단계를 몰아서 시키지 않는다.

▶ ① RED — 테스트만

@tests/test_conjured.py 에 Conjured Mana Cake 요구(일반의 2배: -2/일, 판매일 후 -4, 최소 0)의 테스트 3개만 작성해줘.
구현은 하지 마. 실행해서 실패 이유를 한 줄씩 설명하고, 이미 통과하는 테스트는 '이미 충족'이라고 보고해줘.

▶ ② GREEN — 최소 구현

방금 실패한 Conjured 테스트만 통과하는 최소 코드를 gilded_rose.py 에 추가해줘.
다른 아이템 동작과 승인 파일은 바뀌면 안 돼. 테스트는 수정하지 마. 끝나면 pytest -q 결과를 보여줘.

▶ ③ REFACTOR — 동작 불변 정리

모든 테스트가 통과하는 상태야. 동작은 바꾸지 말고 상·하한 처리를 _degrade / _upgrade 헬퍼로 모으는 것만 해줘.
승인 파일 diff 가 없어야 해.

각 단계의 기대 결과

단계	기대 결과
RED	2개 실패(레거시는 19 · 8), 1개는 이미 통과 → '이미 충족'으로 기록
GREEN	모든 테스트 통과 · 승인 파일 diff 없음
REFACTOR	동작 불변 — 승인 파일은 한 줄도 안 바뀐다

핵심 Conjured 만 '동작을 바꾸는' 변경이다 — 골든에서 빼 두었기 때문에 구현해도 골든은 깨지지 않는다.

테스트 함정과 점검 포인트

안전망이 '있어 보이는데' 실제로는 아무것도 지키지 못하는 일곱 가지 경우.

함정	증상	점검 포인트
가짜 RED	새 테스트가 추가 즉시 통과한다	실패 이유를 먼저 읽는다 · 이미 충족이면 그대로 기록
변경 대상을 골든에 포함	Conjured 구현 순간 골든이 깨진다	바꿀 동작은 골든에서 빼고 명세 테스트로
골든 자동 갱신	실패할 때마다 approved를 덮어쓴다	승인은 사람이 diff를 읽은 뒤에만
비결정적 출력	같은 코드인데 결과가 가끔 다르다	입력 고정 · 두 번 캡처해 diff로 결정성 확인
현재 코드에서 예상값 베끼기	명세 테스트가 버그를 정답으로 굳힌다	예상값은 요구사항에서 · 어긋나면 OQ로
값만 확인하는 테스트	상수·getter 테스트만 늘어난다	동작(입력 → 1일 후 결과)을 검증하는 테스트가 안전망
경계 한쪽만	11은 있는데 10이 없다	경계는 짝으로 — 11/10 · 6/5 · 1/0 · 0/50

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

테스트 점검 프롬프트 — 안전망의 구멍 찾기

있어 보이지만 아무것도 지키지 못하는 테스트를 읽기 전용으로 점검한다.

▶ 프롬프트 — 테스트 리뷰 (읽기 전용)

[역할] 테스트 리뷰어
@tests/ 를 아래 7가지 함정 관점으로 점검해줘. 파일은 수정하지 마.
함정: 가짜 RED · 변경 대상을 골든에 포함 · 골든 자동 갱신 · 비결정적 출력 · 현재 코드에서 예상값 베끼기 · 값만 확인하는 테스트 · 경계 한쪽만
[출력] 표: 함정 | 해당 여부 | 근거(파일:줄) | 제안 — 새 규칙은 만들지 마.

기대 산출물 — 점검 표

함정	해당	근거 · 제안
가짜 RED	아니오	TC-… 는 실패 이유가 '구현 없음'
변경 대상을 골든에 포함	아니오	approval 에 Conjured 없음
골든 자동 갱신	주의	UPDATE 옵션이 기본 켜짐? → 끈다
경계 한쪽만	예	Backstage 6 은 있고 5 가 없음 → 추가

제안은 그대로 반영하지 않는다 — 하나를 골라 사람이 결정 · 적용.

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

B

APPENDIX B

부록 B — 리팩터링 단계별 따라하기

부록 A 의 안전망 위에서 레거시 update_quality() 를 7단계로 정리하고, 새 기능(F&B)을 기존 클래스 수정 없이 추가한다. 코드 스멜 분석 → 리팩토링 계획서 → 단계별 프롬프트 순서로 진행한다.

복습 · 과제용 — 매 단계 전체 테스트

출발점 — 한 함수에 모든 규칙이 들어 있다

gilded_rose.py — 상수만 뽑은 레거시 (부록 A.4 이후)

```
def update_quality(self):
    for item in self.items:
        if item.name != AGED_BRIE and item.name != BACKSTAGE_PASS:
            if item.quality > 0:
                if item.name != SULFURAS:
                    item.quality -= 1
        else:
            if item.quality < 50:
                item.quality += 1
                if item.name == BACKSTAGE_PASS:
                    if item.sell_in < 11:
                        if item.quality < 50:
                            item.quality += 1
                    if item.sell_in < 6:
                        if item.quality < 50:
                            item.quality += 1
        # sellIn 감소 … 그리고 또 조건문 …
```

핵심 새 아이템 하나를 넣으려면 이 중첩 전체를 이해해야 한다 — 리팩터링의 목표는 '타입 하나 = 클래스 하나'이다.

리팩터링 로드맵 — 7단계

1	2	3	4	5	6	7
테스트 먼저	**코드 정리**	**조건 분리**	**메서드 추출**	**클래스 분리**	**추상화**	**기능 추가**
pytest 회귀 테스트	상수·변수 추출, 중복 조건 제거	부정 조건 반전, elif 병합	타입별 _update_*() 분리	아이템 클래스 4개로 이동	GildedRoseItem + 다형성	F&B를 새 클래스로

목표 구조

- 조건문을 다형성으로 바꾼다 (Replace Conditional with Polymorphism — Fowler)
- 처리 순서는 부모 클래스가 고정한다 (Template Method — B.9)
- 새 타입은 새 클래스 + Factory 한 줄

매 단계의 규칙

- 시작 전·후에 전체 테스트 GREEN 확인
- 구조 변경만 — 동작이 바뀌면 되돌린다
- 단계마다 커밋 (refactor: …)

TIP VS Code: Ctrl+Shift+P → Python: Run All Tests (또는 터미널에서 pytest -q) — 도구가 해 준 리팩터링도 결과는 테스트로 확인.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

코드 스멜 분석 프롬프트 — 무엇을, 왜 고칠지 먼저

리팩터링 전에 읽기 전용으로 냄새를 진단한다. '고쳐줘'가 아니라 '진단해줘'.

▶ 프롬프트 — 코드 스멜 분석 (읽기 전용)

[역할] 리팩터링 컨설턴트 (Fowler『리팩토링』 기준)
@gilded_rose.py (상수 정리 후)의 코드 스멜을 분석해줘. 수정하지 마.
[출력] 표: 위치(함수·줄) | 스멜 | 근거 | 위험도(상·중·하) | 처방(기법 · 패턴 후보) | 난이도
후보: 긴 함수 · 깊은 중첩 · 중복 코드 · 매직 넘버 · 반복되는 switch · 기능 편애
마지막에 '고치는 순서'를 의존성 순으로 제안해줘.
[제약] 이미 통과하는 테스트가 있는 범위 안에서만 제안해줘.

기대 산출물 — 스멜 표 (일부)

위치	스멜	위험	처방 · 패턴 후보
update_quality 전체	긴 함수	상	메서드 추출
if 5단계 중첩	깊은 중첩	상	Invert if · 보호절
name == … 반복	반복되는 switch	중	조건문 → 다형성 (Strategy / Template Method)
11 · 6 · 50 숫자	매직 넘버	중	이름 있는 상수
item.* 만 사용	기능 편애	중	동작을 타입 클래스로 이동

고치는 순서(제안): 상수 → 조건 정리 → 메서드 추출 → 클래스 분리 → 추상화

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

리팩토링 계획서 작성 프롬프트 — 단계 · 완료 기준 · 롤백

에이전트가 한꺼번에 바꾸지 못하도록, 계획서(문서)를 먼저 만들고 사람이 승인한다.

▶ 프롬프트 — 리팩토링 계획서 (코드 수정 금지)

위 스멜 분석을 바탕으로 docs/refactoring-plan.md 를 작성해줘. 코드는 수정하지 마.
구성: 1) 목표와 비목표 2) 안전망(테스트 · 골든 마스터 상태, 실행 명령)
3) 단계표(STEP 1…n): 단계 | 변경 내용 | 리팩토링 기법 | 완료 기준(전체 테스트 GREEN) | 롤백 방법
4) 커밋 규칙(refactor: …, 구조 변경과 기능 변경 분리) 5) 위험과 대응
한 단계는 10분 이내, 단계마다 테스트 실행 → 커밋.

기대 산출물 — docs/refactoring-plan.md 의 단계표

단계	변경	기법	완료 기준
1	회귀 테스트 보강	—	전체 GREEN
2	상수 · 변수 · 불필요 코드	Extract Constant	전체 GREEN
3	조건 분리	Invert if · elif 병합	전체 GREEN
4	타입별 메서드	Extract Method	전체 GREEN
5	타입별 클래스	Move Method	전체 GREEN
6	추상화 · Factory	다형성 · Template Method	전체 GREEN
7	새 기능 F&B	클래스 추가 (TDD)	신규 + 기존 GREEN

주의 계획서는 사람이 읽고 승인한 뒤에만 실행한다 — 이후 모든 STEP 프롬프트는 "@docs/refactoring-plan.md 의 STEP n 만"으로 시작한다.

조건문을 다형성으로 — Employee 예제

타입 코드(if/elif 분기)를 하위 클래스로 옮기면, 새 타입은 기존 코드를 고치지 않고 추가된다.

✗ Before — 타입 코드로 분기

```
class Employee:
    ENGINEER, MANAGER, INTERN = range(3)

    def __init__(self, emp_type):
        self.type = emp_type

    def get_pay(self):
        if self.type == Employee.ENGINEER:
            return 5000
        elif self.type == Employee.MANAGER:
            return 8000
        elif self.type == Employee.INTERN:
            return 2000
        raise ValueError('Unknown type')
```

✓ After — ABC + 하위 클래스 + Factory

```
from abc import ABC, abstractmethod

class Employee(ABC):
    @abstractmethod
    def get_pay(self): ...

class Engineer(Employee):
    def get_pay(self): return 5000

class Manager(Employee):
    def get_pay(self): return 8000

class Intern(Employee):
    def get_pay(self): return 2000

def create_employee(t: str) -> Employee:
    return {'ENGINEER': Engineer, 'MANAGER': Manager,
            'INTERN': Intern}[t]()
```

참고 abc.ABC + @abstractmethod는 다른 언어의 추상 클래스와 같은 역할. 이것은 '다형성'이고, 알고리즘 순서까지 부모가 쥐면 Template Method

테스트 먼저 — 회귀 테스트 ①

리팩터링 전에 안전망부터. VS Code: Ctrl+Shift+P → Python: Run All Tests · 터미널: pytest -v

test_gilded_rose.py (1/2)

```
# ① 일반 아이템 — 매일 quality -1, sellIn -1
def test_normal_item_degrades():
    items = [Item('+5 Dexterity Vest', 10, 20)]
    GildedRose(items).update_quality()
    assert items[0].sell_in == 9
    assert items[0].quality == 19

# ② 판매일이 지나면 2배 감소
def test_normal_item_degrades_twice_after_sell_in():
    items = [Item('Normal', 0, 10)]
    GildedRose(items).update_quality()
    assert items[0].quality == 8
```

TIP Backstage 경계값(sellIn 11 · 10 · 6 · 5 · 1 · 0)은 parametrize 테스트를 그대로 가져온다.

테스트 먼저 — 회귀 테스트 ②

test_gilded_rose.py (2/2)

```
# ③ quality는 음수가 되지 않는다
def test_quality_never_negative():
    items = [Item('Normal', 5, 0)]
    GildedRose(items).update_quality()
    assert items[0].quality == 0

# ④ Aged Brie — 오래될수록 quality 증가
def test_aged_brie_increases_quality():
    items = [Item('Aged Brie', 5, 10)]
    GildedRose(items).update_quality()
    assert items[0].quality == 11

# ⑤ quality는 50을 넘지 않는다
def test_quality_never_more_than_50():
    items = [Item('Aged Brie', 5, 50)]
    GildedRose(items).update_quality()
    assert items[0].quality == 50

# ⑥ Sulfuras — sellIn·quality 변화 없음
def test_sulfuras_never_changes():
    items = [Item('Sulfuras, Hand of Ragnaros', 0, 80)]
    GildedRose(items).update_quality()
    assert (items[0].sell_in, items[0].quality) == (0, 80)
```

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 1 프롬프트 — 테스트 먼저 (회귀 안전망 확인)

리팩터링 전에 안전망이 충분한지 확인하고, 모자란 경계만 테스트로 보강한다.

▶ 프롬프트 — STEP 1

@docs/refactoring-plan.md 의 STEP 1 만 수행해줘.
- 부록 A 의 경계값 · 승인 테스트를 실행해 전체가 GREEN 인지 확인
- 빠진 경계(Backstage 11/10 · 6/5 · 1/0, quality 0/50)가 있으면 테스트만 추가
- 프로덕션 코드는 수정 금지
- 결과: 전체 테스트 수 · 통과 수 · 추가한 테스트 목록

무엇을 확인하나

확인 체크

- pytest -q 전체 GREEN (시작점)
- 추가한 테스트도 레거시에서 통과
- gilded_rose.py diff 없음
- 커밋: test: 경계 보강

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

코드 정리 — 상수 추출

Extract Constant · Extract Variable · Remove Unused Code

✗ Before — 문자열 리터럴이 흩어짐

```
class GildedRose:
    def update_quality(self):
        if item.name != 'Aged Brie' and \
           item.name != 'Backstage passes...':
            if item.name != 'Sulfuras...':
                ...
```

✓ After — 상수로 추출 (오타 방지)

```
AGED_BRIE = 'Aged Brie'
BACKSTAGE_PASS = ('Backstage passes to a '
                  'TAFKAL80ETC concert')
SULFURAS = 'Sulfuras, Hand of Ragnaros'
MAX_QUALITY, MIN_QUALITY = 50, 0

def update_quality(self):
    if item.name != AGED_BRIE and \
       item.name != BACKSTAGE_PASS:
        if item.name != SULFURAS: ...
```

TIP VS Code: Pylance에는 전용 'Extract Constant' 명령이 없다 — 상수를 직접 정의한 뒤 Rename Symbol(F2)과 찾아 바꾸기로 일괄 치환

원칙 상수 이름으로 바꾸기만 하고 조건 구조는 건드리지 않는다 — 한 단계에 한 종류의 변경.

코드 정리 — 반복문과 중첩

gilded_rose.py — for-in으로 인덱스 제거

```
# ✗ Before — 인덱스로 접근
for i in range(len(self.items)):
    if self.items[i].quality > 0:
        self.items[i].quality -= 1

# ✓ After — 객체를 직접 순회 (같은 객체를 가리키므로 원본이 바뀐다)
for item in self.items:
    if item.quality > 0:
        item.quality -= 1
```

불필요한 중첩 걷어내기

```
# ✗ if item.quality < 50:
#       item.quality += 1
# ✓ item.quality = min(MAX_QUALITY, item.quality + 1)
```

참고 Python의 for item in self.items는 리스트 안 객체를 그대로 가리킨다 — item의 속성을 바꾸면 self.items의 원본이 바뀐다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 2 프롬프트 — 코드 정리 (상수 · 변수 · 불필요 코드)

구조는 그대로, 읽기 쉬운 모양으로만. 한 가지씩 요청한다.

▶ 프롬프트 — STEP 2

@docs/refactoring-plan.md 의 STEP 2 만 수행해줘. 동작은 바꾸지 마.
① 중복 조건 · 미사용 코드 제거 ② 인덱스 for → for item in self.items
한 가지씩 하고, 각각 pytest -q 를 실행해 결과를 보여줘.
메서드 추출 같은 구조 변경은 아직 하지 마. 테스트는 수정 금지.

무엇을 확인하나

확인 체크

- 각 변경 뒤 전체 GREEN
- diff 가 작고 이해 가능
- 구조 변경이 섞이지 않았다
- 커밋: refactor: 코드 정리

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

조건 분리 — 부정 조건 반전 (Invert if)

Invert if · elif 병합 · 조건 분할 · 불필요 코드 삭제

✗ Before — 이중 부정이라 읽기 어렵다

```
if item.name != AGED_BRIE and item.name != BACKSTAGE_PASS:
    if item.quality > 0:
        if item.name != SULFURAS: item.quality -= 1
else:  # Aged Brie / Backstage 로직
```

✓ After — 긍정 조건으로 먼저 갈라 놓는다

```
if item.name in (AGED_BRIE, BACKSTAGE_PASS):
    if item.quality < MAX_QUALITY: item.quality += 1
    # Backstage 추가 로직
elif item.name == SULFURAS:
    pass  # 아무것도 하지 않는다
else:
    if item.quality > MIN_QUALITY: item.quality -= 1
```

TIP VS Code: 조건문 선택 → Ctrl+. → Quick Fix 제안 확인 (없으면 직접 반전). 긍정 조건을 먼저, 부정 조건은 최소로.

조건 분리 — elif 병합과 같은 타입 조건 합치기

else + if → elif 병합

```
# ✗ Before
else:
    if item.name == SULFURAS:
        pass  # 빈 블록
    else:
        item.quality -= 1

# ✓ After
elif item.name == SULFURAS:
    pass
else:
    if item.quality > 0: item.quality -= 1
```

Aged Brie 조건 두 곳 → 한 곳

```
# ✓ After — sellIn 감소 '전'으로 모은다
if item.name == AGED_BRIE:
    if item.quality < MAX_QUALITY:
        item.quality += 1
    if item.sell_in < 1:      # 0 → 1
        if item.quality < MAX_QUALITY:
            item.quality += 1
```

경계 이동 주의 — 'sell_in < 0'이 'sell_in < 1'로 바뀌는 이유

레거시는 sellIn을 먼저 1 줄인 뒤 'sell_in < 0'을 확인. 판매일 후 처리를 감소 '앞'으로 옮기면 같은 날을 가리키는 조건은 'sell_in < 1'. 두 번째 블록을 지우는 것까지 한 번에 하고, 곧바로 테스트를 진행.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 3 프롬프트 — 조건 분리 (Invert if · elif 병합)

조건의 의미가 바뀌기 쉬운 단계 — 적용 전에 변경 계획부터 받는다.

▶ 프롬프트 — STEP 3

@docs/refactoring-plan.md 의 STEP 3 만 수행해줘.
부정 조건 반전(Invert if) · else + if → elif 병합 · 같은 타입 조건 합치기를 한 번에 하나씩.
적용 전에 '무엇을 어떻게 바꿀지' 계획을 먼저 보여주고, 내가 승인하면 적용해줘.
조건의 의미(특히 sell_in < 0 경계)는 바뀌면 안 돼. 매 단계 pytest -q.

무엇을 확인하나

확인 체크

- 승인 파일 · 경계 테스트 GREEN
- sell_in 경계가 그대로
- 한 번에 조건 하나
- 커밋: refactor: 조건 분리

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

메서드 추출 — Before: 한 함수에 모든 타입

VS Code: 코드 선택 → 우클릭 → Refactor… → Extract Method (Pylance)

gilded_rose.py — 거대한 update_quality()

```
def update_quality(self):
    for item in self.items:
        if item.name == AGED_BRIE:
            if item.quality < MAX_QUALITY: item.quality += 1
            if item.sell_in < 1:
                if item.quality < MAX_QUALITY: item.quality += 1
        elif item.name == BACKSTAGE_PASS:
            ...  # Backstage 로직
        elif item.name == SULFURAS:
            pass
        else:
            if item.quality > 0: item.quality -= 1
            if item.sell_in < 1:
                if item.quality > 0: item.quality -= 1
        if item.name != SULFURAS: item.sell_in -= 1
```

핵심 타입별 블록 하나 = 메서드 하나로 뽑고, sellIn 감소도 _update_sell_in()으로 분리.

메서드 추출 — After: 타입별 메서드

gilded_rose.py

```
def update_quality(self):
    for item in self.items:
        if   item.name == AGED_BRIE:      self._update_aged_brie(item)
        elif item.name == BACKSTAGE_PASS: self._update_backstage_pass(item)
        elif item.name == SULFURAS:       self._update_sulfuras(item)
        else:                             self._update_normal_item(item)
        self._update_sell_in(item)   # sellIn 감소를 분리

def _update_aged_brie(self, item):
    if item.quality < MAX_QUALITY: item.quality += 1
    if item.sell_in < 1 and item.quality < MAX_QUALITY: item.quality += 1

def _update_backstage_pass(self, item):
    if item.quality < MAX_QUALITY: item.quality += 1
    if item.sell_in < 11 and item.quality < MAX_QUALITY: item.quality += 1
    if item.sell_in < 6  and item.quality < MAX_QUALITY: item.quality += 1
    if item.sell_in < 1: item.quality = 0

def _update_sulfuras(self, item): pass   # 변화 없음

def _update_normal_item(self, item):
    if item.quality > 0: item.quality -= 1
    if item.sell_in < 1 and item.quality > 0: item.quality -= 1

def _update_sell_in(self, item):
    if item.name != SULFURAS: item.sell_in -= 1
```

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 4 프롬프트 — 메서드 추출 (타입별 _update_*)

VS Code 의 Extract Method 로 직접 해도 되고, 에이전트에 맡겨도 된다. 결과는 테스트가 판단.

▶ 프롬프트 — STEP 4

@docs/refactoring-plan.md 의 STEP 4 만 수행해줘.
update_quality() 의 타입별 분기를 _update_normal · _update_aged_brie · _update_backstage · _update_sulfuras 와 _update_sell_in 으로 Extract Method 해줘.
반환값과 부수효과는 그대로, 호출 순서(quality → sell_in → 판매일 후처리)도 그대로.
메서드 하나 추출할 때마다 pytest -q 를 실행해줘.

무엇을 확인하나

확인 체크

- 메서드 하나씩 GREEN
- 호출 순서 불변
- update_quality() 는 분기 + 호출만
- 커밋: refactor: 메서드 추출

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

클래스 분리 ① — 타입마다 파일 하나

VS Code: 우클릭 → Refactor… → Move Symbol to file (Pylance) · Ctrl+N 새 파일

aged_brie_item.py — 새 파일

```
class AgedBrieItem:
    def __init__(self, item):
        self._item = item            # 원본 Item을 감싼다 (Item은 수정 금지)

    def update_quality(self):
        i = self._item
        if i.quality < MAX_QUALITY: i.quality += 1
        if i.sell_in < 1 and i.quality < MAX_QUALITY: i.quality += 1
```

- 같은 방식으로 BackstagePassItem · SulfurasItem · NormalItem을 만든다 — 클래스 하나 옮길 때마다 전체 테스트
- self._item은 원본 Item 객체를 가리키므로 속성을 바꾸면 원본이 바뀐다
- 함수 사전(dict)으로 나누는 방법도 있다 — 여기서는 클래스로 나눈다

클래스 분리 ② — 사용하는 쪽

backstage_pass_item.py

```
class BackstagePassItem:
    def __init__(self, item):
        self._item = item

    def update_quality(self):
        i = self._item
        MAX = MAX_QUALITY
        if i.quality < MAX: i.quality += 1
        if i.sell_in < 11 and i.quality < MAX: i.quality += 1
        if i.sell_in < 6  and i.quality < MAX: i.quality += 1
        if i.sell_in < 1: i.quality = 0
```

gilded_rose.py — 사용부

```
def update_quality(self):
    for item in self.items:
        if item.name == AGED_BRIE:
            AgedBrieItem(item).update_quality()
        elif item.name == BACKSTAGE_PASS:
            BackstagePassItem(item).update_quality()
        elif item.name == SULFURAS:
            SulfurasItem(item).update_quality()
        else:
            NormalItem(item).update_quality()
        self._update_sell_in(item)
```

핵심 분기는 아직 남아 있다 — 다음 단계에서 공통 부모와 Factory로 한 곳에 모은다.

주의 점검: _update_sell_in() 호출을 빠뜨리면 Sulfuras를 뺀 모든 테스트가 깨진다. 호출 위치를 옮길 때는 한 번에 하나만.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 5 프롬프트 — 클래스 분리 (타입마다 파일 하나)

Item 은 다른 팀 소유 — 수정 금지. 감싸서(self._item) 쓴다.

▶ 프롬프트 — STEP 5

@docs/refactoring-plan.md 의 STEP 5 만 수행해줘.
각 _update_* 메서드를 AgedBrieItem · BackstagePassItem · SulfurasItem · NormalItem 클래스로 옮겨줘 (파일 하나씩).
Item 클래스는 수정 금지 — 각 클래스가 item 을 감싸도록(self._item).
GildedRose 는 타입에 맞는 클래스를 골라 호출만 하게. 클래스 하나 옮길 때마다 pytest -q.

무엇을 확인하나

확인 체크

- Item diff 없음
- 클래스 하나씩 GREEN
- 분기가 GildedRose 한 곳에 모임
- 커밋: refactor: 클래스 분리

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

추상화와 다형성 ① — 공통 부모 GildedRoseItem

VS Code: Refactor… → Extract superclass가 없으면 직접 작성하는 편이 확실하다.

gilded_rose_item.py — 추상 기반 클래스

```
from abc import ABC, abstractmethod

class GildedRoseItem(ABC):
    def __init__(self, item):
        self._item = item

    @abstractmethod
    def update_quality(self): ...   # 하위 클래스가 각자 구현 (다형성)
```

얻는 것

- 새 타입 = 새 하위 클래스 하나
- 호출하는 쪽은 타입을 몰라도 된다

아직 남은 것

- sellIn 감소는 여전히 GildedRose에 있다
- 처리 순서를 부모가 고정하면 Template Method (B.9 ④)

핵심 OCP — 기존 아이템 클래스를 고치지 않고 확장. 단, 새 타입을 고르는 Factory 한 곳은 바뀐다.

추상화와 다형성 ② — 각 클래스가 부모를 상속

*_item.py

```
class AgedBrieItem(GildedRoseItem):
    def update_quality(self):
        i = self._item
        if i.quality < MAX_QUALITY: i.quality += 1
        if i.sell_in < 1 and i.quality < MAX_QUALITY: i.quality += 1

class BackstagePassItem(GildedRoseItem):
    def update_quality(self): ...    # B.8과 같은 본문

class SulfurasItem(GildedRoseItem):
    def update_quality(self): pass   # 변화 없음

class NormalItem(GildedRoseItem):
    def update_quality(self):
        i = self._item
        if i.quality > 0: i.quality -= 1
        if i.sell_in < 1 and i.quality > 0: i.quality -= 1
```

추상화와 다형성 ③ — Factory로 분기를 한 곳에

gilded_rose.py

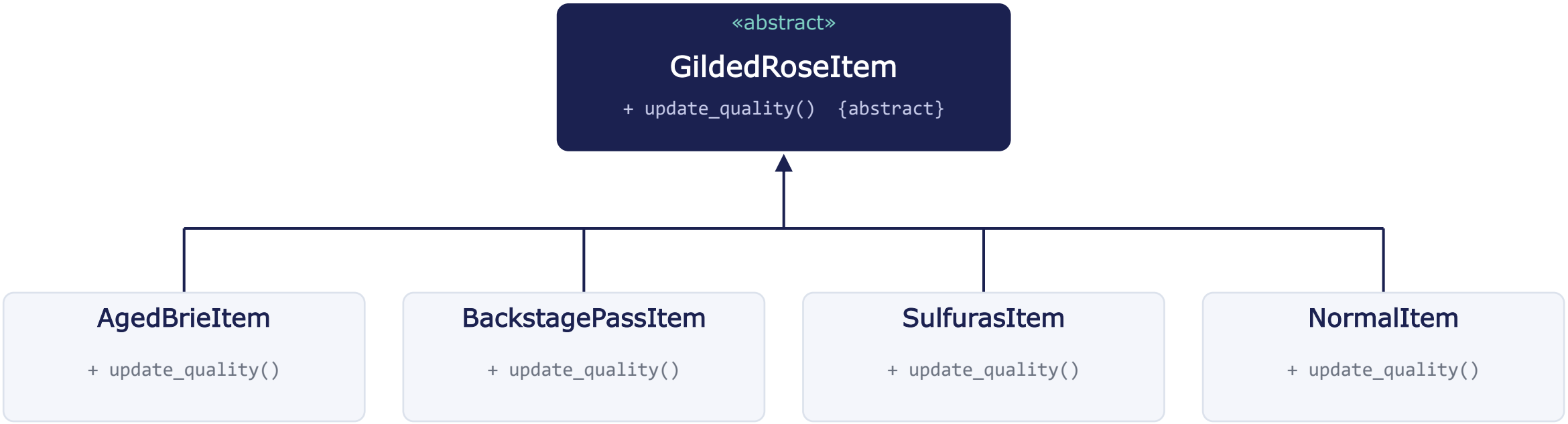
```
def create_item(item) -> GildedRoseItem:
    if   item.name == AGED_BRIE:      return AgedBrieItem(item)
    elif item.name == BACKSTAGE_PASS: return BackstagePassItem(item)
    elif item.name == SULFURAS:       return SulfurasItem(item)
    else:                             return NormalItem(item)

def update_quality(self):
    for item in self.items:
        gilded_item = create_item(item)   # Factory — 타입 판별은 여기 한 곳
        gilded_item.update_quality()      # 다형성
        self._update_sell_in(item)
```

핵심 이름으로 타입을 고르는 규칙(정확 일치 · startswith …)은 create_item 한 곳에만

원칙 구조 변경 커밋: refactor: Factory + 다형성으로 update_quality 정리 (동작 불변 · 전체 테스트 GREEN)

클래스 다이어그램 — 최종 구조



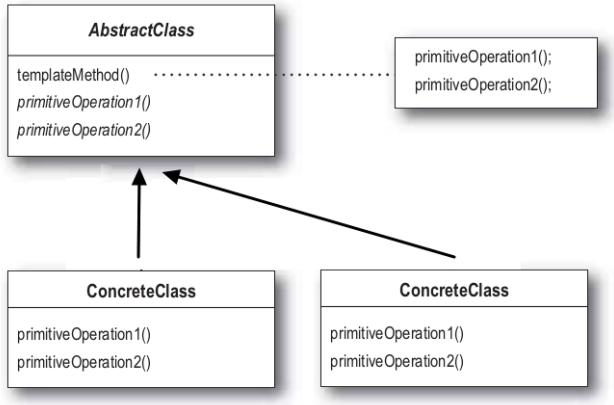
화살표 = 상속(is-a) — 네 클래스 모두 GildedRoseItem이다

핵심 새 아이템 타입 = 이 그림에 상자 하나 + create_item()에 한 줄. 기존 네 클래스는 그대로.

참고 다이어그램과 코드가 어긋나면 코드가 맞다 — 구조를 바꿀 때마다 그림도 함께 고친다.

Template Method로 완성 ④ — 순서는 부모가, 세부는 자식이

지금까지는 다형성이다. 'quality 갱신 → sellIn 감소' 순서까지 부모가 고정하면 Template Method가 된다.



templateMethod() 알고리즘 뼈대
step() 하위 클래스가 채우는 단계

```
# GildedRose.update_quality()
for item in self.items:
    create_item(item).update()
```

gilded_rose_item.py — Template Method

```
class GildedRoseItem(ABC):
    def __init__(self, item):
        self._item = item

    def update(self):                    # 템플릿 메서드: 순서를 고정
        self.update_quality()
        self.update_sell_in()

    @abstractmethod
    def update_quality(self): ...        # 필수 단계

    def update_sell_in(self):            # 기본 구현 (hook)
        self._item.sell_in -= 1

class SulfurasItem(GildedRoseItem):
    def update_quality(self): pass
    def update_sell_in(self): pass       # 필요한 타입만 재정의
```

핵심 _update_sell_in()과 'Sulfuras면 건너뛰기' 조건이 사라진다. 적용 여부는 선택 — 적용한다면 별도 refactor 커밋으로.

변수명 통일 — Rename Symbol

VS Code: F2 → Rename Symbol (Pylance가 모든 참조를 함께 바꾼다)

✗ Before — 타입마다 다른 변수명

```
if item.name == AGED_BRIE:
    agedBrieItem = AgedBrieItem(item)
    agedBrieItem.update_quality()
elif item.name == BACKSTAGE_PASS:
    backstageItem = BackstagePassItem(item)
    backstageItem.update_quality()
```

✓ After — 하나의 이름

```
for item in self.items:
    gilded_item = create_item(item)
    gilded_item.update_quality()
    self._update_sell_in(item)
```

핵심 이름은 역할 — 구체 타입(agedBrieItem)이 아니라 추상 타입(gilded_item)으로. PEP 8 snake_case도 함께 맞춘다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 6 프롬프트 ① — 추상화와 다형성 + 템플릿 메서드

공통 부모를 만들고, 처리 순서는 부모가 고정한다.

▶ 프롬프트 — 추상화 · 템플릿 메서드

@docs/refactoring-plan.md 의 STEP 6 중 '추상화'만 수행해줘.
공통 부모 GildedRoseItem(ABC)을 만들고 네 클래스가 상속하게 해줘.
필수 단계 update_quality() 는 추상 메서드로.
update() 에서 'quality 갱신 → sell_in 감소' 순서를 고정(Template Method)하고, Sulfuras 만 update_sell_in() 을 재정의해줘.
클래스 하나씩 pytest -q.

무엇을 확인하나

확인 체크

- 순서가 부모 한 곳에만 있다
- Sulfuras 만 sell_in 재정의
- 각 단계 GREEN
- 커밋: refactor: 다형성

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 6 프롬프트 ② — Factory 와 이름 통일

분기를 Factory 한 곳으로 모으고, 변수명을 통일한다 (VS Code: F2 Rename Symbol).

▶ 프롬프트 — Factory

create_item(item) Factory 함수를 만들어 타입 분기를 한 곳에 모으고, GildedRose.update_quality() 는 create_item(item).update() 만 호출하게 해줘.
동작은 그대로. pytest -q 결과를 보여줘.

▶ 프롬프트 — 이름 통일

변수명 i · item · _item 이 섞여 있어. Rename 으로만 통일해줘 (논리 변경 금지). 한 이름씩 바꾸고 pytest -q.

무엇을 확인하나

확인 체크

- 분기가 create_item 한 곳에만
- Rename 은 참조를 함께 바꾼다
- GREEN 유지
- 커밋: refactor: Factory · rename

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

새 기능 추가 — Food & Beverage (F&B)

요구사항: 일반의 2배 속도로 감소(-2/일), 판매일이 지나면 -4/일. 이름에 '[F&B]'가 들어간 아이템.

food_beverage_item.py — 새 파일 (기존 아이템 클래스는 수정 없음)

```
class FoodBeverageItem(GildedRoseItem):
    def update_quality(self):
        i = self._item
        amount = 4 if i.sell_in < 1 else 2       # 판매일 후 4, 전 2
        i.quality = max(MIN_QUALITY, i.quality - amount)
```

- 1씩 두 번 빼는 방식과 결과가 같고, 하한(0) 처리가 max() 한 줄로 드러난다
- 순서는 TDD 그대로 — 다음 장의 테스트를 먼저 쓰고(RED), 이 클래스로 통과시킨다(GREEN)

Factory 연결과 테스트

gilded_rose.py — Factory에 한 줄 추가

```
def create_item(item) -> GildedRoseItem:
    if item.name == AGED_BRIE:
        return AgedBrieItem(item)
    if item.name == BACKSTAGE_PASS:
        return BackstagePassItem(item)
    if item.name == SULFURAS:
        return SulfurasItem(item)
    if '[F&B]' in item.name:
        return FoodBeverageItem(item)  # ← 추가
    return NormalItem(item)
```

test_food_beverage.py — 먼저 작성 (RED)

```
def test_degrades_twice_as_normal():
    items = [Item('[F&B] Bread', 5, 20)]
    GildedRose(items).update_quality()
    assert items[0].quality == 18

def test_degrades_four_after_sell_in():
    items = [Item('[F&B] Milk', 0, 10)]
    GildedRose(items).update_quality()
    assert items[0].quality == 6

def test_quality_never_below_zero():
    items = [Item('[F&B] Water', 0, 1)]
    GildedRose(items).update_quality()
    assert items[0].quality == 0
```

핵심 기존 네 아이템 클래스는 한 줄도 바뀌지 않았다 — 바뀐 곳은 새 파일과 Factory 한 줄뿐 (OCP).

주의 점검: '[F&B]' in name 같은 부분 문자열 판별은 오탐 위험이 있다

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 7 프롬프트 — 새 기능 F&B (이번에는 TDD)

구조가 정리되면 기능 추가는 '클래스 하나 + Factory 한 줄'. 동작을 바꾸는 변경이므로 RED 부터.

▶ ① RED — 테스트만

이름에 '[F&B]' 가 들어간 아이템: 일반의 2배 속도로 감소(-2/일), 판매일이 지나면 -4/일, 최소 0.
tests/test_food_beverage.py 에 테스트만 작성하고 실행해 실패 이유를 설명해줘. 구현 금지.

▶ ② GREEN — 클래스 + Factory 한 줄

방금 테스트만 통과하게 FoodBeverageItem 클래스와 Factory 한 줄만 추가해줘.
기존 클래스 · 승인 파일 · 테스트는 수정 금지. 끝나면 pytest -q.

무엇을 확인하나

확인 체크

- 기존 클래스 diff 없음
- 추가한 파일 + Factory 한 줄
- 신규 + 기존 전체 GREEN
- 커밋: feat: F&B (구조 변경과 분리)

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

추가 리팩터링 — 중복 제거와 변수 인라인

STEP 8–9 · 분기마다 반복되던 호출 → 한 곳

```
# ✗ Before
if item.name == AGED_BRIE:
    gi = AgedBrieItem(item)
    gi.update_quality()
elif item.name == BACKSTAGE_PASS:
    gi = BackstagePassItem(item)
    gi.update_quality()
# … 타입마다 같은 패턴
```

```
# ✓ After — Extract Method
gi = create_item(item)
gi.update_quality()
```

STEP 10 · Inline Variable

```
# ✗ Before — 한 번 쓰고 버리는 임시 변수
gi = create_item(item)
gi.update_quality()
```

```
create_item(item).update_quality()   # ✓ After
```

- VS Code: 변수 선택 → 우클릭 → Refactor… → Inline Variable
- 디버깅에 쓰던 변수라면 남겨도 된다 — 읽기 쉬운 쪽이 정답

참고 Python은 변수를 선언 없이 대입하므로 다른 언어의 '선언 분리' 단계가 필요 없다 — 곧바로 중복 제거로 넘어간다.

원칙 매 단계 전체 테스트 GREEN → refactor 커밋. 두 단계를 한 커밋에 섞지 않는다.

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

STEP 8–10 프롬프트 — 중복 제거와 변수 인라인

마무리 정리도 한 번에 하나씩 — 반복 호출을 한 곳으로, 불필요한 임시 변수 제거.

▶ 프롬프트 — 추가 리팩터링

@docs/refactoring-plan.md 의 STEP 8~10 을 순서대로, 한 단계씩만 수행해줘.
8) 분기마다 반복되던 호출을 한 곳으로 9) 한 번만 쓰는 임시 변수 인라인
10) 남은 중복 제거
각 단계마다 변경 계획 → 적용 → pytest -q 결과를 보고해줘. 동작 · 테스트는 바꾸지 마.

무엇을 확인하나

확인 체크

- 단계마다 GREEN · 커밋
- 호출이 한 곳에 모였다
- 코드가 짧아지되 의미는 같다
- 커밋: refactor: 중복 제거

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

Before vs After — 무엇이 달라졌나

✗ Before — 레거시

```
def update_quality(self):
    for i in range(len(self.items)):
        if self.items[i].name != 'Aged Brie' \
           and self.items[i].name != 'Backstage...':
            ...  # 중첩이 계속된다
# 새 아이템? → if 블록 전체를 수정
# 테스트?   → 규칙이 얽혀 단위 테스트가 어렵다
```

✓ After — 리팩터링 결과

```
def update_quality(self):
    for item in self.items:
        create_item(item).update_quality()
        self._update_sell_in(item)

# 새 아이템? → 새 클래스 + Factory 한 줄
class ConjuredItem(GildedRoseItem): ...
```

항목	Before (레거시)	After (리팩터링)	효과
update_quality()	약 40줄 · if 5단계 중첩	본문 3줄 · 분기 없음	읽기 쉬움
새 아이템 추가	if 블록 전체 수정	새 클래스 + Factory 한 줄	OCP
테스트 단위	GildedRose 전체만	아이템 클래스별로	원인 추적 쉬움

TIP Conjured도 이제 ConjuredItem 하나로 끝난다

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

검증 프롬프트 — 레거시와 결과를 전수 비교

리팩터링이 끝난 뒤, 옛 구현과 새 구현의 결과가 같은지 조합 전체로 확인한다.

▶ 프롬프트 — 레거시 대비 검증

legacy/gilded_rose_legacy.py 와 현재 gilded_rose.py 를 비교하는 tests/test_golden_vs_legacy.py 를 만들어줘.
- 아이템 4종(일반 · Aged Brie · Sulfuras · Backstage) × sell_in -2~20 × quality 0~50 × 30일
- [F&B] 는 제외(의도한 동작 변경)
- 다르면 처음 5건을 '입력 · 레거시 결과 · 새 결과' 표로 보여줘
- 프로덕션 코드는 수정하지 마.

기대 결과

터미널

```
> pytest -q tests/test_golden_vs_legacy.py
.  [100%]
1 passed in 0.5s

# 차이가 있다면 (예)
mismatch #1: Backstage sell_in=0 q=20 → legacy 0 / new 3
```

해석

- 같으면: 동작 불변의 근거 (리팩터링 성공)
- 다르면: 첫 5건을 표로 보고 → 어느 STEP 에서 생겼는지 이분 탐색
- 의도한 변경(F&B)은 비교 대상에서 제외

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

리팩터링 함정과 점검 포인트

리팩터링 = 동작을 바꾸지 않는 구조 변경. 그 약속이 깨지는 일곱 가지 경우.

함정	증상	점검 포인트
안전망 없이 시작	고친 뒤에야 무엇이 깨졌는지 모른다	Step 1 전에 전체 테스트 GREEN · 골든(승인) 포함
구조 + 동작을 한 커밋에	되돌리면 새 기능까지 사라진다	refactor 커밋과 feat 커밋을 분리 (Tidy First)
경계 조건 이동	판매일 당일 결과만 틀린다	sellIn 감소 위치를 옮기면 '< 0' → '< 1' 확인
호출 누락	클래스 분리 후 sellIn이 줄지 않는다	옮길 때마다 전체 테스트 · Sulfuras 케이스 포함
판별 규칙이 여러 곳	타입 판별 결과가 곳곳에서 다르다	이름 → 타입 판별은 create_item 한 곳에만
도구를 믿고 확인 생략	자동 리팩터링이 참조 하나를 놓친다	Pylance 결과도 테스트로 확인
OCP 과신	'아무것도 안 바꿨다'는 설명이 사실과 다르다	Factory는 바뀐다 — 기존 클래스가 그대로인지 확인

이어서 사용 C:\DEV\tdd-course\gilded-rose
위에서 연 폴더 그대로 · 프롬프트 gilded-rose\prompts.md

최종 리뷰 프롬프트 — PR 설명과 함정 점검

리팩터링 전체의 근거를 한 문서로 정리하고, 7가지 함정에 걸리지 않았는지 점검한다.

▶ 프롬프트 — 최종 리뷰 (코드 수정 금지)

git diff main 을 읽고 PR 설명을 작성해줘. 코드는 수정하지 마.
구성: 변경 요약 · 단계별 커밋 목록 · 동작 불변의 근거(테스트 수 · 골든 · 레거시 비교) · 위험 · 남은 스멜
그다음 '리팩터링 함정 7가지' 표로 점검해줘: 함정 | 해당 여부 | 근거 | 제안

기대 산출물 — PR 설명 (요약)

PR 설명

```
## 변경 요약
- update_quality() 를 타입별 클래스 + Factory 로 분리
## 동작 불변 근거
- 테스트 20 passed · 골든(30일) 동일 · 레거시 비교 동일
## 단계별 커밋
- refactor: 상수 · 조건 · 메서드 · 클래스 …
## 위험
- Item 은 수정하지 않음
## 남은 스멜
- Backstage 의 상한 검사 중복
```

함정 7가지 점검

- 테스트 없이 시작 · 구조/기능 커밋 혼합 · 큰 한 번에 · …
- (슬라이드 B.13 의 표를 그대로 점검)

TIP 모든 코딩 에이전트 공통 — 본문은 그대로, 파일 첨부 표기만 도구에 맞춘다 (Gemini CLI · Claude Code: @파일, Copilot Chat: #file:파일, 그 밖: 파일 경로를 본문에 적는다).

C

APPENDIX C

부록 C — 속성 테스트 맛보기

예제 몇 개가 아니라 '항상 성립해야 하는 성질'을 수백 개의 무작위 입력으로 검증한다. Python hypothesis 라이브러리로 calc 와 장바구니 불변식을 시험하고, 반례가 사양의 빈틈을 알려 주는 장면을 본다.

선택 · 심화 — pip install hypothesis

설명 · 듣고 이해 부록 C · C.1

속성 테스트(Property-based Testing) — 예제가 아니라 성질을 검증한다

입력을 사람이 고르지 않고 도구가 무작위로 만들어 '항상 성립해야 하는 성질'을 수백 번 확인한다.

1 성질을 말로 정한다
예) 결제 금액은 0 이상, 합계 이하

2 입력 생성 규칙을 쓴다
hypothesis 의 strategies (정수 · 리스트 …)

3 도구가 수백 번 실행
기본 100개 입력을 만들어 assert

4 반례를 찾으면 축소
가장 작은 실패 입력을 보여 준다 (shrink)

	예제 기반 테스트 (오늘 R1~R7 · C1~C7)	속성 기반 테스트 (hypothesis)
입력	사람이 고른 점 몇 개	도구가 만든 수백 개 (경계 근처를 우선 시도)
기대값	구체적인 값 (== 51300)	성질 (0 ≤ 결제 ≤ 합계)
잘하는 것	사양을 못 박기 · 읽기 쉬움	생각 못 한 입력 · 불변식 · 계약 검증
실패 시	고정된 입력으로 재현	반례를 축소해 보여 주고 다음 실행에도 기억

TIP 설치: pip install hypothesis · 실행: 평소처럼 pytest -q (예제: 교안예제/property_based/)

핵심 속성 테스트는 예제 테스트를 대체하지 않는다 — 사양(예제 표)을 못 박은 뒤, 그 위에서 불변식을 넓게 감시한다.

calc 의 add() 에 속성 테스트 — 성질 네 가지

tests/test_add_property.py (발췌)

```
DELIMS = st.sampled_from([",", ";"])
NUMBERS = st.lists(st.integers(min_value=0, max_value=10**6), min_size=1, max_size=20)
@given(NUMBERS, st.data())
def test_sum_matches_for_any_delimiters(numbers, data):       # P1 합계 성질
    delims = [data.draw(DELIMS) for _ in numbers[1:]]
    text = str(numbers[0]) + "".join(d + str(n) for d, n in zip(delims, numbers[1:]))
    assert add(text) == sum(numbers)
@given(st.text(alphabet=" \t\n", max_size=10))
def test_blank_text_is_zero(text):                            # P2 공백만 → 0
    assert add(text) == 0
@given(NUMBERS, st.integers(min_value=1, max_value=10**6))
def test_any_negative_number_raises(numbers, negative):       # P3 음수가 하나라도 있으면 오류
    text = ",".join(map(str, numbers + [-negative]))
    with pytest.raises(ValueError):
        add(text)
@given(NUMBERS, st.randoms())
def test_order_does_not_matter(numbers, rnd):                 # P4 순서 무관
    shuffled = numbers[:]
    rnd.shuffle(shuffled)
    assert add(",".join(map(str, numbers))) == add(",".join(map(str, shuffled)))
```

P	성질
P1	어떤 구분자 조합이든 합계 = 숫자들의 합
P2	공백만 있으면 항상 0
P3	음수가 하나라도 있으면 ValueError
P4	숫자 순서를 섞어도 같은 결과

터미널

```
> pytest -q
....  [100%]
4 passed in 3.6s

# 각 테스트가 무작위 입력 100개씩 사용
```

참고 Day 1 의 R7(공백만 → 0)이 P2 로 '모든 공백 조합'으로 넓어졌다 — 예제 하나가 성질이 되는 순간.

설명 · 듣고 이해 부록 C · C.3

장바구니 불변식과 '틀린 가정' — 반례가 사양을 일깨운다

test_cart_property.py (발췌) — 불변식

```
AMOUNT_ITEM = st.fixed_dictionaries(
    {"price": st.integers(0, 1_000_000),
     "qty": st.integers(0, 20)})
BASKET = st.lists(AMOUNT_ITEM, max_size=10)

@given(BASKET, st.booleans())
def test_total_between_zero_and_subtotal(items, is_vip):
    assert 0 <= final_total(items, is_vip) <= subtotal(items)

@given(BASKET)
def test_vip_never_pays_more(items):
    assert final_total(items, True) <= final_total(items, False)
```

'더 사면 더 낸다' — 틀린 가정

```
AMOUNT = st.integers(40_000, 60_000)   # 경계 근처

@given(AMOUNT, AMOUNT)
def test_larger_subtotal_never_pays_less(a, b):
    low, high = sorted((a, b))
    assert (apply_threshold_discount(low)
            <= apply_threshold_discount(high))
```

hypothesis 가 찾은 반례

```
> pytest tests/counterexample_demo.py -q
E   assert 45001 <= 45000
E    +  where 45001 = apply_threshold_discount(45001)
E    +  and   45000 = apply_threshold_discount(50000)
E   Failing test case:
E       a=45001,
E       b=50000,
1 failed
```

배운 것

- 45,001원이 50,000원보다 더 낸다 — C3 (사양) 때문
- 생성 범위를 경계(40,000~60,000) 근처로 좁히니 바로 발견
- 범위를 0~1,000,000 으로 넓히면 못 찾는다 → 생성 범위도 사양

원칙 틀린 가정은 xfail(strict) 로 남겨 '이 성질은 성립하지 않는다'를 문서화한다.

프롬프트 예시 부록 C · C.4

이어서 사용 C:\DEV\tdd-course\cart
Day 2 에서 쓴 cart 폴더 · @src/cart.py @docs/예시표.md

AI 에게 속성 테스트 시키기 — 프롬프트와 함정

▶ 프롬프트 — 속성 테스트 (모든 코딩 에이전트 공통)

@src/cart.py @docs/예시표.md
예시 표 C6 과 같은 '불변식'을 hypothesis 로 tests/test_cart_property.py 에 작성해줘.
- 입력: price 0~1,000,000, qty 0~20 의 항목 리스트(최대 10개) × VIP 여부
- 성질: 0 ≤ 결제 ≤ 합계, VIP ≤ 일반
- 기대값을 구현 코드로 다시 계산하지 마 (성질만 assert)
- 반례가 나오면 고치지 말고 입력과 함께 보고해줘. src/ 는 수정 금지.

함정	대응
생성 범위가 너무 넓다	경계 근처 전략을 따로 추가
재현이 안 된다	@seed(…) · --hypothesis-seed 로 고정
느리다	@settings(max_examples=…) 로 조절
기대값을 구현으로 계산	동어반복 — 성질만 쓴다
예제 테스트를 지운다	예제(사양)는 남기고 속성은 추가

주의 AI 가 반례를 '테스트를 고쳐서' 없애지 못하게 — 반례는 사양을 다시 묻는 질문이다 (사람이 결정).

핵심 예제 표(사양) → 예제 테스트 → 골든 마스터 → 속성 테스트: 오늘 배운 안전망의 층을 순서대로 쌓는다.

토론 · 짝 활동 Day 2 · 회고

17:00–17:30

KPT 회고와 현업 적용 액션 아이템

Keep — 계속할 것	Problem — 어려웠던 것	Try — 시도할 것
예) 테스트 이름을 문장으로 쓴다	예) 사양이 애매할 때 질문을 못 떠올렸다	예) 다음 기능 하나를 TDD로 해 본다

현업 적용 액션 아이템 — 한 사람에 하나

다음 주 작업 중 함수 하나를 테스트 먼저로	팀 저장소에 GEMINI.md(또는 AI 규칙 파일) 다섯 줄 제안	AI가 바꾼 diff에서 tests/ 수정을 항상 확인

CLOSING

AI로 코드가 빠르게 만들어질수록 사람이 모든 동작을 일일이 확인하기는 어렵다. 따라서 코드 변경으로 기존 기능에 문제가 생기지 않았는지 자동화된 테스트로 검증해야 한다.

요구사항을 테스트로 바꾸고, AI에게는 그 테스트만 통과시키게 하고, 바뀔 때마다 전부 다시 확인한다

테스트는 사람이 · 구현은 AI가

tests/ 수정은 거절

경계는 짝으로

바뀌면 전부 다시 실행

2026 · 김대경 Ph.D.